محرمانه – داخلی MCINEXT

Phoenix — پرونده فنی و مهندسی

IVA Voice Agent · AIVA · Knowledge Base · CCA · AQA

Flow، Architecture، System Designها، کلاس‌ها و فایل‌های کلیدی، Config، Data Model

Deploy و Cookbook توسعه

مخاطب	توسعه‌دهندگان، سرپرستان فنی، CTO، و TPM هنگام Review یا برآورد
مبنا	خوانش مستقیم کد مخزن phoenix؛ ۱۰ پروژه Python فعال (~۹۸ هزار خط) + سرویس‌های Go/Rust پیرامونی
سند همراه مدیریت محصول	`product-management/PRODUCT_DOSSIER_FA.pdf` — فصل‌های ۱ تا ۱۰ و ۲۵ (Fit-Gap، ریسک، Scope، رقبا). شماره فصل‌ها در هر دو سند ثابت مانده تا ارجاع‌هایی مثل «§۵» یا «PRV-03» معتبر بمانند.
تاریخ	بازبینی ۶ :Code مهر ۱۴۰۵ — 2026-09-28 · تفکیک اسناد: ۱۳ مهر ۱۴۰۵

فهرست کامل ←

منو — برای رفتن به هر بخش کلیک کنید

مهندسی

راهنمای منابع — ریشه هر ادعا | مفاهیم پایه Voice AI | مسیر تماس: از شماره‌گیری تا Agent

۱۱. نقشه Repository و Tech Stack | فلسفه Tech Stack و فرمت‌های صوتی

۱۲. Architecture از ابتدا تا انتها | ۱۳. agentlib — هسته Framework | ۱۴. AIVA در عمق

۱۵. IVA Voice Agent در عمق | ۱۶. CCA Pipeline در عمق | ۱۷. AQA در عمق

۱۸. Authentication و Authorization | ۱۹. Data Model و Messaging | ۲۰. Configuration Reference

۲۱. Build، CI/CD و Deployment | ۲۲. Developer Cookbook | ۲۳. کیفیت Code و Technical Debt

۲۴. پیوست — Endpoint Catalog | فهرست اختصارات | واژه‌نامه

فهرست

- راهنمای منابع — ریشه هر ادعا

• مفاهیم پایه Voice AI: VAD، Turn، STT/ASR، TTS، Egress

• مسیر تماس: از شماره‌گیری تا IVA Agent (سرشماره، SIP/RTP، SBC، Backbone، نقطه شروع Pipeline، تست و عیب‌یابی)

۱۱. نقشه Repository و Tech Stack

• فلسفه Tech Stack: چرا Python/Go/Rust، هر Database و سرویس، فرمت‌های صوتی Mono/Stereo و قابلیت‌های استفاده‌نشده

۱۲. Architecture از ابتدا تا انتها

۱۳. agentlib — هسته Framework

۱۴. AIVA در عمق: Event-driven Orchestration

۱۵. IVA Voice Agent در عمق

۱۶. CCA Pipeline در عمق

۱۷. AQA در عمق

۱۸. Authentication و Authorization

۱۹. Data Model و Messaging

۲۰. Configuration Reference

۲۱. Build، CI/CD و Deployment

۲۲. Developer Cookbook: کجا را عوض کنم؟

۲۳. کیفیت Code و Technical Debt

۲۴. پیوست: Endpoint Catalog، فهرست اختصارات و واژه‌نامه

۲۵. به‌روزرسانی کد: ۲۸ شهریور تا ۱۴ مهر ۱۴۰۵

فصل‌های ۱ تا ۱۰ و ۲۵ (کسب‌وکار، Fit-Gap، رقبا) در پرونده مدیریت محصول هستند؛ ردیف‌هایی مثل PRV-03 یا SEG-07 به جدول Fit-Gap آن سند و `FIT_GAP_TRACKER.xlsx` اشاره دارند.

قرارداد نام‌گذاری در این سند: نام Componentها، Branchها، Eventها، مقادیر و اصطلاحات فنی به همان شکل اصلی در Code (انگلیسی) آمده‌اند؛ مثلاً «Branch `clarifying`» وقتی برنده می‌شود که Classifier مقدار `clarifying` برگرداند». عبارات توافق‌نامه هم با همان واژه‌های Fit-Gap (مثل Whitelist، Retention، Transcript) آمده‌اند.

راهنمای منابع — ریشه هر ادعا

زیر هر ادعا یک خط منبع آمده که نشان می‌دهد آن ادعا از کجا آمده است. **همه مسیرها نسبت به ریشه مخزن `phoenix/` هستند** و شماره بعد از `:` شماره خط است.

برچسب	معنا	مثال
CODE	مستقیماً در فایل خوانده شده؛ قابل باز کردن و بررسی.	`python/coa-api/coa/apps/files/views.py:19`
GIT	از تاریخچه Git (Commit hash یا `git log`).	`752f89333 2025-07-13`
DOC	از README یا مستند Markdown داخل مخزن.	`python/aiva/docs/RequestFlow.md:68`
BOARD	از سه عکس وایت‌بورد (Mapper، IVA (Intelligent Virtual Assistant)، COA) — طراحی، نه لزوماً پیاده‌سازی.	board IVA (Intelligent Virtual Assistant)
FIT-GAP	از فایل توافق `HR - meet UAT agreement - Fit-Gap MVP` (کپی در `product/source/fitgap_part*.tsv`).	TEC-14
INFERENCE	استنتاج من از شواهد ذکرشده — مستقیماً در Code دیده نشده. **پیش از تعهد در جلسه تأیید شود.**	«coa-ai در ClickHouse می‌نویسد»
TELECOM	دانش استاندارد مخابرات/VoIP (SIP (Session Initiation Protocol)، RTP (Real-time Transport Protocol)، SBC (Session Border Controller)، Codec، اعداد رایج) — در این مخزن نیست و توپولوژی واقعی اپراتور نیست؛ با تیم Network/VoIP تأیید شود.	«G.711 ≈ 87 kbps در هر جهت»
WEB	وب‌سایت رسمی شرکت، Press Release یا رسانه تخصصی (فقط در فصل ۲۵، تحلیل رقبا)؛ لینک‌ها در ۲۵.۹. ادعای خود شرکت‌هاست — پیش از استناد در مذاکره دوباره چک شود.	«NAVEX Nira for Intake، سپتامبر ۲۰۲۶»
BLAME	زیر هر `path:line`: **چه کسی آن خط را Commit کرده** — نام، تاریخ، Hash و پیام Commit (از `git blame -w`). توجه: آخرین کسی که آن خط را تغییر داده؛ اگر Refactor بوده، نویسنده اصلی Feature ممکن است فرد دیگری باشد (`git log -L` تاریخچه کامل را می‌دهد).	`views.py:19 → name · date · sha`
ADDED	برای فایل بدون شماره خط: کسی که فایل را ساخته (اولین Commit).	`sms.py → name · …`
LAST	برای پوشه: آخرین Commit روی آن پوشه.	`internal/triton/ → …`

برخی افراد با دو نام Git دیده می‌شوند (مثلاً `Hamed Mahmoudkhani` و `h.mahmoudkhani`، یا `Mohammad Ghazanfari` و `m.ghazanfari`) — به احتمال زیاد یک نفر با دو تنظیم Git متفاوت `INFERENCE`.

«`grep … → 0`» یعنی جست‌وجوی آن الگو در پوشه‌های نام‌برده هیچ نتیجه‌ای نداشت؛ این شاهدِ *نبودن* است. شاهد جزئی هر ردیف توافق در ستون «شاهد در کد» فایل `FIT_GAP_TRACKER.xlsx` و در `product/source/verdicts.tsv` است.

مهندسی

مهندسی

System Design، Architecture، Flow و Code — در حدی که بتوانید Code را بخوانید، Review کنید، تغییر کوچک بدهید و برآورد تیم را به چالش بکشید.

مفاهیم پایه Voice AI (Artificial Intelligence) — پیش‌نیاز بخش مهندسی

پنج مفهوم زیر ستون فقرات هر Voice Agent هستند. برای هر کدام: تعریف، و اینکه **در Phoenix کجا و چطور** پیاده شده است.

Voice Activity Detection — VAD (Voice Activity Detection)

تشخیص اینکه در جریان صدا **واقعاً کسی حرف می‌زند** یا فقط سکوت/نویز است. از آن برای شروع و توقف پردازش، فرستادن صدا به STT (Speech-to-Text)، و فهمیدن اینکه کاربر حرفش را تمام کرده استفاده می‌شود.

در Phoenix	جزئیات
Voice Agent (Real-time)	مدل **Silero** VAD (Voice Activity Detection) در هر Worker یک بار Load می‌شود؛ پارامترها: `AGENT_VAD_MIN_SPEECH_DURATION=0.5s` ، `AGENT_VAD_MIN_SILENCE_DURATION=1.0s` ، `AGENT_VAD_ACTIVATION_THRESHOLD=0.5` ، ۲ Prefix Padding ثانیه. `CODE` python/chatbot-iva-agent/src/main.py:89 · python/chatbot-iva-agent/src/config.py:62-67 `BLAME` main.py:89 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) `BLAME` config.py:62 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)
سرویس مستقل VAD (Voice Activity Detection)	`rust/vad` — VAD (Voice Activity Detection) با gRPC Streaming. `GIT` 'feat(vad): grpc streaming v2.0.0' 2026-09-26
Analytics (ضبط استریو)	`voice-segmenter` — دو کانال (مشتری/کارشناس) را جدا و VAD (Voice Activity Detection) موازی روی هر کانال اجرا می‌کند؛ پایه تشخیص Overlap و Silent Agent. `DOC` golang/voice-segmenter/README.md `CODE` python/coa-api/coa/apps/main/models.py:400,417 `INFERENCE` link between segmenter output and overlap/silent flags `BLAME` models.py:400 → a.rahimi · 2026-05-09 · 1935a6396 feat(coa-api): add new tags from asr result v4.23.0 `BLAME` models.py:417 → a.rahimi · 2026-06-14 · 4f59fbc07 fieat(coa-api): add new tag for agent silent during calls v4.35.0

Turn — نوبت گفت‌وگو و Turn-taking

Turn یک واحد از تبادل گفت‌وگوست: نوبت کاربر یا نوبت دستیار. منطق **Turn-taking** تصمیم می‌گیرد کی سیستم گوش‌دادن را متوقف و شروع به پاسخ کند، یا برعکس — عامل اصلی طبیعی بودن مکالمه صوتی.

در Phoenix	جزئیات
Endpointing	پس از پایان گفتار، سیستم بین `0.5` تا `3.0` ثانیه صبر می‌کند تا مطمئن شود نوبت تمام شده؛ حالت `fixed` یا تطبیقی. **CODE** python/chatbot-iva-agent/src/config.py:69-73 **BLAME** config.py:69 → y.fallah · 2026-08-03 · f36e6cc2f feat(chatbot-iva-agent): add openai turn detector (v0.30.0)
Turn Detection	سه حالت: `deactive` (فقط سکوت VAD (Voice Activity Detection))، `mci` (مدل اختصاصی همراه اول)، `openai` (یک LLM (Large Language Model) احتمال «تمام‌شدن نوبت» را ۰ تا ۱ می‌دهد؛ آستانه `0.5`، با ۶ نوبت تاریخچه). **CODE** python/chatbot-iva-agent/src/config.py:82-110 · python/chatbot-iva-agent/src/utils/mci_turn_detector.py · python/chatbot-iva-agent/src/utils/openai_turn_detector.py **BLAME** config.py:82 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat... **ADDED** mci_turn_detector.py → y.fallah · 2025-06-21 · 6213f04ac feat(chatbot-iva): add turn-detection. v0.6.0 **ADDED** openai_turn_detector.py → y.fallah · 2026-08-03 · f36e6cc2f feat(chatbot-iva-agent): add openai turn detector (v0.30.0)
Barge-in	اگر کاربر وسط صحبت Agent حرف بزند (بیش از `0.4s`)، پخش TTS (Text-to-Speech) قطع و نوبت به کاربر داده می‌شود؛ پیام قطع‌شده با `interrupted=true` ثبت می‌شود. **CODE** python/chatbot-iva-agent/src/config.py:68 · python/chatbot-iva-agent/src/schema.py:22 (interrupted) **BLAME** config.py:68 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) **BLAME** schema.py:22 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...
سکوت طولانی	User-away: پس از ۱۰ ثانیه سکوت، Agent خودش نوبت را می‌گیرد («من هستم اگه کاری داشتی»). **CODE** python/chatbot-iva-agent/src/config.py:401-427 · python/chatbot-iva-agent/src/main.py:197 **BLAME** config.py:401 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0) **BLAME** main.py:197 → y.fallah · 2026-06-03 · 1fb743b0b feat(chatbot-iva-agent): add support for phone number feed in creat...
Turn در مغز AIVA (Agentic Intelligent Virtual Assistant)	هر Turn کاربر یک `UserInputEvent` است که Branchهای موازی را راه می‌اندازد (بخش ۱۴). **DOC** python/aiva/docs/RequestFlow.md:16

Speech-to-Text — STT (Speech-to-Text) (ASR (Automatic Speech Recognition))

تبدیل صدای گفتار به متن تا بتوان آن را به LLM (Large Language Model) یا منطق کسب‌وکار داد. نام دیگرش **ASR (Automatic Speech Recognition)** است.

در Phoenix	جزئیات
Real-time (هر Turn)	کلاینت OpenAI-compatible با ۳ Retry و ۱۰ Timeout ثانیه؛ خروجی هر جمله همراه `transcript_confidence`. **CODE** python/chatbot-iva-agent/src/config.py:29-31,364 · python/chatbot-iva-agent/src/schema.py:26 **BLAME** config.py:29 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) **BLAME** config.py:364 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) **BLAME** schema.py:26 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...
Offline (تماس ضبط‌شده)	`grogu` فایل کامل تماس را برای ASR (Automatic Speech Recognition) می‌فرستد، پشت Circuit Breaker. **CODE** golang/grogu/internal/asr/service.go:58,85 **BLAME** service.go:58 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0) **BLAME** service.go:85 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)
API (Application Programming Interface) برای چت	`POST /api/v1/chat/asr` در `adcs` (WAV (Waveform Audio File Format) → متن). **CODE** python/adcs/adcs/chat/api.py:17 **DOC** python/adcs/README.md **BLAME** api.py:17 → s.hajili · 2026-08-19 · 0bd0127f3 feat(adcs): add tts asr
مدل	مدل‌های TTS/ASR (Automatic Speech Recognition) روی Triton و از طریق `neurobox-gateway` ارائه می‌شوند. **CODE** golang/neurobox-gateway/internal/triton/client.go **INFERENCE** that the IVA agent's STT_BASE_URL points at neurobox-gateway in production **ADDED** client.go → a.salimi · 2025-12-20 · 47989aa49 feat(neurobox-gateway): initialize project (v0.1.0)

Text-to-Speech — TTS (Text-to-Speech)

تبدیل متن (مثلاً پاسخ LLM (Large Language Model)) به صدای گفتاری برای پخش به کاربر.

در Phoenix	جزئیات
Sentence Streaming	پاسخ LLM (Large Language Model) جمله‌به‌جمله (جداکننده `". "`) به TTS (Text-to-Speech) می‌رود تا اولین جمله قبل از کامل‌شدن کل پاسخ پخش شود. **CODE** python/chatbot-iva-agent/src/config.py:164 · python/chatbot-iva-agent/src/utils/sentence_tokenizer.py **BLAME** config.py:164 → y.fallah · 2026-08-03 · f36e6cc2f feat(chatbot-iva-agent): add openai turn detector (v0.30.0) **ADDED** sentence_tokenizer.py → y.fallah · 2025-06-21 · 6213f04ac feat(chatbot-iva): add turn-detection. v0.6.0

در Phoenix	جزئیات
Resilience	۳ Retry، Timeout ۱۰ ثانیه؛ Status `204` خطای Non-retryable. `CODE` python/chatbot-iva-agent/src/config.py:36-43,382 `GIT` 'fix(chatbot-iva-agent): add tts non retryable handler (v0.36.0)' `BLAME` config.py:36 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) `BLAME` config.py:382 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)
سایر مصرف‌کننده‌ها	`POST /api/v1/chat/tts` در adcs؛ تولید صدای Announcement در coa-api. `CODE` python/adcs/adcs/chat/api.py:18 · python/coa-api/coa/apps/iva/services.py:254 `BLAME` api.py:18 → s.hajili · 2026-08-19 · 0bd0127f3 feat(adcs): add tts asr `BLAME` services.py:254 → a.rahimi · 2026-09-26 · 0a54d0d24 feat(coa-api): add voice pages v5.14.0
Cache	neurobox-gateway خروجی TTS (Text-to-Speech) را Cache می‌کند (برای جملات تکراری مثل Greeting). `GIT` 57f969bc6 add tts caching (v3.5.0) `INFERENCE` benefit for repeated greetings

Egress — مسیر خروجی رسانه

مسیر خروجی رسانه/داده از سیستم. دو معنا دارد: (۱) صدای سنتزشده‌ای که به سمت تماس تلفنی یا Client WebRTC (Web Real-Time Communication) فرستاده می‌شود؛ (۲) در LiveKit به‌طور خاص، **Egress** یعنی ضبط یا Stream کردن خروجی یک Room به مقصدی دیگر.

در Phoenix	جزئیات
صدای Agent به تماس‌گیرنده	صدای Agent TTS (Text-to-Speech) را به‌عنوان Track در LiveKit Room منتشر می‌کند و آن Room را از مسیر SIP (Session Initiation Protocol) به تماس‌گیرنده می‌رساند. `CODE` python/chatbot-iva-agent/src/entrypoint.py:66-94 `BOARD` IVA board: LiveKit ↔ Kamailio `BLAME` entrypoint.py:66 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

در Phoenix	جزئیات
Egress Recording	سه نوع ضبط: `participant` (کاربر) و `agent` — پیش‌فرض روشن؛ `room` (ترکیبی) — پیش‌فرض خاموش. مقصد: MinIO، باکت `chatbot-mci-iva`. **CODE** python/chatbot-iva-agent/src/config.py:124-138,158 · python/chatbot-iva-agent/src/utils/egress_recorder.py:47 · python/chatbot-iva-agent/src/enums.py (EgressClientEnum) **BLAME** config.py:124 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat... **BLAME** config.py:158 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) **BLAME** egress_recorder.py:47 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat... **ADDED** enums.py → y.fallah · 2025-06-21 · 6213f04ac feat(chatbot-iva): add turn-detection. v0.6.0
ردیابی ضبط	مسیر هر ضبط در Session Summary روی Kafka: `room_record_path`، `participant_track_path`، `agent_track_path` — همین فایل‌ها بعداً توسط `grogu` برای ASR (Automatic Speech Recognition) و تحلیل خوانده می‌شوند. **CODE** python/chatbot-iva-agent/src/schema.py:115-131 · golang/grogu/internal/call/service.go:515-574 (downloadVoiceTracks → mergeIVAVoice → ASR) **BLAME** schema.py:115 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

کنار هم: مسیر کامل یک Turn صوتی

```
caller audio ──▶ VAD (Silero)          is someone speaking?
             ──▶ Turn detection         has the user finished? (endpointing 0.5-3s)
             ──▶ STT                    speech -> text (+ transcript_confidence)
             ──▶ LLM (aiva)             text -> answer, streamed
             ──▶ TTS                    sentence-by-sentence -> audio
             ──▶ Egress                 audio track -> LiveKit room -> SIP -> caller
                 └─ Egress recording    participant + agent tracks -> MinIO
while TTS plays, VAD keeps listening: user speech > 0.4s => barge-in, TTS stops, user's turn
```

CODE python/chatbot-iva-agent/src/main.py:157 (session wiring) · python/chatbot-iva-agent/src/config.py **INFERENCE** ordering diagram assembled from the config and lifecycle code
BLAME main.py:157 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)
ADDED config.py → y.fallah · 2025-06-21 · 6213f04ac feat(chatbot-iva): add turn-detection. v0.6.0

چرا برای Product مهم است: تأخیری که تماس‌گیرنده حس می‌کند (Time-to-first-audio) تقریباً برابر است با **تأخیر Endpointing** + **اولین جمله** STT (Speech-to-Text) + LLM (Large Language Model) + **اولین تکه** TTS (Text-to-Speech). هر عدد Latency که در جلسه قول بدهید، مجموع همین چهار جزء است — و اولی (Endpointing) مستقیماً با `AGENT_MIN_ENDPOINTING_DELAY` قابل تنظیم است.

CODE python/chatbot-iva-agent/src/config.py:70-72 INFERENCE latency decomposition
BLAME config.py:70 → y.fallah · 2026-08-03 · f36e6cc2f feat(chatbot-iva-agent): add openai turn detector (v0.30.0)

مسیر تماس: از شماره‌گیری تا IVA (Intelligent Virtual Assistant) — نقطه شروع Agent Pipeline

همه چیز در Phoenix با یک **تماس تلفنی** شروع می‌شود؛ اما Code این مخزن فقط از لحظه‌ای مسئولیت را می‌گیرد که تماس به‌شکل یک Participant وارد **LiveKit Room** شده است. قبل از آن، زنجیره‌ای طولانی از لایه‌های اپراتور، Backbone، دیتاسنتر، SBC (Session Border Controller) و SIP (Session Initiation Protocol) قرار دارد که اگر هرکدام آماده نباشد، بهترین LLM (Large Language Model) هم یک کلمه به تماس‌گیرنده نمی‌رسد. این فصل کل این زنجیره را — از «سرشماره» تا اولین جمله خوش‌آمدگویی — لایه‌به‌لایه باز می‌کند، مرز «بیرون از مخزن / داخل مخزن» را دقیق مشخص می‌کند و برای هر لایه معیار «تمام شد»، تست، علائم خرابی و سؤال جلسه می‌دهد.

> **برچسب جدید** `TELECOM` : دانش استاندارد مخابرات/VoIP (رفتار SIP (Session Initiation Protocol)، RTP (Real-time Transport Protocol)، SBC (Session Border Controller)، Codec، اعداد رایج). این‌ها **در این مخزن دیده نمی‌شوند** و توپولوژی واقعی شبکه همراه اول (SBC، Backbone، Zone، مسیر فیبر) در این سند فرض نشده است — فقط الگوی رایج صنعت. هر عددی که با این برچسب آمده را پیش از تعهد با تیم Core/Network/VoIP تأیید کنید. ادعاهای مربوط به Phoenix همچنان با `CODE` مستند شده‌اند.

فهرست اختصارات این فصل — مرجع سریع

در متن هم هر اختصار در **اولین استفاده در هر بخش** با فرم کامل آمده است؛ این جدول برای مراجعه سریع است (نام پیام‌های SIP مثل INVITE و BYE اختصار نیستند و نام Method هستند).

اختصار	فرم کامل	معنی / نقش
SIP	Session Initiation Protocol	پروتکل Signaling تماس (برقراری/پایان)
RTP	Real-time Transport Protocol	حمل خود صدا روی UDP
SDP	Session Description Protocol	مذاکره IP/پورت/Codec داخل پیام SIP
SBC	Session Border Controller	دروازه امنیتی/کنترلی SIP در مرز شبکه
PBX	Private Branch Exchange	مرکز تلفن سازمانی
STT	Speech-to-Text	گفتار به متن
TTS	Text-to-Speech	متن به گفتار
LLM	Large Language Model	مدل زبانی بزرگ
RAG	Retrieval-Augmented Generation	بازیابی دانش و افزودن به Prompt
VAD	Voice Activity Detection	تشخیص وجود گفتار در صدا
API	Application Programming Interface	واسط برنامه‌نویسی

اختصار	فرم کامل	معنی / نقش
KB	Knowledge Base	پایگاه دانش
HA	High Availability	در دسترس بودن بالا (چند Node)
DC	Data Center	دیتاسنتر
PoP	Point of Presence	نقطه حضور شبکه
LLD	Low-Level Design	طراحی سطح‌پایین شبکه
QoS	Quality of Service	اولویت‌بندی ترافیک
DSCP	Differentiated Services Code Point	برچسب اولویت در سرآیند IP
ACL	Access Control List	فهرست مجاز/ممنوع ترافیک
VLAN	Virtual Local Area Network	شبکه محلی مجازی
NAT	Network Address Translation	ترجمه آدرس شبکه (مشکل رایج صدا)
CAC	Call Admission Control	محدودسازی تعداد تماس پذیرفته‌شده
DTMF	Dual-Tone Multi-Frequency	تن‌های شماره‌گیری (فشردن کلید)
MOS	Mean Opinion Score	امتیاز کیفیت صدا (۱ تا ۵)
WER	Word Error Rate	نرخ خطای کلمه در STT
TPM	Technical Product Manager	مدیر فنی محصول
EX	Employee Experience	تجربه کارمند (پروژه Pilot)
KPI	Key Performance Indicator	شاخص کلیدی عملکرد
APM	Application Performance Monitoring	پایش عملکرد برنامه
SLA	Service Level Agreement	توافق سطح خدمت
IP	Internet Protocol	پروتکل اینترنت (آدرس‌دهی)
UDP	User Datagram Protocol	انتقال بدون اتصال (برای RTP)
TCP	Transmission Control Protocol	انتقال با اتصال
DTLS	Datagram Transport Layer Security	نسخه TLS برای UDP (تبادل کلید SRTP)
SRTP	Secure Real-time Transport Protocol	RTP رمزنگاری‌شده
ICE	Interactive Connectivity Establishment	انتخاب بهترین مسیر بین دو نقطه (WebRTC)
STUN	Session Traversal Utilities for NAT	کشف آدرس عمومی پشت NAT
TURN	Traversal Using Relays around NAT	Relay صدا وقتی مسیر مستقیم ممکن نیست
MTU	Maximum Transmission Unit	بزرگ‌ترین بسته قابل ارسال روی لینک (~۱۵۰۰ بایت)
SASL	Simple Authentication and Security Layer	چارچوب احراز هویت (Kafka)

اختصار	فرم کامل	معنی / نقش
SCRAM	Salted Challenge Response Authentication Mechanism	مکانیزم رمز SASL در Kafka (SHA-512)
RTT	Round-Trip Time	زمان رفت‌وبرگشت یک بسته
DSCP	Differentiated Services Code Point	علامت‌گذاری QoS در Header IP
TLS	Transport Layer Security	رمزنگاری انتقال (SIP روی 5061)
BGP	Border Gateway Protocol	پروتکل مسیریابی بین شبکه‌ها
MPLS	Multiprotocol Label Switching	سوئیچینگ بر پایه برچسب (Backbone)
ACK	Acknowledgement	تأیید دریافت 200 OK در SIP
ASR	Answer-Seizure Ratio	نسبت تماس‌های پاسخ‌داده‌شده (Telecom) — نه Speech Recognition
CPU	Central Processing Unit	پردازنده
GPU	Graphics Processing Unit	پردازنده گرافیکی (مدل‌های AI)
QPS	Queries Per Second	تعداد درخواست در ثانیه
RTT	Round-Trip Time	زمان رفت‌وبرگشت بسته
E2E	End-to-End	سرتاسری (از سیم‌کارت تا پاسخ)
SIM	Subscriber Identity Module	سیم‌کارت
AI	Artificial Intelligence	هوش مصنوعی
RFC	Request for Comments	سند استاندارد اینترنت (مثلاً RFC 3261 = SIP)
PCMA	Pulse Code Modulation A-law	Codec صدای G.711 نسخه A-law (اروپا/ایران)
G.711	ITU-T G.711	Codec استاندارد تلفن: ۸ kbps، ۶۴ kHz
E.164	ITU-T E.164	فرمت بین‌المللی شماره: +98…
PCMU	Pulse Code Modulation μ-law	G.711 نسخه μ-law (آمریکا/ژاپن)
MCI	Mobile Telecommunication Company of Iran	همراه اول

۱. نمای کلی — دو نیمه یک مسیر

```
Caller (MCI SIM) ── dials the service number (سرشماره)
  ▼
Core network "Zone"   number routing, signaling       [operator core]
  ▼
Backbone              IP/MPLS over fiber              [operator transport]
  ▼
DC edge               router → firewall/ACL → VLAN    [DC network]
  ▼
SBC                   SIP trunk terminates here       [VoIP team]
  ▼
SIP server            Kamailio → LiveKit SIP · HA      [VoIP team]
  ▼  SIP INVITE (signaling) + RTP (media, rtpengine?)
═══════════════════ Phoenix starts here ═══════════════════
LiveKit SIP → room    caller = participant "sip_+98…"
  ▼  job dispatch
chatbot-iva-agent     VAD · turn · STT · TTS · egress
  ▼  HTTP
aiva                  /api/v1/chat/session, /completions
audio back: TTS → room → LiveKit SIP → RTP → SBC → operator → caller
```

`TELECOM` layers above the double line `CODE` python/chatbot-iva-agent/src/main.py:79,108-109 · python/aiva/aiva/chat/api.py:6 `TEAM` Kamailio is the SIP server in use (team confirmation 2026-10-04; its config is not in this repo) `GIT` 265367421 feat(rtpengine): init 2026-08-05 — cpp/rtpengine (submodule + CI build) `INFERENCE` RTP relayed by rtpengine next to Kamailio

نیمه اول (بالای خط دوتایی) زیرساخت تلفنی است: متعلق به اپراتور و تیم‌های Network/VoIP، در Git این مخزن هیچ ردی ندارد و Dependency اصلی Deadline است. (**در Phoenix از Kamailio به‌عنوان SIP Server استفاده می‌کنیم:** تماس از SIP به Kamailio می‌رسد و Kamailio آن را به LiveKit SIP می‌فرستد تا LiveKit برای تماس Room بسازد. نقش Kamailio Routing، Failover و توزیع SIP (Session Initiation Protocol) بین Nodeهای LiveKit است — توضیح کامل در بخش ۳.) **نیمه دوم** همان چیزی است که در فصل‌های ۱۲ و ۱۵ دیدید. نقطه اتصال این دو نیمه — اینکه SIP (Session Initiation Protocol) Headerها سالم برسند، RTP (Real-time Transport Protocol) پایدار باشد و یک Worker آزاد وجود داشته باشد — شکننده‌ترین بخش کل محصول است.

۲. ۳۰ گام تا Production — Checklist کامل (مسیر IVA)

ترتیب رایج یک پروژه Voicebot در اپراتور. ستون «در Phoenix» نشان می‌دهد کدام گام در این مخزن ردپا دارد و کدام کاملاً بیرونی است. این جدول فقط مسیر **بلادرنگ** IVA (Intelligent Virtual Assistant) (در طول تماس) است. در این مسیر صدا هرگز در MinIO ذخیره یا از آن خوانده نمی‌شود: صدای تلفن از RTP (Real-time Transport Protocol) به LiveKit می‌رسد و Agent آن را به‌صورت `AudioFrame` در حافظه می‌گیرد. ذخیره صدا در MinIO و تبدیل آن به متن پس از تماس متعلق به مسیر جداگانه CCA (Call Center Analyze) است — بخش ۲.۱.

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
1	Business Requirement	سرویس برای چیست؟ فقط Inbound یا Outbound هم؟ ساعات سرویس؟ ضبط مکالمه؟ انتقال به اپراتور انسانی؟ رنج شماره‌ها؟ سیاست Routing؟	سند نیازمندی امضاشده شامل ظرفیت، مقصد انتقال و سیاست ضبط	Product / EX (Employee Experience) + TPM (Technical Product Manager)	ضبط: Egress فعال · مقصد انتقال پیش‌فرض `6040`
2	ظرفیت و Call Flow	تعداد تماس هم‌زمان (۱۰۰ / ۵۰۰ / ۱۰۰۰) در ساعت اوج؛ نمودار Call Flow: خوش‌آمد → گفت‌وگو → خروج / انتقال / قطع به‌خاطر سکوت.	عدد Peak Concurrency + نمودار Flow تأییدشده	TPM (Technical Product Manager) + معمار Voice	هر تماس = یک LiveKit Job؛ Flow در بخش ۴ و ۵ همین فصل
3	تخصیص سرشماره (Number Service)	شماره‌ای که تماس با آن وارد سرویس می‌شود. **سرشماره به‌تنهایی یعنی سرویس آماده نیست** — فقط شناسه ورود است؛ هنوز معلوم نیست تماس کجا Route شود.	شماره تخصیص‌یافته و ثبت‌شده در Numbering Plan	اپراتور (Numbering / تجاری)	**بیرونی**
4	Routing در شبکه اپراتور	Core Network باید بفهمد «این شماره متعلق به کدام سرویس است و تماس را کجا بفرستم» (Trunk Group مقصد). Signaling تعیین می‌کند تماس‌گیرنده کیست، مقصد کجاست، آزاد است یا نه، برقرار شد یا قطع.	تماس آزمایشی از سیم‌کارت همراه اول به SBC (Session Border Controller) می‌رسد (INVITE در Log SBC)	Core Network	**بیرونی**
5	طراحی Connectivity	کدام دیتاسنتر، کدام PoP (Point of Presence)، Redundancy (دو لینک / دو DC (Data Center))، پهنای باند. تخمین سرانگشتی: G.711 با Overhead شبکه ≈ **۸۷ kbps در هر جهت برای هر تماس** → ۵۰۰ تماس ≈ ۴۴ Mbps در هر جهت (بدون Egress ضبط). `TELECOM`	LLD (Low-Level Design) (Low-Level Design) تأییدشده	Network Architect	**بیرونی**
6	Backbone Connectivity	Backbone شبکه اصلی و پرظرفیت انتقال است (فیبر + تجهیزات Core). «وصل شدن به Backbone» یعنی **Connectivity + Routing + IP (Internet Protocol) Security + + تجهیزات انتقال**، نه فقط کشیدن یک کابل.	مسیر Provision شده از DC (Data Center) تا Zone	Transport / Backbone	**بیرونی**

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
7	نصب فیبر	**فیبر ≠ SIP (Session Initiation Protocol)**. فیبر فقط Medium است: Fiber → Optical Transceiver → Switch/Router → IP (Internet Protocol) → SIP. سؤال کلیدی: «فیبر از کجا به کجا؟»	Link Up، سطح توان نوری در محدوده مجاز	DC / Transport (Data Center)	بیرونی
8	Router / Switch VLAN (Virtual / Local Area IP / Network) (Internet Protocol)	IP (Internet Protocol) Address، Subnet، VLAN (Virtual Local Area Network)، Gateway، Routing (Static یا BGP (Border Gateway Protocol)) بین شبکه اپراتور (مثلاً `10.x.x.x`) و شبکه IVA (Intelligent Virtual Assistant) (`10.y.y.y`).	Ping و Traceroute دوطرفه، بدون Asymmetric Routing	DC (Data Center) Network	بیرونی
9	ACL / Firewall (Access Control QoS / List) (Quality of Service)	SIP (Session Initiation Protocol) بازکردن UDP (User `5060`) Protocol)، `5061` TCP/Datagram Protocol) و TLS (Transport Layer Security)) و **رنج پورت UDP برای RTP (Real-time Transport Protocol)**؛ علامت‌گذاری QoS (Quality of Service): صدا `DSCP EF (46)`، Signaling `CS3/AF31`. `TELECOM` برای SIP، Ping کافی نیست — خود Session و Media باید تست شوند.	تست پورت SIP (Session Initiation Protocol) و RTP (Real-time Transport Protocol) در هر دو جهت	Security / Network	بیرونی
10	SBC (Session Border Controller)	دروازه امنیتی و کنترلی SIP (Session Initiation Protocol): کنترل Session، امنیت، NAT (Network Address Translation) Traversal، Topology Hiding، محدودسازی دسترسی، کنترل ترافیک CAC (Call Admission Control)، Interconnect بین شبکه‌ها، Normalization پیام‌های SIP.	SBC (Session Border Controller) پیام `OPTIONS` اپراتور را دریافت و پاسخ می‌دهد	VoIP	بیرونی

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
1.1	SIP (Session Initiation Protocol) Trunk	ارتباط منطقی SIP (Session Initiation Protocol) بین اپراتور و SBC (Session Border Controller): IP (Internet Protocol)های Peer، Transport، Keepalive با `OPTIONS`، فرمت شماره E.164 (مثل `+98…`)، حداکثر کانال هم‌زمان.	`OPTIONS → 200 OK` دوطرفه؛ سقف کانال = عدد گام ۲	VoIP + اپراتور	بیرونی
1.2	SIP (Session Initiation Protocol) Configuration	Route در **Kamailio** (SIP Server ما) به LiveKit SIP (Session Initiation Protocol) و Inbound Trunk + Dispatch Rule در LiveKit SIP که برای هر تماس Room می‌سازد؛ **حفظ Headerهای سفارشی** (`X-originalCallID`، `X-subCityID`، `X-channelId`) و Headerهای استاندارد (`Call-ID`، `Via`) تا Agent.	Headerها در `participant.attributes` دیده می‌شوند	VoIP + Platform	**وابستگی مستقیم** جدول Headerها در بخش ۴
1.3	RTP (Real-time Transport Protocol) / Codec / Media	مذاکره SDP (Session Description Protocol)؛ Codec رایج شبکه تلفن **G.711 A-law** (**PCMA (Pulse Code Modulation A-law)**)، ۸ kHz، Narrowband، `ptime=20ms`؛ DTMF (Dual-Tone Multi-Frequency) با RFC (Request for Comments) 4733. تبعات برای AI (Artificial Intelligence): STT (Speech-to-Text) صدای ۸ kHz تلفنی می‌گیرد، نه صدای استودیو. **TELECOM**	صدای دوطرفه، Loss < ۱٪، Jitter < ۳۰ms	VoIP	`AGENT_SIP_RTP_STABILIZATION_DELAY` (پیش‌فرض ۰)

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
1 4	اتصال به IVA (Intelligent Virtual Assistant)	LiveKit SIP (Session Initiation Protocol) برای تماس یک Room می‌سازد، Caller را Participant می‌کند و Job را به یک Worker آزاد `chatbot-iva-agent` Dispatch می‌کند. Agent با `AutoSubscribe.AUDIO_ONLY` وارد Room می‌شود، منتظر Participant تماس‌گیرنده می‌ماند، Session را Start می‌کند، به اندازه `AGENT_SIP_RTP_STABILIZATION_DELAY` (پیش‌فرض ۰s) صبر می‌کند تا مسیر RTP (Real-time Transport Protocol) پایدار شود و سپس پیام خوش‌آمد را پخش می‌کند (`AGENT_GREETING_ALLOW_INTERRUPTION=true`: کاربر می‌تواند وسط خوش‌آمد صحبت کند). همزمان Background Player روشن می‌شود: صدای محیط اداری (`OFFICE_AMBIENCE`، Volume 0.8) و صدای «در حال فکر کردن» (`static/ts0.wav`، Volume 0.4) هنگام انتظار برای پاسخ. IVA (Intelligent Virtual Assistant) از MinIO استفاده نمی‌کند؛ تنها کاری که Agent برای CCA (Call Center Analyze) انجام می‌دهد درخواست ضبط Egress است (بخش ۲.۱، گام C1).	Log «starting voice assistant for participant sip_...»	Platform + تیم IVA (Intelligent Virtual Assistant)	`main.py:108-109` · `entrypoint.py:66-94,110-158` · `config.py:50-55,172-216,391-398`

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
1 5	دریافت صدای زنده (RTP (Real-time Transport Protocol) → AudioFrame)	صدای تماس‌گیرنده به‌صورت RTP (Real-time Transport Protocol) (G.711، ۸ kHz) به LiveKit SIP (Session Initiation Protocol) می‌رسد، به Audio Track در Room تبدیل می‌شود و Agent با Subscribe کردن آن، `AudioFrame` های PCM (Pulse Code Modulation) را **فقط در حافظه (RAM)** دریافت می‌کند — هیچ فایلی روی دیسک یا MinIO نوشته نمی‌شود. همین Frameها هم‌زمان به VAD (Voice Activity Detection) و STT (Speech-to-Text) داده می‌شوند. **جداسازی کانال در زمان تماس لازم نیست:** در Room، تماس‌گیرنده و Agent دو Participant با دو Track جدا هستند؛ Agent فقط Track تماس‌گیرنده را به STT (Speech-to-Text) می‌دهد و صدای TTS (Text-to-Speech) خودش را منتشر می‌کند. همین دو Track جدا بعداً با Egress جداگانه ضبط می‌شوند (بخش ۲.۱، C1). **توجه** پکیج `livekit-plugins-noise-cancellation` نصب است اما **فعال نیست** (`RoomOptions()` بدون Noise Cancellation)؛ AEC (Acoustic Echo Cancellation) هم Warmup پیش‌فرض ۰s است. نویز محیطی بدون فیلتر به VAD (Voice Activity Detection) و STT (Speech-to-Text) می‌رسد.	Agent برای هر تماس Track صوتی Caller را Subscribe کرده و Frame دریافت می‌کند (Prometheus در Metric VAD)	Platform + تیم IVA (Intelligent Virtual Assistant)	`main.py:108` · `entrypoint.py:68-72` · `requirements.txt:67`

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
16	VAD (Voice Activity Detection) – تشخیص صدا از سکوت	مدل **Silero VAD (Voice Activity Detection)** روی هر Frame احتمال «گفتار بودن» را می‌دهد. یک بار در `prewarm` هر Process بارگذاری می‌شود (نه در هر تماس). پارامترها: آستانه فعال‌سازی `AGENT_VAD_ACTIVATION_THRESH OLD=0.5`؛ حداقل طول گفتار `AGENT_VAD_MIN_SPEECH_DURATI ON=0.5s` (صداهای کوتاه‌تر مثل سرفه نادیده)؛ حداقل سکوت برای پایان گفتار `AGENT_VAD_MIN_SILENCE_DURAT ION=1.0s`؛ Padding ابتدای گفتار `AGENT_VAD_PREFIX_PADDING_DU RATION=2s` (تا شروع جمله بریده نشود). خروجی: رویدادهای START_OF_SPEECH / END_OF_SPEECH که وضعیت کاربر را speaking / listening می‌کنند.	روی نمونه تماس واقعی ۸ kHz: نویز و نفس شروع گفتار حساب نشود، مکث‌های کوتاه وسط جمله آن را نبرد	تیم IVA (Intelligent Virtual Assistant)	`main.py:87-96` · `config.py:61-67`
17	Voice Segmentation – تقطیع گفتار	چون STT (Speech-to-Text) غیر Streaming است، `StreamAdapter` بین START و END VAD (Voice Activity Detection) همه Frameها را در حافظه جمع می‌کند و هر بازه را یک **Segment** می‌سازد؛ با END_OF_SPEECHهای Frame، آن Segment به یک WAV (در حافظه) تبدیل و یک‌جا به STT (Speech-to-Text) فرستاده می‌شوند. پس Latency هر جمله = طول جمله + ۱s سکوت VAD (Voice Activity Detection) + زمان STT (Speech-to-Text). یک Turn کاربر ممکن است چند Segment باشد.	هر جمله کاربر دقیقاً یک درخواست STT (Speech-to-Text) می‌سازد؛ جمله‌ها وسط کلمه بریده نمی‌شوند	تیم IVA (Intelligent Virtual Assistant)	`agentlib.py:288-322` · `utils/stt.py:18-46`

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
18	Turn Detection / Endpointing — پایان نوبت کاربر	تصمیم اینکه «کاربر حرفش تمام شد و Agent باید جواب دهد». مدل از `AGENT_TURN_MODEL`: `mci` (مدل فارسی داخلی روی `AGENT_MCI_TURN_URL`)، `openai` (یک LLM (Large Language Model) مثل `gemma-4-31B-it`) احتمال پایان نوبت را برمی‌گرداند) یا `deactive` (پیش‌فرض). مدل ۶ Turn آخر را می‌بیند (`AGENT_MAX_HISTORY_TURNS`) و اگر احتمال پایان کمتر از `AGENT_UNLIKELY_THRESHOLD=0.5` باشد، Agent بیشتر صبر می‌کند. Endpointing: حالت `fixed`، حداقل تأخیر ۰.۵s، حداکثر ۳s. **ریسک بحرانی** از نسخه v0.36.0 (commit 859f8fbf5، 2026-09-28) Session از `turn_detection` به `"vad"` تغییر کرده. در livekit-agents 1.6.4، حالت manual پایان نوبت خودکار را (با VAD (Voice Activity Detection)، Final Transcript یا STT (Speech-to-Text)) غیرفعال می‌کند و فقط `commit_user_turn()` نوبت را می‌بندد — که در این مخزن هیچ‌جا صدا زده نمی‌شود. با `AGENT_TURN_MODEL=deactive` (پیش‌فرض) Agent احتمالاً به کاربر **پاسخ نمی‌دهد** و قطع صحبت Agent با صدای کاربر هم خاموش است. با `mci` یا `openai` مدل Agent روی Session اولویت دارد و رفتار مبتنی بر VAD (Voice Activity Detection) برمی‌گردد. باید مقدار `AGENT_TURN_MODEL` در Production چک شود.	Agent پس از هر جمله کامل جواب می‌دهد و وسط مکث‌های فکر کردن کاربر نمی‌پرد؛ مقدار AGENT_TURN_MODEL در Production مشخص و ثبت شده	تیم IVA (Intelligent Virtual Assistant)	· `main.py:157-167` · `chatbotlib.py:69-86` · `utils/mci_turn_detector.py` · `utils/openai_turn_detector.py` · `config.py:68-120` **GIT** 859f8fbf5 y.fallah 2026-09-28 "vad" → "manual" **LIB** livekit-agents 1.6.4 voice/agent_activity.py:230-250,1919-1922 · voice/audio_recognition.py:225,1090,1167,1276-1281 **INFERENCE** production value of AGENT_TURN_MODEL unknown (env)

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
19	Interruption (Barge-in) — قطع صحبت Agent	اگر کاربر وسط صحبت Agent حرف بزند، پخش TTS (Text-to-Speech) متوقف می‌شود. حالت `vad`: صدای کاربر حداقل `AGENT_MIN_INTERRUPTION_DURATION=0.4s` باشد؛ حداقل کلمه `AGENT_MIN_INTERRUPTION_WORDS=0`. پیام‌های قطع‌شده در تاریخچه با `interrupted=true` ثبت و در Kafka Summary ارسال می‌شوند. (وابسته به ریسک گام ۱۸: در حالت manual قطع با صدا خاموش است.)	کاربر با یک کلمه وسط پاسخ Agent آن را قطع می‌کند؛ نویز کوتاه یا «اوهوم» قطع نمی‌کند	تیم IVA (Intelligent Virtual Assistant)	`main.py:168-173` · `config.py:68-75` · `entrypoint.py:295-310`

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
20	STT (Speech-to-Text)	تبدیل هر Segment (گام ۱۷) به متن؛ دقت روی صدای ۸ kHz فارسی. **صدای زنده کجاست؟** فقط در حافظه (RAM)، هیچ فایلی روی دیسک یا MinIO نیست: RTP (Real-time Transport Protocol) → LiveKit SIP (Session Initiation Protocol) → Audio Track در Room → Agent در Subscribe کردن آن، `AudioFrame` های PCM (Pulse Code Modulation) را دریافت می‌کند. چون STT (Speech-to-Text) از نوع OpenAI-compatible و غیر Streaming است، `StreamAdapter` فریم‌ها را با کمک VAD (Voice Activity Detection) در حافظه بافر می‌کند؛ وقتی VAD (Voice Activity Detection) پایان جمله را تشخیص داد (سکوت `AGENT_VAD_MIN_SILENCE_DURATION`، پیش‌فرض ۱s)، فریم‌ها در حافظه به WAV تبدیل و با یک HTTP POST به `/audio/transcriptions` فرستاده می‌شوند. **Non-speech:** اگر STT (Speech-to-Text) متن خالی برگرداند (نویز، صدای نامفهوم)، پس از `AGENT_NON_SPEECH_OCCURENCE_THRESHOLD=1` بار Agent می‌گوید «صدای شما نامفهوم بود، لطفا بلندتر صحبت کنید». پیش‌فرض خاموش و فقط در ساختار `agentlib` (در `chatbotlib` پیش‌فرض، پیاده‌سازی نشده).	WER (Word Error Rate) روی نمونه تماس واقعی زیر آستانه توافق‌شده	تیم AI (Artificial Intelligence)	OpenAI-compatible STT (Speech-to-Text)، Timeout ۱۰s، ۳ Retry · `agentlib.py:288-322` (StreamAdapter + VAD (Voice Activity Detection)) · `utils/stt.py:18-46` (WAV در حافظه → HTTP) · Non-speech: `agentlib.py:363-393`, `config.py:430-446`

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
2 1	AI (Artificial Intelligence) LLM / Agent (Large Language Model) RAG / (Retrieval-Augmented Generation)	Intent Detection، RAG (Retrieval-Augmented Generation)، Tool Calling، Business Logic. متن کاربر (خروجی گام ۲۰) با HTTP به `aiva` می‌رسد و این مراحل را طی می‌کند: **(۱) Vectorizing دانش — از قبل، آفلاین:** پایگاه دانش (`adcs-knowledgebase`) پاسخ‌ها و Guardrailها را با Celery Task `sync_qdrant_task` به Embedding تبدیل و در Vector DB **Qdrant** Upsert می‌کند. هر متن یک بردار عددی می‌شود که معنای آن را نشان می‌دهد. **(۲) Embedding پرسش کاربر:** همان جمله کاربر با مدل `text-embedding-3-large` (`EMBEDDING_MODEL`، API (Application Programming Interface) (OpenAI-compatible به بردار تبدیل می‌شود. **(۳) جست‌وجوی برداری در Qdrant** (gRPC، پورت `6334`): نزدیک‌ترین بردارها در سه Collection — `guardrail` (موضوعات ممنوع/حساس)، `aiva-contexts` (دانش برای پاسخ؛ حداکثر ۱۰ نتیجه، حداقل Score `0.5`) و `aiva-instructions` (دستورالعمل رفتار Agent). نتایج Context و Instruction سپس با یک LLM (Large Language Model) سبک فیلتر می‌شوند تا فقط موارد واقعاً مرتبط بمانند (قابل خاموش‌کردن با `*_FILTER_LLM_ENABLED`). **(۴) Tool Calling برای انجام نیاز کاربر:** ReAct Agent با Context بازیابی‌شده، حافظه کاربر و تاریخچه گفت‌وگو تصمیم می‌گیرد مستقیم جواب دهد یا یک Tool صدا بزند، نتیجه Tool را بخواند و دوباره فکر کند (حداکثر `REACT_MAX_ITERS=10` دور).	سناریوهای طلایی (Golden Set) پاس می‌شوند: برای هر پرسش نمونه، Context درست از Qdrant بازیابی می‌شود، Guardrail موارد ممنوع را می‌گیرد و Tool درست با Argumentهای درست صدا زده می‌شود	تیم AI (Artificial Intelligence) + تیم پایگاه دانش	`aiva` — فصل ۱۴ · `aiva/chat/handlers/chat.py:55-140` · `aiva/chat/config.py:32,54-80` · `aiva/tools/registry.py` · `agents/intent_detection/tools.py` **CODE** `python/agentlib/agentlib/retrieve/qdrant_retriever.py:45-72` · `python/adcs-knowledgebase/adcs_kb/celery/tasks/qdrant.py:67-245,331` **INFERENCE** knowledgebase writes to the same `aiva-contexts` collection — its collection names come from env (`IVA_CONTEXTS_COLLECTION_NAME`), not a default in code

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
		Toolهای فعال پیش‌فرض: `get_gold_price`، `set_reminder`، `read_reminders`، `send_sms`، `tell_joke` (خودش در Collection `aiva-jokes` جست‌وجوی برداری می‌کند). Agent تشخیص Intent دو Tool جدا دارد: `redirect_to_human_agent` (انتقال به اپراتور) و `exit_tool` (پایان تماس). **(۵)** پاسخ نهایی Agent به Stream صوتی برمی‌گردد → گام ۲۳ (TTS (Text-to-Speech)).			
2	APIs / Backend (Application Programming Interfaces)	مثال: «بسته اینترنت من کی تموم میشه؟» → `Intent = CHECK_PACKAGE` → API (Application Programming Interface) → Gateway → Customer Service → «۲ گیگابایت باقی مانده».	API (Application Programming Interface) اپراتور در دسترس + SLA (Service Level Agreement)	تیم Backend اپراتور	Toolهای `aiva/tools/` (SMS (Short Message Service)، یادآور، …)

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
23	TTS (Text-to-Speech)	تبدیل متن پاسخ به صدا، جمله‌به‌جمله. متن Stream شده از LLM (Large Language Model) ابتدا از Markdown و Emoji پاک می‌شود، سپس `SentenceTokenizer` آن را روی جداکننده‌های `AGENT_SENTENCE_TOKENIZER_SEPARATORS` (پیش‌فرض فقط `.` `"`) به جمله می‌شکند. هر جمله جداگانه به TTS (Text-to-Speech) (OpenAI-compatible، مدل `tts-1`، سرعت ۱) فرستاده و Frameهای صوتی آن بلافاصله در Room پخش می‌شوند؛ پس کاربر جمله اول را می‌شنود در حالی که بقیه هنوز تولید می‌شوند. کد وضعیت `204` Retry نمی‌شود. **توجه** جداکننده فقط «. » است؛ علامت‌های فارسی «؟» و «!» و «،» جمله را جدا نمی‌کنند و پاسخ‌های طولانی بدون نقطه دیرتر شروع به پخش می‌شوند.	کیفیت شنیداری تأییدشده روی گوشی واقعی	تیم AI (Artificial Intelligence)	Streaming جمله‌به‌جمله · `chatbotlib.py:342-375` · `utils/sentence_tokenizer.py` · `utils/tts.py` · `config.py:36-43,164-167,383-388`
24	سکوت کاربر (User Away)	اگر کاربر `AGENT_USER_AWAY_TIMEOUT=10`s صحبت نکند، وضعیت `away` می‌شود و Agent می‌گوید «من هستم اگه کاری داشتی»؛ تا `AGENT_USER_AWAY_OCCURENCE_THRESHOLD=3` بار تکرار و سپس پیام خداحافظی و قطع تماس. صحبت کاربر حلقه را Reset می‌کند. پیش‌فرض `AGENT_USER_AWAY_MODE=disabled` و فقط در ساختار `agentlib` ؛ در `chatbotlib` (پیش‌فرض) فقط Timer وضعیت اجرا می‌شود و تماس ساکت باز می‌ماند تا یکی از مسیرهای خروج بخش ۵.	تماس ساکت پس از حداکثر ~۴۰s با پیام خداحافظی بسته می‌شود (در صورت فعال بودن)	تیم IVA (Intelligent Virtual Assistant)	`utils/agent_session.py:23-52` · `agentlib.py:395-425` · `chatbotlib.py:402-409` · `config.py:401-427`
25	End-to-End Test	کل زنجیره از سیم‌کارت تا پاسخ صوتی (بخش ۶).	چک‌لیست بخش ۶ کامل	QA + همه تیم‌ها	—

#	گام	چه اتفاقی می‌افتد / چه باید تصمیم گرفت	معیار «تمام شد»	مالک معمول	در Phoenix
26	Load Test	۵۰۰ تماس = ۵۰۰ SIP (Session Initiation Protocol) Session = ۵۰۰ RTP (Real-time Transport Protocol) Stream = ۵۰۰ Session هوش مصنوعی.	عدد گام ۲ با Latency و Success Rate قابل‌قبول	QA / SRE	فقط `websocall/tests/loadtest.js` (مسیر WebSocket، بدون SIP (Session Initiation Protocol))
27	HA (High Availability) / Failover	حداقل دو Node IVA (Intelligent Virtual Assistant)، ترجیحاً دو دیتاسنتر.	قطع یک Node وسط بار، سرویس ادامه دارد	SRE	Drain Timeout ۱۸۰۰s برای Deploy
28	Monitoring	Network → SIP (Session Initiation Protocol) → Application → AI (Artificial Intelligence).	داشبورد و Alert برای هر لایه	SRE	Prometheus روی پورت `9400` + APM (Application Performance Monitoring)
29	Pilot	ترافیک محدود (گروه شماره یا درصد تماس).	KPI (Key Performance Indicator)های Pilot در بازه توافقی	TPM (Technical + Product Manager) + EX (Employee Experience)	**توجه** Whitelist شماره در Phoenix ورود تماس را محدود نمی‌کند (یافته ۹) — محدودیت Pilot باید در Routing/SBC باشد
30	Production	باز شدن کامل ترافیک + Runbook پشتیبانی.	Go-live Sign-off	همه	—

TELECOM steps 1-13, 25-30 (industry pattern) **CODE** python/chatbot-iva-agent/src/config.py:50-55,255 · python/chatbot-iva-agent/src/agents/agentlib.py:288-322 · python/chatbot-iva-agent/src/utils/stt.py:18-46 (step 20) · python/chatbot-iva-agent/src/main.py:87-96,157-175 (steps 16-19) · python/chatbot-iva-agent/src/config.py:61-120,164-216,383-446 · golang/websocall/tests/loadtest.js **GIT** 377b34e6c feat(websocall): load test script 2026-09-28 **CODE** python/aiva/aiva/memory/config.py:20-21 (finding 9)

۲.۱ مسیر CCA (Call Center Analyze) پس از تماس — ASR (Automatic Speech Recognition) → MinIO → Kafka

CCA (Call Center Analyze) (در Code با پیشوند `coa` : `coa-api` و Worker `grogu`) کاملاً از مسیر بلادرنگ جداست و **بعد از پایان تماس** اجرا می‌شود: صدای تماس در MinIO ذخیره می‌شود، فقط **آدرس فایل** یا شناسه تماس روی Kafka می‌رود، و `grogu` فایل را می‌خواند و با ASR (Automatic Speech Recognition) به متن تبدیل می‌کند. دو منبع تماس دارد: تماس‌های IVA (Intelligent Virtual Assistant) و تماس‌های اپراتور انسانی از سامانه TelC. Latency این مسیر روی تجربه تماس‌گیرنده اثری ندارد.

#	گام	تماس IVA (Intelligent Virtual Assistant)	تماس TelC (اپراتور انسانی)	Code
C1	ذخیره صدا	Agent در شروع تماس ضبط LiveKit Egress را درخواست می‌دهد (`participant` و `agent` جداگانه، `.ogg`). سرویس Egress فایل را در MinIO، باکت `chatbot-mci-iva`، آپلود می‌کند. **INFERENCE** آپلود پس از پایان Egress (پایان تماس) انجام می‌شود — رفتار LiveKit Egress، نه Code این مخزن.	صدا در سامانه TelC است؛ هنوز در MinIO نیست.	`entrypoint.py:94,160-180` · `egress_recorder.py:23,113-125`
C2	اعلام روی Kafka	پایان تماس: Session Summary با `participant_track_path` و `agent_track_path` (فقط آدرس، نه صدا) روی topic `chatbot-mci-iva` (`AGENT_KAFKA_TOPIC`).	پیام با `call_id` و فلگ `online` روی `kafka_asr_call_topic`.	`schema.py:115-131` · `config.py:315-318` · grogu `app.go:314,331`
C3	دریافت صدا در grogu	دانلود هم‌زمان دو Track از باکت IVA (Intelligent Virtual Assistant) → **جداسازی دو کانال** با ffmpeg: هر Track به Mono تبدیل می‌شود (`pan=mono|c0=FL`) و با فیلتر `join` صدای **تماس‌گیرنده در کانال چپ** و صدای **Agent در کانال راست** یک فایل استریو قرار می‌گیرد (16 kHz، ۸ MP3، kbps) → آپلود در باکت CCA (Call Center Analyze) با کلید `call_id`. مسیر خالی یا `"failed"` → پیام Skip.	فایل TelC از قبل دوکاناله است و grogu آن را Merge نمی‌کند (**INFERENCE** از ساختار TelC، با قرینه بررسی ch1/ch2 در C4). `online=true`: دریافت صدا از API (Application Programming Interface) سامانه TelC و آپلود در باکت CCA (Call Center Analyze). `online=false`: دانلود مستقیم از باکت CCA (Call Center Analyze) با `call_id`.	`voice/merge.go:36-38,195-212` · `call/service.go:515-574,894,948-982` (IVA (Intelligent Virtual Assistant)) · `:359,469-473,873-891` (TelC)

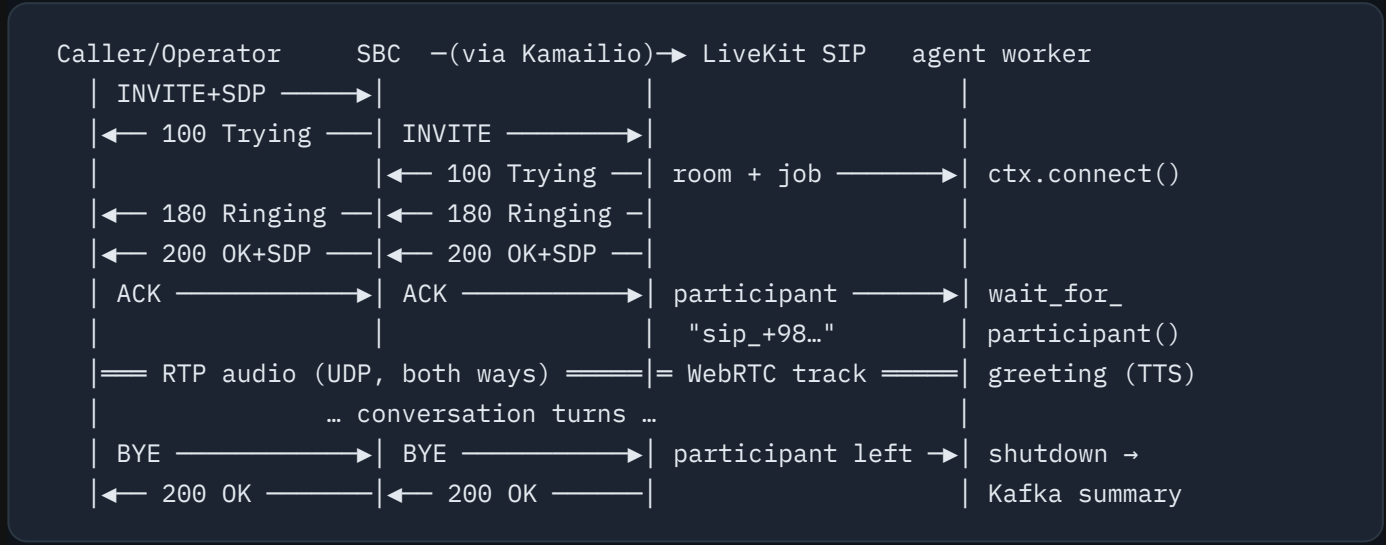
#	گام	تماس IVA (Intelligent Virtual Assistant)	تماس TelC (اپراتور انسانی)	Code
C4	ASR (Automatic Speech Recognition)	فایل کامل تماس (دو کانال) با `GetVoiceTranscriptions` و Circuit Breaker به ASR (Automatic Speech Recognition) فرستاده می‌شود. هر کانال را جدا به متن تبدیل می‌کند و هر Segment برچسب `channel` دارد: • `ch1` = کانال چپ = **مشتری** • `ch2` = کانال راست = **Agent / اپراتور** (منبع: `ticket/prompt.go:12-13`) **بررسی‌های سلامت نتیجه:** • تماس IVA (Intelligent Virtual Assistant): اگر در نتیجه ASR (Automatic Speech Recognition) **هیچ Segment برای** `ch1` (مشتری) نباشد → مشتری حرف نزده یا فایل خراب است (`ErrEmptyASRSegments`). • تماس TelC: اگر Segmentها **فقط برای** `ch1` باشند و برای `ch2` (اپراتور) هیچ Segment نباشد → صدای اپراتور در تماس نیست (حرف نزده، تماس قبل از پاسخ‌دادن قطع شده، یا کانال او خراب ضبط شده) (`ErrAgentSilentASRSegments`). تماسی که در آن هر دو کانال حرف دارند از این بررسی رد می‌شود. **تشخیص هم‌صحبتی (Interrupt):** اگر Segmentی از `ch1` و Segmentی از `ch2` بیش از `voice_overlap_threshold=1.0s` هم‌زمان باشند، هم‌پوشانی ثبت می‌شود تا معلوم شود چه کسی وسط حرف دیگری پریده است. **توجه** فایل ارسالی MP3 است اما با نام `audio.wav` به ASR (Automatic Speech Recognition) داده می‌شود.	`asr/service.go:27,85,188-194` · `call/service.go:41,149-157,984-1030` · `voice/service.go:11-50` · `config/config.go:100` **CODE** `golang/grogu/internal/ticket/prompt.go:12-13 (ch1 = customer, ch2 = agent)` **INFERENCE** `ch1 = left / ch2 = right channel mapping is the ASR service convention (ASR not in repo); TelC files assumed already two-channel (grogu does not merge them)`
C5	ذخیره و انتشار نتیجه	Upsert در `call_records` و `call_ai_tags` ، ارسال نتیجه روی Kafka؛ خطای دیگر → Dead-letter (`call_asr_dl`).	`grogu/worker.go:224,329`
C6	نمایش در داشبورد	`coa-api` فایل صوتی را از MinIO برای پخش در داشبورد Stream می‌کند (یافته IDOR در بخش ۵).	`coa-api/coa/apps/files/views.py:19,46`

ریسک در تماس IVA (Intelligent Virtual Assistant)، Agent آدرس فایل را روی Kafka می‌فرستد بدون اینکه منتظر پایان آپلود Egress بماند؛ ممکن است پیام زودتر از فایل برسد. `grogu` برای همین دانلود IVA (Intelligent Virtual Assistant) را با Retry و Backoff انجام می‌دهد و Metric `MinioVoiceFileNotFoundTotal` را می‌شمارد؛ اگر Retryها تمام شوند، تماس به Dead-letter می‌رود. تأیید قطعی زمان آپلود: Timestamp فایل‌ها در باکت پس از یک تماس آزمایشی.

CODE `python/chatbot-iva-agent/src/entrypoint.py:94,160-180` · `python/chatbot-iva-agent/src/utils/egress_recorder.py:23,113-125,604-606` · `python/chatbot-iva-agent/src/config.py:152-158,315-318` · `python/chatbot-iva-agent/src/schema.py:115-131` · `golang/grogu/internal/grogu/app.go:314,331` · `golang/grogu/internal/call/service.go:115-148,359,469-473,515-574,873-891,894,948-982,984` · `golang/grogu/internal/minio/service.go:113-117,265-311` · `python/coa-api/coa/apps/files/views.py:19,46` **INFERENCE** `Egress upload timing (LiveKit behavior)`

۳. SIP (Session Initiation Protocol) در برابر RTP (Real-time Transport Protocol) — Signaling و Media

مهم‌ترین تمایزی که در جلسه‌های Voice باید بلد باشید: **SIP (Session Initiation Protocol) صدای مکالمه را حمل نمی‌کند.** SIP می‌گوید تماس چطور برقرار، تغییر و تمام شود (Signaling)؛ صدای واقعی با **RTP (Real-time Transport Protocol)** روی UDP (User Datagram Protocol) جابه‌جا می‌شود (Media). مشخصات Media (IP (Internet Protocol)، پورت، Codec) داخل بدنه **SDP (Session Description Protocol)** در پیام‌های SIP مذاکره می‌شود — برای همین Firewall/NAT اشتباه معمولاً باعث «تماس وصل شد ولی صدا نمی‌آید» می‌شود، نه «تماس وصل نشد».



TELECOM SIP ladder (RFC 3261) CODE python/chatbot-iva-agent/src/main.py:108-109 · src/entrypoint.py:68,86,93 INFERENCE exact timing of room creation vs. 200 OK depends on LiveKit SIP config (not in repo)

Kamailio در Phoenix: ما از Kamailio به‌عنوان **SIP Server** استفاده می‌کنیم تا تماس‌ها به LiveKit برسند. Kamailio خودش Room نمی‌سازد: `INVITE` را به سرویس LiveKit SIP می‌فرستد و LiveKit SIP با Inbound Trunk و Dispatch Rule خود Room را می‌سازد، Caller را Participant `sip_+98…` می‌کند و Job را به Agent می‌دهد. در نمودار بالا، فلش SBC → LiveKit SIP در عمل از Kamailio عبور می‌کند.

جزء	نقش	حمل صدا (RTP)؟
SIP (Session Initiation Trunk Protocol)	سرور نیست؛ ارتباط توافقی اپراتور ↔ SBC (IPهای Transport، Peer، فرمت شماره، سقف کانال).	—
SBC (Session Border Controller)	دروازه امنیتی لبه شبکه: NAT، Topology Hiding، سقف تماس هم‌زمان، Normalization پیام‌ها. معمولاً متعلق به تیم VoIP یا اپراتور.	بله
Kamailio (SIP Server ما)	Router داخلی SIP (Session Initiation Protocol): تماس را به LiveKit SIP می‌رساند؛ توزیع بار بین چند Node LiveKit SIP؛ Failover با پایش `OPTIONS`؛ Routing بر اساس شماره یا گروه Pilot (محل مناسب محدودیت Pilot — یافته ۹)؛ اصلاح Headerها و Rate Limit. افزودن یا Drain کردن Node بدون تغییر در SBC.	خیر (فقط Signaling)؛ INFERENCE صدا احتمالاً از rtpengine عبور می‌کند
LiveKit SIP	Endpoint تماس: SIP/RTP را به Room و WebRTC Track تبدیل می‌کند و Job را به `chatbot-iva-agent` Dispatch می‌کند.	بله

TEAM Kamailio is the SIP server in use (team confirmation 2026-10-04) BOARD IVA board: Kamailio -> LiveKit GIT 265367421 feat(rtpengine): init TELECOM component roles (industry pattern) INFERENCE rtpengine media relay; Kamailio/LiveKit SIP config not in repo

پیام / کد	معنا	وقتی در Log می‌بینید...
INVITE	«می‌خواهم تماس برقرار کنم» + SDP (Session Description Protocol) offer (پورت/IP (Internet Protocol)/Codecهای پیشنهادی)	تماس به این لایه رسیده؛ اگر نرسیده، مشکل Routing/ACL پیش از آن است
100 Trying	«دریافت شد، در حال پردازش» (Hop-by-hop)	لایه بعدی زنده است
183 / 180 Ringing Session Progress	مقصد در حال زنگ خوردن / Early Media	—
200 OK	«تماس پذیرفته شد» + SDP (Session Description Protocol) answer (Codec انتخاب‌شده)	از اینجا صورت‌حساب و RTP (Real-time Transport Protocol) شروع می‌شود
ACK	تأیید دریافت 200 OK	اگر نرسد، 200 OK تکرار و تماس حدود ۳۲ ثانیه بعد قطع می‌شود
CANCEL / BYE	پایان تماس برقرار / لغو پیش از پاسخ	چه کسی BYE فرستاد؟ Caller، Agent یا Timeout SBC (Session Border Controller)
OPTIONS	Keepalive / سلامت Trunk	Trunk زنده است حتی بدون تماس
MESSAGE	پیام متنی درون SIP (Session Initiation Protocol)	**Phoenix برای انتقال به کارشناس از همین استفاده می‌کند** (بخش ۵)
480 / 404 / 403	ممنوع (Trunk/ACL (Access Control List)) / شماره نامعتبر / مقصد موقتاً در دسترس نیست	۴۰۳: IP (Internet Protocol) در Whitelist SBC (Session Border Controller) نیست · ۴۰۴: فرمت شماره · ۴۸۰: Worker آزاد نیست
503 / 486 / 408	Timeout / مشغول / سرویس در دسترس نیست	۵۰۳ در بار بالا: سقف کانال Trunk یا ظرفیت Worker پر شده

TELECOM RFC 3261 status semantics; 32 s = 64×T1 transaction timeout INFERENCE 480/503 mapping to LiveKit worker exhaustion depends on LiveKit SIP config

۳.۱ دانش شبکه — TCP (Transmission ،UDP (User Datagram Protocol) Control Protocol)، TLS (Transport Layer Security) و تفاوت‌ها

هر ارتباط در Phoenix روی چند لایه سوار است: IP آدرس و مسیر را تعیین می‌کند؛ **TCP یا UDP** نحوه تحویل بسته‌ها را؛ **TLS یا DTLS** (اختیاری) رمزنگاری و احراز هویت را؛ و در بالا پروتکل کاربردی (SIP، RTP، HTTP، gRPC، Kafka) قرار دارد. بیشتر خرابی‌های «تماس وصل شد ولی ...» از اینکه کدام لایه مشکل دارد معلوم می‌شود، نه از Code برنامه.

۱. مقایسه پروتکل‌ها

پروتکل	چیست / چه می‌کند	تحویل مطمئن و به‌ترتیب؟	هزینه شروع (Handshake)	رمزنگاری؟	کاربرد در Phoenix
UDP	بسته مستقل می‌فرستد؛ بدون اتصال، بدون تأیید دریافت، بدون ارسال مجدد.	خیر — بسته ممکن است گم، تکراری یا نامرتب برسد	هیچ؛ بسته اول همان داده است	خیر	صدای تماس (RTP)، SIP روی 5060، مسیر WebRTC
TCP	اتصال دوطرفه؛ شماره‌گذاری بسته‌ها، تأیید دریافت، ارسال مجدد، کنترل ازدحام.	بله — همه بایت‌ها کامل و به‌ترتیب	۱ RTT (Handshake سه‌مرحله‌ای)	خیر (خودش)	HTTP به STT/LLM/TTS/aiva، Kafka، PostgreSQL، MinIO، gRPC، WebSocket
TLS	لایه امنیتی روی TCP: رمزنگاری، جلوگیری از دستکاری، احراز هویت سرور با Certificate (و در mTLS هویت کلاینت).	همان TCP	اضافه بر TCP: ۱ ← TLS 1.3 RTT؛ ۲ ← TLS 1.2 RTT	بله	HTTPS، SIP روی 5061 (SIPS)، gRPC امن
DTLS	همان TLS برای UDP (با تحمل گم‌شدن و نامرتبی بسته).	خیر (مثل UDP)	چند بسته رفت‌وبرگشت	بله	تبادل کلید SRTP در WebRTC (LiveKit)
RTP / SRTP	قالب حمل صدا روی UDP: شماره ترتیب و Timestamp برای بازسازی زمان‌بندی. SRTP همان با رمزنگاری Payload.	خیر — Receiver با Jitter Buffer جبران می‌کند	هیچ (کلید از SIP/SDP یا DTLS می‌آید)	RTP: خیر · SRTP: بله	صدای ۸ kHz بین اپراتور و LiveKit؛ داخل Room با WebRTC (SRTP)
SIP	پروتکل کاربردی Signaling (برقراری/پایان تماس)؛ روی UDP، TCP یا TLS.	وابسته به Transport؛ روی UDP خودش Retransmit دارد	وابسته به Transport	فقط با TLS	اپراتور ↔ Kamailio ↔ SBC LiveKit SIP
HTTP / HTTPS	درخواست–پاسخ روی TCP؛ HTTPS یعنی HTTP روی TLS.	بله	TCP + (TLS در HTTPS)؛ Keep-Alive اتصال را نگه می‌دارد	فقط HTTPS	Agent → STT / LLM (aiva) / TTS؛ grogu → ASR؛ Egress/grogu → MinIO (S3 API)
WebSocket	اتصال دوطرفه ماندگار که با HTTP Upgrade شروع می‌شود (روی TCP).	بله	یک بار HTTP + Upgrade	wss:// بله · ws:// خیر	`websocall` (gorilla/websocket)
gRPC	RPC باینری (Protobuf) روی HTTP/2 و TCP؛ چند درخواست هم‌زمان روی یک اتصال.	بله	TCP (+TLS اختیاری)	اختیاری	aiva → Qdrant (پورت `6334`)

پروتکل	چیست / چه می‌کند	تحویل مطمئن و به‌ترتیب؟	هزینه شروع (Handshake)	رمزنگاری؟	کاربرد در Phoenix
ICE / STUN / TURN	پیدا کردن مسیر بین دو نقطه پشت NAT: STUN آدرس عمومی را می‌گوید؛ TURN اگر مسیر مستقیم ممکن نبود، صدا را Relay می‌کند؛ ICE بهترین مسیر را انتخاب می‌کند.	—	چند ثانیه‌ای در اتصال اولیه	—	WebRTC بین Agent و LiveKit (داخل شبکه معمولاً مسیر مستقیم)

۲. چرا صدا روی UDP است نه TCP

در مکالمه زنده، بسته‌ای که با ۲۰۰ms تأخیر برسد دیگر به درد نمی‌خورد. در TCP یک بسته گم‌شده تا ارسال مجدد، **همه بسته‌های بعدی را هم نگه می‌دارد** (Head-of-Line Blocking) و صدا ثانیه‌ای قطع می‌شود و بعد فشرده می‌رسد. در UDP همان بسته گم‌شده فقط یک «پَر» کوتاه (۲۰ms) در صدا می‌سازد و مکالمه ادامه می‌دهد. برای همین در RTP گم‌شدن را می‌پذیریم و با Jitter Buffer، نوسان زمان رسیدن را صاف می‌کنیم.

معیار	UDP	TCP
گم‌شدن بسته	رها می‌شود؛ مکالمه ادامه دارد	ارسال مجدد؛ همه چیز پشت آن منتظر می‌ماند
تأخیر	کمترین؛ بدون Handshake	بیشتر؛ Handshake + انتظار برای تأیید
مناسب برای	صدا/تصویر زنده، Signaling کوتاه	داده‌ای که باید کامل برسد (متن، فایل، پیام)
عبور از Firewall/NAT	سخت‌تر: «اتصال» ندارد و باید بازه پورت صریح باز شود	ساده‌تر: Firewall حالت اتصال را دنبال می‌کند
علامت مشکل	صدای بریده، یک‌طرفه، یا بی‌صدا	Timeout، کندی، اتصال Reset

محاسبه پهنای باند یک تماس G.711: هر ۲۰ms یک بسته → ۵۰ بسته در ثانیه؛ ۱۶۰ Payload بایت + ۱۲ (RTP Header) + UDP ۸ + IP ۲۰ = (۴۰ ۲۰۰ = بایت → ۸۰ kbps در لایه IP؛ با Header اترنت (۱۸ بایت) ≈ **۸۷ kbps** در هر جهت — همان عددی که در گام ۵ چک‌لیست برای ظرفیت‌سنجی استفاده شد. چون بسته‌ها کوچک و پرتعدادند، **تعداد بسته در ثانیه** برای Firewall و Router مهم‌تر از حجم بایت است.

۳. SIP: انتخاب Transport — UDP یا TCP یا TLS

Transport	پورت	مزیت	عیب / تله
UDP	`5060`	ساده و سبک؛ پیش‌فرض اغلب اپراتورها؛ SIP خودش Retransmit می‌کند	پیام بزرگ (INVITE با Header و SDP زیاد) از MTU (~۱۵۰۰ بایت) بیشتر می‌شود و Fragment یا گم می‌شود؛ طبق RFC 3261 پیام بیش از حدود ۱۳۰۰ بایت باید روی TCP برود. Keep-Alive برای باز ماندن NAT لازم است.
TCP	`5060`	پیام بزرگ بدون Fragment؛ اتصال ماندگار؛ Firewall-friendly	اتصال قطع شود باید دوباره برقرار شود؛ Head-of-Line بین تماس‌های هم‌اتصال؛ هزینه نگهداری اتصال برای هزاران Trunk

Transport	پورت	مزیت	عیب / تله
TLS	`5061`	Signaling رمز می‌شود؛ هویت طرف مقابل با Certificate تأیید می‌شود	**انقضای Certificate** کلاسیک‌ترین علت قطع ناگهانی Trunk است؛ CA و نام (CN/SAN) باید دو طرف مطابق باشند؛ Handshake اولیه کندتر. TLS فقط Signaling را رمز می‌کند؛ **صدا (RTP) رمز نمی‌شود مگر SRTP هم فعال باشد.**

۴. نقشه پروتکل هر ارتباط در Phoenix

ارتباط	پروتکل / پورت	جزئیات و ریسک	مرجع
اپراتور ↔ SBC ↔ Kamailio ↔ LiveKit SIP (Signaling)	SIP روی TCP/UDP 5060 یا 5061 TLS	انتخاب Transport توافق VoIP و اپراتور است و در این مخزن نیست. ICMP/Ping سالم بودن SIP را ثابت نمی‌کند؛ خود Session باید تست شود.	گام ۹، ۱۱ چک‌لیست **بیرونی**
اپراتور ↔ LiveKit SIP (صدا)	RTP روی UDP، بازه پورت	G.711 A-law، ۸ kHz، ptime ۲۰ms. Firewall باید **کل بازه UDP** را در هر دو جهت باز کند و QoS را با DSCP (46) EF حفظ کند.	گام ۹، ۱۳ چک‌لیست
LiveKit ↔ Agent (Room)	WebRTC: ICE + SRTP-DTLS روی UDP؛ سیگنال‌دهی و Job Dispatch با WebSocket	اگر UDP بسته باشد WebRTC به TCP یا TURN برمی‌گردد و تأخیر و Jitter بالا می‌رود. پورت‌های LiveKit (مثلاً 7880 سیگنال، TCP 7881، یک بازه/پورت UDP برای Media) را باید تیم Platform تأیید کند.	**INFERENCE** رفتار کتابخانه livekit-agents؛ آدرس و پورت‌های سرور LiveKit در env است نه Code
Agent → STT / aiva / TTS	HTTP(S) روی TCP	Timeout اتصال ۵s، خواندن ۱۰s، نوشتن ۵s؛ Retry ۳؛ Pool تا ۱۰۰ اتصال، ۵۰ Keep-Alive با انقضای ۳۰۰s. کندی شبکه مستقیماً Latency هر Turn صوتی را بالا می‌برد.	`chatbot-iva-agent/src/config.py:9-24`
aiva → Qdrant	gRPC (HTTP/2) روی TCP — پورت `6334`	یک اتصال برای چند جست‌وجوی هم‌زمان (Guardrail + Context + Instruction).	`aiva/aiva/chat/config.py:67-68`
Agent / grogu ↔ Kafka	TCP + SASL/SCRAM-SHA-512	Agent پایتون صراحتاً `SASL_PLAINTEXT`: **احراز هویت دارد ولی رمزنگاری انتقال ندارد.** در grogu هم SASL فعال است و در فایل پیکربندی ردی از TLS پیدا نشد. داده شامل Transcript مکالمه است.	`chatbot-iva-agent/src/utils/kafka.py:62-65` · `grogu/internal/kafka/config.go:88-94`
grogu → PostgreSQL	TCP، `sslmode=disable`	اتصال بدون TLS؛ فقط به اتکای امن‌بودن شبکه داخلی.	`grogu/internal/database/service.go:69`
Egress / grogu / coa-api ↔ MinIO	HTTP(S) روی S3 API	http یا https بودن را مقدار Endpoint در env تعیین می‌کند (در Code ثابت نیست). فایل‌های ضبط صدای مشتری از این مسیر می‌گذرند.	`chatbot-iva-agent/src/config.py:152-158`
انتقال به اپراتور انسانی	پیام SIP MESSAGE (مثال: UDP 5060)	sip-messager یک MESSAGE با Header `X-Destination` می‌فرستد و حداکثر ۱۵s منتظر `202 Accepted` می‌ماند. Transport از Header Via تماس اصلی گرفته می‌شود.	بخش ۵ همین فصل · `sip-messager/internal/api/api.go:165-190`

ارتباط	پروتکل / پورت	جزئیات و ریسک	مرجع
کلاینت‌ها ↔ websocall	WebSocket (ws/wss) روی TCP	اتصال ماندگار؛ هر کاربر یک اتصال → سقف پورت/File Descriptor در Load Test مهم است.	`golang/websocall/go.mod:7`

TELECOM protocol semantics (RFC 3261 §18.1.1 UDP size limit; RFC 3550 RTP; RFC 5764 DTLS-SRTP; RFC 8446 TLS 1.3) CODE python/chatbot-iva-agent/src/config.py:9-24,152-158 · src/utils/kafka.py:62-65 · python/aiva/aiva/chat/config.py:67-68 · golang/grogu/internal/kafka/config.go:88-94 · golang/grogu/internal/database/service.go:69 · golang/websocall/go.mod:7 INFERENCE LiveKit ports / WebRTC transport details and SIP transport choice (config outside repo); no TLS flag found in grogu Kafka config; no NAT or firewall rules for SIP/RTP exist in the repo (only metrics ports: chatbot-iva-agent config.py:49 → 9400, grogu/Dockerfile:44 → 2112); NAT/firewall behavior described is general VoIP practice

۵. NAT (Network Address Translation) — چرا دشمن شماره‌یک صدا است

NAT آدرس خصوصی داخل شبکه (مثلاً `10.x.x.x`) را موقع خروج به آدرس عمومی/دیگر تبدیل می‌کند و برای برگشت، جدول «چه کسی کدام اتصال را باز کرد» نگه می‌دارد. انواع رایج: **SNAT** (تغییر آدرس مبدأ برای خروج)، **DNAT** (تغییر آدرس مقصد برای ورود، مثل Port Forward) و **PAT** (چند دستگاه با یک IP عمومی و پورت‌های متفاوت). برای وب و API بی‌ضرر است؛ برای VoIP مشکل‌ساز است چون:

۱. SIP **آدرس را داخل متن پیام می‌برد.** IP و پورت صدا در بدنه SDP (خط `c=` و `m=`) و در Headerهای `Via` و `Contact` نوشته می‌شود. NAT فقط Header بسته IP را عوض می‌کند، نه متن SIP را؛ پس طرف مقابل صدا را به آدرس **خصوصی** می‌فرستد که از بیرون قابل‌دسترس نیست → «تماس وصل شد ولی صدا نیست» یا «صدا یک‌طرفه است».

۲. RTP **روی** UDP **است و «اتصال» ندارد.** NAT یک «سوراخ» موقت برای هر جریان باز می‌کند و اگر مدتی (معمولاً ۳۰s تا چند دقیقه) بسته‌ای عبور نکند آن را می‌بندد. سکوت طولانی کاربر یا Hold می‌تواند صدا را قطع کند.

۳. **پورت صدا پویا است.** هر تماس پورت RTP تازه دارد (در SDP توافق می‌شود)، پس نمی‌توان یک قانون ثابت DNAT برای آن نوشت.

راه‌حل	چطور کار می‌کند	نکته
IP عمومی/ثابت برای سرورهای SIP (NAT ۱:۱ یا بدون NAT)	آدرسی که داخل SDP می‌نویسند همان آدرس قابل‌دسترس است؛ ریشه مشکل برداشته می‌شود.	بهترین گزینه برای Kamailio و LiveKit SIP
Symmetric RTP	صدا را به همان IP:Port می‌فرستیم که بسته‌ها از آن رسیده‌اند، نه آدرس داخل SDP.	اغلب SBC و rtpengine انجام می‌دهند
Media Anchoring (SBC / rtpengine)	صدا از یک نقطه ثابت و با آدرس درست عبور می‌کند و SDP بازنویسی می‌شود.	INFERENCE rtpengine در مخزن به‌عنوان Submodule و CI هست (`cpp/rtpengine`)؛ نقش دقیق آن در مسیر تماس تأیید نشده
Keep-Alive	`OPTIONS` دوره‌ای برای SIP و جریان RTP (حتی بسته سکوت) تا «سوراخ» NAT بسته نشود.	OPTIONS گام ۱۱ چک‌لیست همین کار را می‌کند

راه‌حل	چطور کار می‌کند	نکته
ALG (Application Layer SIP Gateway)	Router/Firewall متن SIP را می‌خواند و آدرس‌های داخلش را بازنویسی می‌کند.	**توجه** معمولاً باید **خاموش** شود؛ اغلب پیاده‌سازی‌هایش اشتباه است، Headerهای سفارشی مثل `X-originalCallID` را خراب می‌کند و عیب‌یابی را سخت می‌کند
TURN / STUN / ICE	در WebRTC هر دو طرف آدرس‌های ممکن را امتحان می‌کنند؛ آخرین راه Relay با TURN است.	برای مسیر LiveKit ↔ Agent؛ SIP کلاسیک ICE ندارد

در Phoenix: شبکه اپراتور و شبکه IVA هر دو آدرس خصوصی (`10.x`) دارند (گام ۸ چک‌لیست)، پس اگر بین SBC و LiveKit SIP NAT/Firewall باشد، همه موارد بالا مطرح می‌شود. در این مخزن هیچ پیکربندی NAT یا IP عمومی برای SIP/RTP وجود ندارد و باید با تیم Network/VoIP تأیید شود (`INFERENCE`). تنها اثر مرتبط در Code، `AGENT_SIP_RTP_STABILIZATION_DELAY` (پیش‌فرض ۰s) است که Agent را قبل از پخش خوش‌آمد کمی منتظر می‌گذارد تا مسیر RTP پایدار شود.

۶. Firewall و ACL (Access Control List) — چه چیزی باید باز باشد

Firewall بسته‌ها را بر اساس **مبدأ، مقصد، پورت و پروتکل** اجازه یا رد می‌کند. دو نوع: **Stateless** (هر بسته جدا بررسی می‌شود؛ هر دو جهت باید صریح باز باشد) و **Stateful** (اتصال را دنبال می‌کند و بسته برگشتی را خودکار اجازه می‌دهد). برای TCP حالت اتصال روشن است؛ برای UDP Firewall فقط یک «جریان موقت» با Timeout می‌سازد. ACL (Access Control List) فهرست ساده Allow/Deny روی Router و Switch است.

مسیر	پروتکل / پورت	قانون و نکته
اپراتور / SBC → Kamailio	TCP/UDP `5060` ، TLS `5061`	فقط از IPهای Peer اپراتور (Allowlist)؛ از اینترنت یا شبکه‌های دیگر نه.
Kamailio → LiveKit SIP	همان SIP	فقط بین این دو جزء.
SBC / rtpengine ↔ LiveKit SIP (صدا)	UDP، **کل بازه پورت RTP**	**هر دو جهت.** پورت‌ها در هر SDP تماس پویا انتخاب می‌شوند، پس باید بازه کامل باز باشد، نه یک پورت. بازه خیلی کوچک → «تماس سوم به بعد بدون صدا».
LiveKit سرور ↔ Agent	WebSocket (TCP) + WebRTC (UDP)، یا TCP/TURN اگر UDP بسته باشد)	فقط داخل شبکه Platform. پورت‌های دقیق در env/تنظیمات LiveKit است.
Agent → STT / aiva / TTS، grogu → ASR	TCP (HTTP/HTTPS)	Egress از Agent به این سرویس‌ها؛ Timeout اتصال ۵s پس Drop بی‌صدا Firewall ۵ ثانیه تماس را قفل می‌کند.
aiva → Qdrant	TCP `6334` (gRPC)	فقط از Podهای aiva و ابزار KB.
Egress / grogu / coa-api → MinIO	TCP پورت S3	فایل‌های ضبط صدا از این مسیر می‌گذرند؛ خارج از شبکه داخلی باز نباشد.

مسیر	پروتکل / پورت	قانون و نکته
سرویس‌ها → Kafka / PostgreSQL	TCP	چون Kafka و PostgreSQL در Code بدون رمزنگاری انتقال‌اند (بخش ۴)، Firewall تنها لایه حفاظت شبکه‌ای آن‌ها است؛ فقط سرویس‌های مجاز.
Prometheus → سرویس‌ها	TCP `9400` (Agent)، `2112` (grogu)	فقط از Prometheus.

- **حمله و اسکن SIP:** پورت 5060 در اینترنت دائماً اسکن می‌شود. Allowlist مبدأ و Rate Limit (در SBC/Kamailio) از Flood و تماس‌های جعلی جلوگیری می‌کند. ۵۰۰ تماس هم‌زمان ≈ ۵۰۰×۵۰۰ = ۲۵,۰۰۰ بسته RTP در ثانیه در هر جهت؛ Firewall باید این نرخ بسته (نه فقط پهنای باند) را تحمل کند.
- **جدول اتصال (Conntrack):** هر جریان UDP یک ردیف می‌گیرد؛ پر شدن جدول باعث Drop تماس‌های جدید می‌شود. Timeout UDP باید از مکث معمول مکالمه بلندتر باشد.
- **تست درست:** Ping (ICMP) و باز بودن پورت TCP چیزی درباره UDP نمی‌گوید. باید پیام `OPTIONS` واقعی فرستاد و با Packet Capture (مثلاً `tcpdump` یا `sngrep`) دید INVITE می‌رسد، SDP چه آدرسی دارد و RTP در هر دو جهت رد می‌شود (گام ۹ چک‌لیست).

۷. عیب‌یابی: علامت ← لایه محتمل

علامت	لایه محتمل	چه چیزی چک شود
تماس وصل می‌شود اما صدا نیست	RTP / UDP / NAT	بازه پورت UDP در Firewall (دو جهت)، IP داخل SDP (آدرس خصوصی به جای عمومی)، Packet Capture روی SBC
صدا فقط یک‌طرفه است	NAT / Routing نامتقارن	مسیر برگشت RTP، SBC یا rtpengine، Symmetric RTP
صدا بریده یا رباتی است	Loss / Jitter روی UDP	Loss < ۱٪ و Jitter < ۳۰ms (گام ۱۳)، QoS با DSCP EF، ازدحام لینک
تماس اصلاً INVITE نمی‌گیرد، Ping سالم است	SIP / پورت 5060-5061	ICMP ≠ UDP؛ OPTIONS، Trunk IP Whitelist، ACL دوطرفه
INVITE بزرگ گم می‌شود (Timeout بعد ~۳۲s)	MTU / Fragmentation روی UDP	گذاشتن SIP روی TCP، کم‌کردن Headerها، MTU مسیر
Trunk یک‌باره از کار می‌افتد و Handshake خطا می‌دهد	TLS	تاریخ انقضای Certificate، زنجیره CA، SAN، نسخه TLS
پاسخ AI دیر می‌رسد، STT Timeout می‌دهد	HTTP / TCP	Retry و Timeout گام ۲۰ (۵/۱۰s)، RTT تا سرویس، ازدحام، DNS
Kafka Consumer قطع و وصل می‌شود	SASL / TCP	اتصال Broker، رمز SCRAM، Rebalance

۴. نقطه شروع Pipeline در Code — تماس وقتی به Phoenix می‌رسد

از اینجا همه چیز در مخزن قابل خواندن است. ترتیب دقیق اتفاقات برای یک تماس ورودی:

#	اتفاق	جزئیات
0	Prewarm (قبل از هر تماس)	هر Process کاری یک بار **Silero** VAD (Voice Activity Detection) را Load و یک `IVAAgentSession` آماده می‌سازد تا زمان پاسخ اولین تماس صرف Load مدل نشود. `AGENT_NUM_IDLE_PROCESSES=1` یعنی فقط یک Process گرم در رزرو است. **CODE** python/chatbot-iva-agent/src/main.py:88 · src/config.py:50 **BLAME** main.py:84 → y.fallah · 2025-06-21 · 6213f04ac feat(chatbot-iva): add turn-detection. v0.6.0
1	Dispatch و اتصال به Room	LiveKit یک Job برای Room تماس به Worker می‌دهد؛ Agent با `AutoSubscribe.AUDIO_ONLY` وصل می‌شود (ویدئو Subscribe نمی‌شود). **CODE** python/chatbot-iva-agent/src/main.py:108 **BLAME** main.py:113 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0)
2	انتظار برای Caller	`wait_for_participant()` — طرف SIP (Session Initiation Protocol) با Identity به شکل `sip_+989121234567` وارد Room می‌شود. **CODE** python/chatbot-iva-agent/src/main.py:109
3	استخراج و نرمال‌سازی شماره	Regex `sip_\+?([a-zA-Z0-9]+)` ؛ سپس: ۱۰ رقم → `98` + شماره؛ ۱۱ رقم با `0` → `98` + بقیه؛ در غیر این صورت همان‌طور. مثال: `9121234567` ، `09121234567` و `+989121234567` همه → `989121234567` . اگر Regex نخورد، شماره خالی است (در agentlib به `"agentlib"` تبدیل می‌شود). **CODE** python/chatbot-iva-agent/src/main.py:79,320-335 · src/agents/agentlib.py:95-98 **BLAME** main.py:328 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)
4	انتخاب نوع Agent	`AGENT_LLM_STRUCTURE` : `agentlib` (مسیر AIVA (Agentic Intelligent Virtual Assistant) با Session) یا `chatbotlib` (قدیمی، Greeting ثابت). **CODE** python/chatbot-iva-agent/src/main.py:145
5	خواندن SIP (Session Initiation Protocol) Headerها	از `participant.attributes` با پیشوند `sip.h.` — جدول زیر.
6	ساخت Session در AIVA (Agentic Intelligent Virtual Assistant)	Agent مقدار `{"phone": …}` را به `AGENT_LLM_SESSION_BASE_URL` POST می‌کند؛ AIVA (Agentic Intelligent Virtual Assistant) کاربر را با شماره `get_or_create` می‌کند، یک Session تازه می‌سازد و `GreetingAgent` با **حافظه بلندمدت همان شماره** یک خوش‌آمد شخصی تولید می‌کند. پاسخ: `{id, greeting_message}` با `201` . اگر خطا دهد: `session_id="unknown"` و متریک `agent_llm_session_failed_requests_total` . **CODE** python/chatbot-iva-agent/src/agents/agentlib.py:64-93 · python/aiva/aiva/chat/api.py:6 · python/aiva/aiva/chat/views/session.py:55,72 **INFERENCE** AGENT_LLM_SESSION_BASE_URL points to aiva /api/v1/chat/session in deployment **BLAME** agentlib.py:70 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session data towards kafka and enabling human handover (v0.17.0) **BLAME** session.py:55 → Hamed Mahmoudkhani · 2026-07-19 · 8e766717d refactor(aiva): use adcs data models

#	اتفاق	جزئیات
7	شروع Session صوتی	`session.start(room, agent)` — از این لحظه VAD (Voice Activity Detection)، Turn Detection، STT (Speech-to-Text) و TTS (Text-to-Speech) روی Track صوتی Caller فعال‌اند. CODE python/chatbot-iva-agent/src/entrypoint.py:68
8	مکث پایداری RTP (Real-time Transport Protocol)	`AGENT_SIP_RTP_STABILIZATION_DELAY` (پیش‌فرض `0.0` ثانیه). CHANGELOG علت را ثبت کرده: «در بعضی موارد مدتی طول می‌کشد تا تماس SIP (Session Initiation Protocol) فعال شود» — اگر ابتدای خوش‌آمد بریده شنیده می‌شود، این اولین پیچ تنظیم است. CODE python/chatbot-iva-agent/src/entrypoint.py:77 · src/config.py:54-58 DOC python/chatbot-iva-agent/CHANGELOG.md:31 BLAME entrypoint.py:77 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0)
9	پخش خوش‌آمد	`session.say(greeting)`؛ پیش‌فرض قابل قطع (`AGENT_GREETING_ALLOW_INTERRUPTION=True`). متن پیش‌فرض chatbotlib: «سلام، من دستیار هوشمند همراه اول هستم، چطور می‌تونم کمکتون کنم؟» CODE python/chatbot-iva-agent/src/entrypoint.py:80-90 · src/config.py:391-398 BLAME entrypoint.py:86 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)
10	صدای پس‌زمینه، Summary و ضبط	Player صدای Ambient/Thinking، ثبت Callback خاتمه (ارسال `SessionSummarySchema` به Kafka topic `chatbot-mci-iva`) و شروع Egress Recording (Participant، Agent، Room). CODE python/chatbot-iva-agent/src/entrypoint.py:92-94,160-180 · src/config.py:315-318

از قدم ۱۰ به بعد، حلقه Turn صوتی شروع می‌شود که در «مفاهیم پایه Voice AI (Artificial Intelligence)» و فصل ۱۵ آمده است.

Headerهایی SIP (Session Initiation Protocol) که Agent می‌خواند

Attribute	نوع	کاربرد در Phoenix	اگر نرسد
`sip.h.x-originalcallid`	سفارشی اپراتور	`call_id` و `original_call_id` در Summary Kafka — کلید اتصال جلسه IVA (Intelligent Virtual Assistant) به رکورد تماس اپراتور	`"unknown"` یا `session_id` طبق `AGENT_ORIGINAL_CALL_ID_FALLBACK`
`sip.h.x-subscityid`	سفارشی اپراتور	`city_code` در Summary (تحلیل استانی)	`"unknown"`
`sip.h.call-id`	استاندارد SIP (Session Initiation Protocol)	شناسه Dialog برای انتقال به کارشناس	انتقال انجام نمی‌شود
`sip.h.via`	استاندارد SIP (Session Initiation Protocol)	IP (Internet Protocol):Port آخرین SIP Hop (Session Initiation Protocol) — مقصد پیام `MESSAGE` انتقال (Regex `SIP/2.0/\w+` `ip:port`)	انتقال انجام نمی‌شود

Attribute	نوع	کاربرد در Phoenix	اگر نرسد
`sip.h.X-channelId`	سفارشی سوئیچ	شناسه کانال برای انتقال	انتقال انجام نمی‌شود

CODE python/chatbot-iva-agent/src/agents/chatbotlib.py:56-59,227-240 · src/agents/agentlib.py:32,208-227 · src/entrypoint.py:272-274 · src/enums.py:27-30

BLAME chatbotlib.py:56 → y.fallah · 2026-08-15 · 5f4dd7199 feat(chatbot-mci-iva): add arbaeen_send_sms function call & original_call_id fallback scenarios (v0.31.0)

BLAME chatbotlib.py:58 → y.fallah · 2026-06-21 · fdacd4433 fix(chatbot-iva-agent): add support for sip header subcityID (v0.24.0)

BLAME agentlib.py:208 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session data towards kafka and enabling human handover (v0.17.0)

ریسک پنهان مرز Telecom/Phoenix: SBC (Session Border Controller)ها معمولاً Headerهای ناشناخته `X-*` را برای Topology Hiding حذف می‌کنند. اگر `X-originalCallID` حذف شود، تماس کار می‌کند اما در Summary با `call_id="unknown"` ثبت می‌شود و **اتصال جلسه** IVA (Intelligent Virtual Assistant) **به رکورد تماس اپراتور و تحلیل‌های بعدی از دست می‌رود** — بدون هیچ خطای آشکار. اگر `X-channelId` / `Via` حذف شوند، انتقال به کارشناس بی‌صدا شکست می‌خورد. **عبور هر پنج Header باید معیار پذیرش** SIP (Session Initiation Protocol) **Trunk (گام ۱۲) باشد.**

۵. پایان تماس — چهار مسیر خروج

مسیر	چه می‌شود	شاهد
Caller قطع می‌کند	`BYE` → Participant از Room خارج می‌شود → Shutdown Callback → Summary به Kafka و بستن Egress.	CODE src/entrypoint.py:93,182-215
Tool `exit`	LLM (Large Language Model) تصمیم به پایان می‌گیرد → صبر تا پخش کامل جمله آخر → `delete_room()` → LiveKit پای SIP (Session Initiation Protocol) را می‌بندد (BYE).	CODE src/agents/chatbotlib.py:377-380 INFERENCE BYE sent by LiveKit SIP
Tool `redirect_to_human_agent`	صبر تا پخش جمله → IP (Internet Protocol):Port از `Via` → POST به `sip-messager` → پیام SIP (Session Initiation Protocol) `MESSAGE` به `sip:msg@<ip>:<port>` با `X-Resolution: Done` و `X-Destination: 6040` → سوئیچ اپراتور تماس را به صف کارشناس می‌برد → در هر حال `delete_room()`.	CODE src/agents/agentlib.py:202-286
سکوت کاربر (User Away)	اگر `AGENT_USER_AWAY_MODE` فعال باشد (پیش‌فرض **غیرفعال**): پس از `10s` سکوت «من هستم اگه کاری داشتی»؛ پس از بیش از `3` بار، پیام خداحافظی و `delete_room()`.	CODE src/config.py:401-427 · src/utils/agent_session.py:36-52 · src/agents/agentlib.py:469

انتقال به کارشناس انسانی در جزئیات

```
chatbot-iva-agent            sip-messager (Go)          operator switch
 redirect_to_human_agent()
 wait_for_playout()
 Via "SIP/2.0/UDP 10.a.b.c:5060" → ip, port
 POST /api/v1/send-sip-message ─▶ SIP MESSAGE
  from: sip:<REDIRECT_URL>        To: sip:msg@10.a.b.c:5060
  to: sip:msg@10.a.b.c + to_port    Call-ID, CSeq
  call_id, x_channel_id             X-Resolution: Done
  x_call_id = aiva session_id       X-Destination: 6040
  x_resolution, x_destination       body: "ping" ──────────▶ transfer to
  body: "ping"                    ◀── 202 Accepted ≤15s ── queue 6040
 finally: delete_room()   (agent leaves; the call now belongs to the switch)
```

`CODE` python/chatbot-iva-agent/src/agents/agentlib.py:202-286 · src/config.py:237-257 · golang/sip-messager/internal/api/api.go:165-190,194-241 · golang/sip-messager/config/config.go:23,125

`INFERENCE` switch-side handling of MESSAGE → transfer

`BLAME` api.go:166 → MohammadMahdi Afshari · 2025-10-28 · dceccd02c feat(sip-messager): initilize project (v0.1.0)

`BLAME` config.py:255 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session data towards kafka and enabling human handover (v0.17.0)

نکته برای جلسه: این انتقال SIP (Session Initiation Protocol) **REFER استاندارد نیست**؛ یک پیام `MESSAGE` خارج از Dialog است که سوئیچ طرف اپراتور باید آن را بفهمد (قرارداد اختصاصی با Headerهای `X-Resolution` و `X-Destination`). پس «انتقال به کارشناس» یک Dependency دوطرفه است: Phoenix فقط پیام را می‌فرستد؛ اجرای انتقال با تیم سوئیچ است. متریک موفق/ناموفق: `agent_sip_redirect_requests_total` . در صورت شکست، کاربر می‌شنود «متاسفانه امکان انتقال به کارشناس انسانی وجود ندارد.» و Room بسته می‌شود.

`CODE` python/chatbot-iva-agent/src/utils/prometheus_metrics.py:49-53 · src/config.py:240-243

۶. تست — لایه‌به‌لایه و عیب‌یابی

«فیبر وصل شد، تمام» معیار پذیرش نیست. هر لایه تست خودش را دارد و تست لایه بالاتر فقط وقتی معنا دارد که لایه پایین‌تر سبز باشد.

لایه	چه چیزی تست شود	ابزار رایج	معیار پاس (پیشنهادی)	قلاب در Phoenix
Network	Link Up، IP (Internet Protocol)، Routing، Firewall، Packet Loss، Latency	`ping` ، `mtr` ، `iperf3`	Loss < ۰٫۱٪، RTT (Round-Trip Time) پایدار	—
SIP (Session Initiation Protocol)	`OPTIONS` ، جریان INVITE → 180 → 200 → ACK (Acknowledgement)، رسیدن Headerها	`sngrep` ، `SIPp` ، `tcpdump`	۱۰۰٪ تماس‌های آزمایشی 200 OK؛ ۵ Header حاضر	Log Debug `X-` و `originalCallID` `X-` `subCityID`

لایه	چه چیزی تست شود	ابزار رایج	معیار پاس (پیشنهادی)	قلاب در Phoenix
Media	صدای دوطرفه (نه یک‌طرفه)، بدون قطع‌ووصل، Codec، Loss، Latency، Jitter	Wireshark RTP (Real-time Transport Protocol) Analysis، آمار SBC (Session Border Controller)	Loss < ۱٪، Jitter < ۳۰ms، MOS (Mean Opinion Score) ≥ ۴	ضبط جداگانه Participant/Agent در MinIO برای شنیدن
AI (Artificial Intelligence)	Voice → STT (Speech-to-Text) → Agent → Backend/RAG → TTS (Text-to-Speech) → Voice	مجموعه تماس طلایی ضبط‌شده	WER (Word Error Rate)، دقت Intent، پاسخ صحیح در سناریوها	`transcript_confidenc` `e` هر Utterance؛ تنها Eval مخزن در `aqa` است (یافته ۵)
E2E (End-to-End) Latency	از پایان حرف کاربر تا اولین صدای پاسخ	Timestamp روی ضبط دوکاناله	عدد توافقی (مثلاً < ۲s)	متریک‌های `durat/` `agent_*_ttft` APM (Application + `ion` Performance Monitoring) Spans
Load	عدد Concurrency گام ۲ به مدت پایدار	`SIPp` با فایل صوتی (مسیر کامل)	Success Rate ≥ ۹۹٪ و Latency زیر سقف	`websocall/tests/load` Pipeline فقط `test.js` AI (Artificial Intelligence) را بدون SIP (Session Initiation Protocol) می‌سنجد — **تست بار SIP در مخزن وجود ندارد**
Failover	خاموش کردن یک Node/Worker وسط بار؛ قطع یک لینک	Chaos دستی	تماس‌های جدید به Node سالم می‌روند	Drain `1800s` در Deploy

TELECOM tools and thresholds (common VoIP targets, confirm with operator SLA) CODE golang/websocall/tests/loadtest.js · python/chatbot-iva-agent/src/config.py:51 INFERENCE grep for sipp/SIP load tooling in repo → 0

عیب‌یابی: علامت → لایه مشکوک

علامت	لایه	اول چه چیزی را چک کنید
زنگ نمی‌خورد / بوق اشغال سریع	Routing (۳-۴)	آیا INVITE اصلاً به SBC (Session Border Controller) می‌رسد؟ اگر نه، Numbering/Routing اپراتور.
`403` در SBC (Session Border Controller)	ACL (Access Control List) / Trunk (۹-۱۱)	IP (Internet Protocol) مبدأ در Whitelist SBC (Session Border Controller) و Firewall.
وصل می‌شود ولی هیچ صدایی نیست	Media (۹، ۱۳)	رنج پورت RTP (Real-time Transport Protocol) در Firewall؛ IP (Internet Protocol) داخل SDP (Session Description Protocol) NAT (Network Address Translation) قابل دسترسی است؟
صدای یک‌طرفه	NAT (Network Address Translation) / Routing	Asymmetric Routing یا Firewall فقط یک جهت را باز کرده.

علامت	لایه	اول چه چیزی را چک کنید
قطع شدن حدود ۳۲ ثانیه بعد از پاسخ	SIP (Session Initiation Protocol)	`ACK` به مقصد نمی‌رسد (مشکل Record-Route/NAT).
قطع در زمان ثابت (مثلاً ۱۵ یا ۳۰ دقیقه)	SBC (Session Border Controller)	Session Timer یا سقف مدت تماس در SBC (Session Border Controller).
ابتدای خوش‌آمد بریده است	RTP (Real-time Transport Protocol) ↔ Agent	افزایش `AGENT_SIP_RTP_STABILIZATION_DELAY` (مثلاً ۰٫۵s).
صدای رباتیک / تکه‌تکه	Media یا CPU (Central Processing Unit)	Jitter/Loss و QoS (Quality of Service)؛ یا CPU (Central Processing Unit) پر روی Worker Agent.
وصل می‌شود ولی Agent حرف نمی‌زند	Platform / Agent	Worker آزاد برای Dispatch هست؟ API (Application Programming Interface) Session جواب داد (`agent_llm_session_failed_requests_total`)؟ TTS (Text-to-Speech) در دسترس؟
`call_id="unknown"` در Summaryها	SIP (Session Initiation Protocol) Headers (۱۲)	`X-originalCallID` در SBC (Session Border Controller)/Proxy حذف می‌شود.
«امکان انتقال به کارشناس … وجود ندارد»	Redirect	`Via` / `X-channelId` رسیده؟ sip-messager کد `202` گرفت؟ سوئیچ پیام MESSAGE را می‌شناسد؟

۷. ظرفیت، HA (High Availability) و Monitoring

```
500 concurrent calls
 = 500 SIP sessions (trunk channels)        ← SBC licence / trunk max channels
 = 500 RTP streams × 2 directions           ← ~44 Mbps each way with G.711
 = 500 LiveKit rooms + 500 agent jobs       ← worker processes, CPU (VAD per call)
 = 500 streaming STT sessions               ← STT throughput (GPU)
 = 500 concurrent aiva sessions             ← LLM tokens/s (GPU), Qdrant QPS
 = 500 streaming TTS sessions               ← TTS throughput (GPU)
 + 3 egress recordings per call (participant, agent, room if enabled) → MinIO
```

TELECOM bandwidth estimate CODE python/chatbot-iva-agent/src/entrypoint.py:178-180 · src/config.py:134-143

HA (High Availability): حداقل دو Node IVA (Intelligent Virtual Assistant) پشت SBC (Session Border Controller) و در حالت ایده‌آل دو دیتاسنتر (Operator → DC-A / DC-B). اگر Node 1 از کار بیفتد، تماس‌های جدید به Node 2 می‌روند؛ **تماس‌های در جریان روی Node خراب قطع می‌شوند** (وضعیت مکالمه در حافظه Process است). در Deploy عادی، Worker تا `AGENT_DRAIN_TIMEOUT=1800s` صبر می‌کند تا تماس‌های جاری تمام شوند؛ `AGENT_MAX_UNRECOVERABLE_ERRORS=3` Session را پس از سه خطای غیرقابل‌بازیابی می‌بندد.

CODE python/chatbot-iva-agent/src/config.py:48,51 · src/main.py:367 INFERENCE in-flight calls lost on node crash (no session state externalised)

لایه Monitoring	Metric لازم	در Phoenix
Network	Link، Loss، Latency، Bandwidth	بیرونی
SIP (Session Initiation Protocol)	Concurrent Calls، Call Success Rate (ASR (Answer-Seizure Ratio)، مدت)، SIP (Session Initiation Protocol) Error Rate تماس	بیرونی (SBC (Session Border Controller) / LiveKit (SIP (Session Initiation Protocol))
Media	Packet Loss، Jitter RTP (Real-time Transport Protocol)، MOS (Mean Opinion Score)	بیرونی
Application	Session، Redirect، Exit، User Away، Kafka، Egress شروع	`agent_sip_redirect_requests_total`، `agent_sip_exit_requests_total`، `agent_user_away_total`، `agent_kafka_messages_delivered_total`، `agent_egress_requests_total`
AI (Artificial Intelligence)	Error، TTFT و LLM/TTS Latency/STT (Speech-to-Text)	`agent_stt_*`، `agent_llm_ttft_seconds`، `agent_llm_duration_seconds`، `agent_tts_*`؛ Failed Requests، APM Spans (Application Performance Monitoring) (user_speaking → agent_thinking → agent_speaking)

CODE python/chatbot-iva-agent/src/utils/prometheus_metrics.py:30-260 · src/main.py:200-235 · src/config.py:49 (port 9400)

بودجه Latency — مثال:

```
User stops speaking
   ↓ endpointing   500 ms   (AGENT_MIN_ENDPOINTING_DELAY)
   ↓ STT           500 ms
   ↓ LLM first sentence 800 ms
   ↓ TTS first chunk    400 ms
   ≈ 2.2 s to first audio   (+ network/RTP ≈ 50-150 ms each way)
```

INFERENCE illustrative numbers; measure with the metrics above

۸. برای TPM (Technical Product Manager) — «الان در کدام لایه هستیم؟»

اگر در جلسه گفتند **«سرشماره گرفته شده ولی هنوز قابل استفاده نیست»**، کار تقریباً تمام نیست. بپرسید: **«الان در کدام لایه هستیم؟»** و وضعیت را این‌طور ثبت کنید:

```
Number allocated ✅ → Routing ⏳ → Fiber ⏳ → IP connectivity ⏳ → SBC ⏳
→ SIP trunk ⏳ → Headers verified ⏳ → IVA integration ⏳ → E2E test ⏳
→ Load test ⏳ → HA ⏳ → Monitoring ⏳ → Pilot ⏳ → Production
```

لایه	سؤالی که باید بپرسید	شاهد «تمام شد» که بخواهید
سرشماره / Routing	تماس از سیم‌کارت واقعی به کجا می‌رسد؟ Trunk Group مقصد تعریف شده؟	Log INVITE در SBC (Session Border Controller) با شماره مقصد
فیبر / Backbone	فیبر از کجا به کجا؟ Redundant است؟ چه کسی مالک هر سر آن است؟	Link Up + نمودار مسیر
IP (Internet Protocol) / Firewall	رنج پورت RTP (Real-time Transport Protocol) باز شده؟ QoS (Quality of Service) علامت‌گذاری می‌شود؟	تست پورت دوطرفه، نه فقط Ping
SBC (Session Border Controller) / SIP (Session Initiation Protocol) Trunk	سقف کانال هم‌زمان چند است؟ Codec چیست؟ Headerهای X-* عبور می‌کنند؟	OPTIONS 200 + Capture یک INVITE کامل
اتصال IVA (Intelligent Virtual Assistant)	Dispatch Rule LiveKit کدام Agent را صدا می‌زند؟ چند Worker داریم؟	یک تماس با خوش‌آمد کامل و Summary در Kafka با call_id واقعی
انتقال به کارشناس	سوئیچ پیام MESSAGE با X-Destination را پیاده کرده؟ صف 6040 درست است؟	یک انتقال موفق End-to-End
بار / HA (High Availability)	تست بار با SIP (Session Initiation Protocol) انجام شده یا فقط WebSocket؟ Node دوم کجاست؟	گزارش SIPp در عدد Peak

قاعده تعهد Deadline: هر لایه بیرونی (گام ۳ تا ۱۳) یک Dependency با مالک مشخص است. طبق قاعده فصل ۸، Deadline تیم IVA (Intelligent Virtual Assistant) را به‌صورت «`T + n` روز کاری از زمان سبز شدن گام ۱۲ (Headerها تأیید شد)» بنویسید، نه تاریخ تقویمی.

۹. RAG (Retrieval-Augmented Generation) یا Fine-tuning — جایگاه هرکدام در Phoenix

RAG (Retrieval-Augmented Generation) دانش موردنیاز را در زمان Inference از منابع بیرونی به مدل می‌دهد؛ **Fine-tuning** پارامترهای مدل را با داده آموزشی تغییر می‌دهد تا رفتار یا توانایی خاصی را بهتر یاد بگیرد. Fine-tuning راه واردکردن دانش متغیر نیست — «لیست قیمت بسته‌ها را Fine-tune کنیم» معمولاً ایده بدی است چون قیمت‌ها عوض می‌شوند.

نیاز	راه‌حل	در Phoenix
اطلاعات به‌روز شرکت (بسته‌ها، قیمت، رویه‌ها)	RAG (Retrieval-Augmented Generation)	همین کار انجام می‌شود: Top-10 از Qdrant `aiva-contexts` با Similarity ≥ ۰٫۵ و ویرایش KB (Knowledge Base) از `adcs-knowledgebase`
داده محرمانه و متغیر سازمان	RAG (Retrieval-Augmented Generation)	همان مسیر؛ تغییر دانش بدون آموزش مجدد

نیاز	راه‌حل	در Phoenix
لحن، فرمت و استاندارد پاسخ‌گویی	Fine-tuning (یا Prompt)	امروز با System Prompt و نقش‌های LLM (Large Language Model) قابل‌تنظیم انجام می‌شود
یادگیری یک Task خاص	Fine-tuning	کاندید منطقی: مدل STT (Speech-to-Text) روی صدای ۸ kHz فارسی تلفنی `INFERENCE`

`DOC` `python/aiva/docs/RequestFlow.md` `INFERENCE` `grep "fine-tun|lora|peft" over python/{aiva,agentlib,adcs} → only python/aiva/docs/aiva-runtime-architecture.html; no training code in repo`

جمله آماده برای جلسه: «RAG (Retrieval-Augmented Generation) دانش را در لحظه پاسخ از Knowledge Base به مدل می‌دهد و Fine-tuning رفتار مدل را با آموزش تغییر می‌دهد؛ در Phoenix دانش از RAG می‌آید و هیچ مدلی Fine-tune نشده است.»

۱۱. نقشه Repository و Tech Stack

`phoenix` یک Monorepo چندزبانه روی GitLab است (`git.mci.dev:mse/sre/phoenix`). هر پوشه زبان، CI (Continuous Integration) مخصوص خودش را دارد که در `.gitlab-ci.yml` ریشه `include` می‌شود.

پوشه	محتوا
`python/`	۶۰+ پروژه؛ ۱۰ پروژه فعال: `coa-api`، `agentlib`، `aiva`، `adcs`، `aqa`، `chatbot-iva-agent`، `adcs-knowledgebase`، `adcs-auth`، `fastlib`، `aiva-knowledgebase`
`golang/`	`grogu`، `neurobox-gateway`، `websocall`، `dobby`، `sip-messager`، `voice-segmenter`، `chatbot-controller`، ...
`rust/`	`vad` (gRPC Streaming، VAD (Voice Activity Detection)، `qdrant-backup`
`javascript/`	`coa-ui` (Dashboard CCA (Call Center Analyze)، `websocall-ui`
`cpp/`	`envoy`، `rtpengine` (مسیر رسانه)
`helm/`، `ansible-roles/`	Chart مشترک `helm/cloud`؛ Roleهای monitoring، ingress، metallb، keepalived، envoy edge/relay، k3s stack
`dockerfiles/`	Imageهای کمکی (backup، livekit-cli، mlflow، tritonserver-torch...)

CODE repo root listing (ls, find -maxdepth 2) · .gitlab-ci.yml:2-10 (includes)
BLAME .gitlab-ci.yml:2 → a.salimi · 2023-02-22 · b358582d8 Initial commit

Tech Stack

لایه	فناوری
Python Runtime	Python 3.13 (Voice Agent: 3.12)، Package Manager: `uv`، Lint: `ruff`، Type: `pyright`

CODE python/coa-api/Dockerfile:1 · python/chatbot-iva-agent/Dockerfile:1 · python/*/ruff.toml · python/*/pyproject.toml
BLAME Dockerfile:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0
BLAME Dockerfile:1 → y.fallah · 2025-06-02 · da3ce5abe feat(chatbot-iva): add chatbot-iva

Web Frameworks	Django + DRF (Django REST Framework) (`coa-api`، `aiva`، `adcs`، `adcs-auth`)؛ FastAPI (`aqa`، `fastlib`)؛ Gin در Go

CODE python/coa-api/manage.py · python/aqa/aqa/api/app.py · golang/websocall/main.go:15
ADDED manage.py → maghsood95 · 2023-05-28 · 2d5b60bc7 incidentmanager project added to phoneix
ADDED app.py → Hamed Mahmoudkhani · 2026-08-15 · ba39ad73e feat(AQA): fastapi project structure
BLAME main.go:15 → h.mahmoudkhani · 2026-07-04 · badc74c47 feat: websocall (v0.1.0)

لایه	فناوری
Realtime Voice	LiveKit Agents، Silero VAD (Voice Activity Detection)، Kamailio، rtpengine، Hedra (Avatar) CODE python/chatbot-iva-agent/src/main.py:37 (silero) · python/chatbot-iva-agent/src/entrypoint.py:15 (hedra) · cpp/rtpengine BOARD IVA board (Kamailio) BLAME main.py:37 → y.fallah · 2025-06-21 · 6213f04ac feat(chatbot-iva): add turn-detection. v0.6.0 BLAME entrypoint.py:15 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat... LAST cpp/rtpengine/ → AliDotS · 2026-08-05 · 26954a754 fix(rtpengine): add missing env to ci
AI (Artificial Intelligence)	مدل‌ها از Gateway سازگار با OpenAI: `openai/gpt-oss-120b`، `google/gemma-4-31B-it`، Embedding `text-embedding-3-large` یا Hakim؛ TTS/ASR (Automatic Speech Recognition) روی Triton از `neurobox-gateway` CODE python/aiva/aiva/chat/config.py:32,37,174 · golang/neurobox-gateway/internal/triton/ BLAME config.py:32 → a.salimi · 2026-09-05 · 8a06d3d2f feat(aiva): add guadraril retrieval BLAME config.py:37 → Hamed Mahmoudkhani · 2026-07-25 · 7cf4b2f4a refactor(aiva): use adcs retriever BLAME config.py:174 → a.salimi · 2026-09-28 · 638179e33 feat(aiva): add voice bridge, classification and clarification LAST golang/neurobox-gateway/internal/triton/ → a.razghandi · 2026-07-22 · f02b41c42 feat(ngw): implement round-robin load balancing for gRPC v3.4.0
Data	PostgreSQL، ClickHouse، Qdrant (gRPC)، MinIO (S3 (Simple Storage Service))، Redis (در برخی سرویس‌ها) CODE python/coa-api/coa/config.py · python/aiva/aiva/retrieve/qdrant.py · python/adcs/adcs/auth/utils/blacklist.py (Redis) ADDED config.py → h.mahmoudkhani · 2024-08-26 · 7899c5e66 Sentiment analysis project structure (v0.0.1) ADDED qdrant.py → h.mahmoudkhani · 2026-04-28 · 2c586fccd feat(adcs): custom retriever changes ADDED blacklist.py → pooria · 2026-07-12 · 91556e3f2 feat(adcs): check cache for access token blacklisted
Messaging	Kafka (Producer Idempotent در Sarama ،Python در Go) CODE python/chatbot-iva-agent/src/utils/kafka.py:21 · python/chatbot-iva-agent/src/config.py:305 · golang/grogu (sarama) BLAME kafka.py:21 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat... BLAME config.py:305 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) LAST golang/grogu/ → a.abolghasemi · 2026-09-27 · 6afc0c82c feat(grogu): adding flag for enable and disable account overview v0...
Async Jobs	Celery و Django Management Commandهای Daemon CODE python/coa-api/coa/celery.py · python/coa-api/entrypoint_*.sh ADDED celery.py → h.mahmoudkhani · 2024-10-06 · 6b76c24f0 Sentiment Analysis: v1 (v0.1.0)

لایه	فناوری
Observability	Prometheus (`django_prometheus`)، Elastic APM (Application Performance Monitoring)، Phoenix/OpenTelemetry، Sentry (در `fastlib`)
Deploy	Docker (Base: `astral-sh/uv:python3.13-bookworm-slim`) روی k3s/Kubernetes، Helm

CODE python/coa-api/coa/urls.py (django_prometheus) · python/chatbot-iva-agent/src/utils/apm.py **DOC** python/aqa/README.md (Phoenix) · python/fastlib/README.md:76 (Sentry)

ADDED urls.py → p.safaei · 2024-06-08 · 8b78eefd3 [chatbot-django/Library] Chatbot_lib django to main

ADDED apm.py → y.fallah · 2026-06-03 · 1fb743b0b feat(chatbot-iva-agent): add support for phone number feed in creat...

CODE python/coa-api/Dockerfile:1 · helm/cloud/templates/

BLAME Dockerfile:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0

LAST helm/cloud/templates/ → h.mahmoudkhani · 2026-08-16 · 7db6eb9ee fix(helm): empty checksums for deployments without configmap

فلسفه Tech Stack — هر زبان، Database و سرویس: چرا، کی، و چه کار دیگری

بخش ۱۱ می‌گوید **چه چیزی** در Repository هست؛ این فصل می‌گوید **چرا** هست، **کی** باید از آن استفاده کرد، **کی نباید**، و چه قابلیت‌هایی از همین ابزارها هنوز استفاده نشده و می‌تواند به Feature محصول تبدیل شود. هدف این است که یک CTO (Chief Technology Officer) یا TPM (Technical Program/Product Manager) بتواند بدون خواندن Code، درباره انتخاب ابزار، هزینه و Roadmap تصمیم بگیرد.

> **قاعده برچسب‌ها در این فصل:** ردیف «در Phoenix» از Code آمده است (`CODE`). ردیف «چه کار دیگری می‌توان کرد» قابلیت رسمی و مستند خود ابزار است، **نه** چیزی که در Code ما وجود دارد؛ برای همین با `INFERENCE` علامت خورده و قبل از تعهد باید با تیم مهندسی اعتبارسنجی شود. ابزارهایی که آخرین Commit آن‌ها قبل از ۲۰۲۶ است با برچسب `Legacy/Idle` مشخص شده‌اند.

اصل کلی: Polyglot از روی ضرورت، نه سلیقه

Phoenix چندزبانه است چون سه نوع کار کاملاً متفاوت انجام می‌دهد: (۱) **منطق هوش مصنوعی** که سرعت تغییرش بالاست و اکوسیستمش در Python است؛ (۲) **Pipeline و Gateway** که باید هزاران I/O هم‌زمان را با Memory کم مدیریت کند — جای Go؛ (۳) **پردازش صوت فریم‌به‌فریم** که Latency آن باید قابل پیش‌بینی و بدون مکث Garbage Collector باشد — جای Rust. هر زبان اضافه هزینه دارد (Hiring، CI، Library تکراری، دانش پراکنده)؛ پس قاعده این است که زبان جدید فقط وقتی وارد شود که زبان فعلی با **اندازه‌گیری** (Profile، Load test) شکست خورده باشد — همان کاری که برای `rust/vad` با Load benchmark انجام شده است.

`DOC` `rust/vad/README.md (gRPC load benchmark, 16→32 sessions)` `INFERENCE` `principle`

ماتریس انتخاب زبان

معیار	Python	Go	Rust
اکوسیستم AI/LLM	بهترین LiveKit Agents، OpenAI SDK، Torch، Transformers	متوسط Clientها هستند، مدل‌سازی نه	محدود ONNX Runtime، Candle
هم‌زمانی I/O (Concurrency)	متوسط asyncio؛ GIL برای CPU	بهترین Goroutine ارزان، Channel	عالی Tokio؛ ولی پیچیده‌تر
Latency قابل پیش‌بینی	ضعیف	خوب GC کوتاه ولی هست	بهترین بدون GC
سرعت توسعه و تغییر	بهترین	خوب	کند Borrow checker، Compile طولانی
Footprint و Deploy	Image سنگین (Torch چند GB)	یک Binary استاتیک، چند ده MB	یک Binary استاتیک، کوچک
بازار نیروی کار در ایران	گسترده	خوب و رو به رشد	کمیاب — ریسک Bus factor

معیار	Python	Go	Rust
کی انتخاب کنیم	Orchestration مدل، Prompt، RAG، Agent، Django Admin/CRUD، Data science، هر چیزی که هفته‌ای عوض می‌شود	Consumer کافکا، Gateway، Proxy، پروتکل شبکه (SIP)، Kubernetes Operator، سرویس پرترافیک و Stateless	Hot path محاسباتی روی هر فریم صوت، کتابخانه‌ای که Memory safety حیاتی است، ابزار کنار Qdrant (که خودش Rust است)
در Phoenix	`aiva` ، `agentlib` ، `chatbot-iva` ، `agent` ، `coa-api` ، `aqa` ، `adcs*`	`grogu` ، `neurobox-gateway` ، `voice-segmenter` ، `sip-messager` ، `websocall` ، `dobby` ، `log-operator` ، `pg-scaler`	`rust/vad` ، `rust/qdrant-backup`

`CODE` `python/*/pyproject.toml · golang/*/go.mod · rust/*/Cargo.toml` `INFERENCE` `ratings are engineering judgement, not measurements`

هر زبان در عمق

زبان	چرا (فلسفه)	کی نه / ریسک
Python	زبان مادری AI است: LiveKit Agents SDK اول برای Python منتشر می‌شود، و Torch، Transformers، Silero و OpenAI SDK همه Python هستند. Django برای سرویس‌هایی که Admin، ORM، Migration و Auth آماده می‌خواهند (`coa-api` ، `aiva` ، `adcs`)؛ FastAPI برای API سبک و Async با Streaming/SSE (`aqa` ، `fastlib`). ابزار: `uv` (نصب سریع و قفل وابستگی)، `ruff` ، `pyright` .	کار CPU-bound (GIL)، هزاران Connection هم‌زمان، و Imageهای حجیم: `chatbot-iva-agent` به‌خاطر TensorFlow/Torch/ONNX چند GB است. Celery در `coa-api` با `concurrency=1` اجرا می‌شود.
Go	برای «لوله‌کشی» سیستم: خواندن Kafka، دانلود از MinIO، صدا زدن ASR با (`gobreaker`)، Circuit breaker (`ants`)، Worker pool Retry. یک Binary کوچک، شروع سریع، مصرف Memory کم؛ برای Kubernetes Operator هم زبان رسمی (`controller-runtime`) است. `dobby` نسخه Go ابزارهای `aiva.tools` است که به‌صورت MCP (Model Context Protocol) Server در دسترس هر LLM Client قرار می‌گیرد.	مدل‌سازی ML، Notebook و تحلیل داده. ریسک: `dobby` و `aiva.tools` «رفتار یکسان» دارند — یعنی هر تغییر ابزار باید در دو Codebase انجام شود.
Rust	`rust/vad` هر ۱۰۰ میلی‌ثانیه صوت را از gRPC Stream می‌گیرد و Silero را با ONNX اجرا می‌کند؛ اینجا یک مکث GC می‌تواند Turn-taking را خراب کند. بدون GC، Latency یکنواخت است و هر Pod Session بیشتری نگه می‌دارد. `qdrant-backup` هم با Client رسمی Rust خود Qdrant نوشته شده است.	سرعت توسعه پایین و نیروی کمیاب. فقط برای Hot path اندازه‌گیری‌شده؛ نه برای CRUD یا منطق کسب‌وکار.
C / C++	Code ما نیست؛ ابزارهای زیرساختی که به آن‌ها Patch یا Extension اضافه کرده‌ایم: `rtpengine` (Relay رسانه RTP) و Extensionهای Envoy (انتخاب تصادفی IP مبدأ برای DNS و Connection، Build با Bazel).	`cpp/envoy` از ۲۰۲۴ تغییر نکرده **Legacy/Idle** ؛ Build آن به VPN و Bazel Cache وابسته است — دانش آن در معرض فراموشی است.

زبان	چرا (فلسفه)	کی نه / ریسک
TypeScript / JS	Next.js + React + Tailwind برای `coa-ui` (Dashboard تحلیل تماس) و `websocall-ui` ؛ `static/javascribe` یک صفحه ساده Nginx.	—
Java	پروژه `java/lake` با Data lake — Cassandra قدیمی.	آخرین Commit دسامبر ۲۰۲۳ **Legacy/Idle** ؛ نباید Feature جدیدی روی آن بنا شود.
Bash / Ansible / Helm	زیرساخت به‌صورت Code: ساخت VM (Packer/vSphere)، نصب k3s، شبکه و Storage با Ansible Role؛ Deploy سرویس‌ها با Helm Chart مشترک `helm/cloud` .	—

CODE golang/grogu/go.mod (ants, gobreaker, sarama, minio-go) · golang/dobby/README.md («Go port of the Python aiva.tools package, kept behaviorally identical») · rust/vad/src/streaming_vad.rs:3 (SileroVad) · rust/vad/Cargo.toml:11 · rust/qdrant-backup/Cargo.toml · cpp/envoy/README.md · javascript/coa-ui/package.json GIT last commit java/ 2023-12-19 · cpp/envoy/ 2024-03-18

Databaseها و Storage

قاعده ساده: **هر نوع داده جای خودش را دارد.** تراکنش و رابطه → PostgreSQL؛ تحلیل روی میلیون‌ها ردیف → ClickHouse؛ جستجوی معنایی → Qdrant؛ داده موقت و سریع → Redis؛ فایل (صوت، Backup) → MinIO.

PostgreSQL — منبع حقیقت تراکنشی

چیست	Database رابطه‌ای با ACID، Index غنی، JSONB و Extension.
فلسفه انتخاب	پیش‌فرض امن: هر داده‌ای که باید درست، یکتا و قابل Join باشد (کاربر، نقش، تماس، Annotation، Trend، Tag) اینجا می‌رود. Django و GORM هر دو آن را بومی پشتیبانی می‌کنند.
در Phoenix	Schema مشترک `coa-api` و `grogu` (جدول‌های `calls` ، `call_records` ، `call_ai_tags`)؛ `aiva` ، `adcs` ، `aqa` . **Broker سلری `coa-api` هم خود Postgres است** (`sqla+postgresql`). HA با Stolon، `postgres-exporter` برای Metric، `archive_old_calls` با `pg_dump` به MinIO، و یک Operator آزمایشی `pg-scaler` .
چه کار دیگری می‌توان کرد INFERENCE	**Partitioning** جدول `calls` بر اساس ماه به‌جای حذف دستی — آرشیو یعنی Detach یک Partition؛ **Row-Level Security** برای جدا کردن داده HR از داده مشتری در یک Database (Multi-tenant)؛ **pgvector** برای Collectionهای کوچک به‌جای یک سرویس جدا؛ **Logical replication / CDC** (Debezium) برای فرستادن تغییرات به Kafka بدون Code اضافه؛ **LISTEN/NOTIFY** برای رویدادهای سبک؛ **Full-text search** روی متن Ticketها.
کی نه / ریسک	Aggregation سنگین روی میلیون‌ها تماس (کار ClickHouse است)؛ Postgres به‌عنوان Celery broker در مقیاس بالا Polling زیادی ایجاد می‌کند. اشتراک یک Schema بین دو سرویس (`coa-api` و `grogu`) یعنی هر Migration باید هماهنگ شود.

CODE python/coa-api/coa/settings.py:239-243 (CELERY_BROKER_URL = sqla+postgresql) · ansible-roles/monitoring_stack (stolon:master-pg14, postgres-exporter) · golang/pg-scaler (controller-runtime) · golang/grogu/go.mod (gorm postgres)

ClickHouse — موتور تحلیل (OLAP)

چیست	Database ستونی؛ فقط ستون‌های لازم خوانده و فشرده می‌شوند، پس Aggregation روی صدها میلیون ردیف در حد ثانیه است.
فلسفه انتخاب	تحلیل تماس (Topic، Sentiment، رضایت به تفکیک استان و ساعت) Query خواندنی و تجمیعی است، نه تراکنشی. Postgres در این حجم کند و گران می‌شود.
در Phoenix	خروجی تحلیل AI تماس‌ها در ClickHouse است؛ `coa-api` با Daemonهای `clickhouse_sync` و `clickhouse_update_sync` آن را به Postgres کپی می‌کند؛ `aqa` سؤال زبان طبیعی را به SQL روی آن تبدیل می‌کند؛ `adcs` هم Client آن را دارد.
چه کار دیگری می‌توان کرد `INFERENCE`	**Kafka table engine**: ClickHouse مستقیم از Kafka بخواند — بدون Consumer سفارشی؛ **Materialized View**: Dashboard و Trend در لحظه محاسبه شود، نه با Batch؛ **TTL**: Retention به‌صورت Declarative (مثلاً حذف Transcript بعد از N ماه — مستقیماً برای ردیف‌های Retention در Fit-Gap)؛ **S3 table function**: Query مستقیم روی فایل‌های Parquet در MinIO؛ توابع تقریبی (`uniq` ، `quantile`) برای Trend؛ توابع فاصله برداری (`cosineDistance`) برای خوشه‌بندی Topicها.
کی نه / ریسک	Update و Delete تک‌ردیفی (کند و Async است)، تراکنش، و Join پیچیده. **ریسک معماری:** کپی کردن ClickHouse به Postgres برای Dashboard یعنی دو منبع حقیقت و چند Daemon برای هماهنگی؛ Dashboard می‌تواند مستقیم از ClickHouse بخواند.

`CODE` `python/coa-api/pyproject.toml (clickhouse-connect) · python/aqa/pyproject.toml · python/adcs/pyproject.toml · python/coa-api/entrypoint_*.sh (clickhouse_sync, clickhouse_update_sync)`

Qdrant — حافظه معنایی (Vector Database)

چیست	Vector DB نوشته‌شده با Rust؛ Embedding را با Index از نوع HNSW نگه می‌دارد و «نزدیک‌ترین معنا» را همراه با فیلتر روی Payload برمی‌گرداند.
فلسفه انتخاب	Self-hosted (داده و مدل داخل شبکه می‌ماند، بدون وابستگی به Cloud خارجی)، gRPC سریع، فیلتر Payload قوی، و Snapshot. برای RAG (Retrieval-Augmented Generation)، Guardrail و Classification بر اساس مثال.
در Phoenix	Collectionهای `aiva-contexts` (دانش، Top-10 با شباهت ≥ ۰٫۵)، `guardrail` ، `aiva-instructions` ، `aiva-jokes` ، `aiva-classification` ؛ یک Embedding به پنج Retrieval پخش می‌شود. Backup با ابزار سفارشی `rust/qdrant-backup` (Avro).
چه کار دیگری می‌توان کرد `INFERENCE`	**Hybrid search** (Dense + Sparse/BM25 با Fusion از نوع RRF) — برای فارسی که کلمه کلیدی دقیق (نام طرح، کد تعرفه) مهم است تأثیر مستقیم بر دقت دارد؛ **Quantization** (Scalar/Binary) برای کاهش ۴ تا ۳۲ برابری RAM؛ **Multi-tenancy** با Payload index (یک Collection برای IVA، HR و مشتری با فیلتر `tenant`)؛ **Recommendation/Discovery API** (مثال مثبت و منفی)؛ **Semantic cache** برای پاسخ‌های تکراری LLM (کاهش هزینه هر تماس)؛ خوشه‌بندی سؤال‌های بی‌پاسخ برای پیشنهاد محتوای جدید به Knowledge Base؛ **Snapshot بومی** به‌جای ابزار Backup سفارشی.
کی نه / ریسک	داده ساختاریافته و گزارش (کار Postgres/ClickHouse). تغییر مدل Embedding یعنی Re-index کامل همه Collectionها — باید در Roadmap دیده شود.

`CODE` `python/aiva/aiva/chat/config.py:70,74,77,79,178 · python/aiva/aiva/retrieve/qdrant.py · rust/qdrant-backup/Cargo.toml (qdrant-client, apache-avro)`

Redis — حافظه موقت و سریع

فلسفه	داده‌ای که از دست رفتنش فاجعه نیست ولی باید در حد میکروثانیه خوانده شود: Blacklist، Cache توکن، شمارنده Rate limit، صف سبک.
در Phoenix	Blacklist توکن JWT در `adcs` و `adcs-auth`؛ Django cache؛ Broker سلری در `adcs-knowledgebase` و `aiva-knowledgebase`؛ Cache داده MyMCI در `grogu`؛ **Rate limit ابزارها در** `dobby` با `redis_rate`.
چه کار دیگری `INFERENCE`	Rate limit برای `send_sms` در `aiva` (الگوی آن در `dobby` آماده است)؛ **Redis Streams** و Pub/Sub برای Dashboard زنده (تعداد تماس فعال)؛ Distributed lock برای Daemonهای Sync؛ State مکالمه برای Failover بین Workerها.
کی نه	داده ماندگار و تنها نسخه؛ صف با تضمین تحویل و Replay (کار Kafka).

`CODE` `python/adcs/adcs/auth/utils/blacklist.py` · `python/adcs-auth/adcs_auth/jwtoken/blacklist.py` · `python/adcs-knowledgebase/adcs_kb/settings.py:189` · `golang/grogu/internal/cache/service.go` · `golang/dobby/internal/toolrl/toolrl.go:15,46`

MinIO — Object Storage سازگار با S3

فلسفه	فایل بزرگ (صوت، ویدئو، Dump) جای Database نیست. MinIO همان API آمازون S3 را داخل شبکه می‌دهد؛ پس هر ابزار سازگار با S3 (Egress لایوکیت، ClickHouse، django-storages، boto3) بدون تغییر کار می‌کند.
در Phoenix	Bucket `chatbot-mci-iva` برای ضبط Egress؛ `grogu` صوت را برای ASR دانلود می‌کند؛ `archive_old_calls` فایل `pg_dump` را آپلود می‌کند.
چه کار دیگری `INFERENCE`	**Lifecycle (ILM)**: حذف یا انتقال خودکار صوت بعد از N روز — اجرای Retention بدون Code؛ **Object Lock / WORM**: فایل غیرقابل تغییر برای ممیزی و حقوقی؛ **Server-side encryption** برای صوت کارکنان (داده شخصی)؛ **Bucket notification به Kafka**: پردازش به محض آپلود، به‌جای انتظار برای پیام جدا؛ **Presigned URL**: پخش امن صوت در Dashboard بدون باز کردن Bucket؛ Versioning.
کی نه	Query روی محتوا یا داده‌ای که مدام تغییر می‌کند.

`CODE` `python/chatbot-iva-agent/src/config.py:124-138,158` · `python/chatbot-iva-agent/src/utils/egress_recorder.py:47` · `golang/grogu/go.mod (minio-go)`

سایر Storageها

ابزار	نقش در Phoenix و فلسفه	قابلیت استفاده‌نشده `INFERENCE`
Elasticsearch	مقصد Log در `log-operator` و Backend مربوط به Elastic APM. برای جستجوی متنی سریع روی Log.	جستجوی Full-text فارسی روی Transcript تماس‌ها برای تحلیلگر.
etcd	Key-value توزیع‌شده با اجماع (Raft) — مغز Kubernetes؛ در `chatbot-controller` هم Store است؛ Role `etcd_backup` و اسکریپت `bash/etcd-recovery`.	Leader election و Config پویا — فقط برای داده کوچک.
VictoriaMetrics	Time-series DB سازگار با Prometheus، با مصرف Memory و دیسک کمتر، همراه Grafana.	نگهداری Metric کسب‌وکار (هزینه هر تماس) کنار Metric فنی.
Cassandra	Wide-column برای نوشتن انبوه؛ فقط در `java/lake` **Legacy/Idle**.	—

ابزار	نقش در Phoenix و فلسفه	قابلیت استفاده‌نشده INFERENCE
BemiDB	Replica تحلیلی Postgres (ستونی/Iceberg)؛ یک Dockerfile آزمایشی از ۲۰۲۵ . Legacy/Idle	جایگزین سبک ClickHouse برای تحلیل روی داده Postgres.
SQLite	فقط در تست‌های Go (GORM).	—

CODE golang/log-operator/go.mod (go-elasticsearch) · golang/chatbot-controller/internal/store/store.go (etcd) · ansible-roles/monitoring_stack/templates/victoria_metrics.yaml.j2 · dockerfiles/bemidb/Dockerfile GIT last commit dockerfiles/bemidb 2025-05-06

Messaging و Async

Kafka — ستون فقرات رویدادها

چیست	Log توزیع‌شده و ماندگار؛ پیام بعد از خوانده شدن پاک نمی‌شود، پس چند Consumer group مستقل و **Replay** ممکن است.
فلسفه انتخاب	**جدا کردن مسیر تماس از مسیر تحلیل.** Agent صوتی نباید منتظر ClickHouse یا ASR بماند؛ خلاصه را در Kafka می‌گذارد و می‌رود. اگر `grogu` یک ساعت خاموش باشد، هیچ تماسی از دست نمی‌رود.
در Phoenix	Session summary تماس (`chatbot-mci-iva` Topic) IVA؛ مصرف آن در `aiva` برای Memory؛ ورود رکورد تماس TelC، تماس IVA و Ticket به `grogu` با Dead-letter؛ Analytics در `neurobox-gateway` ؛ ارسال Log در `log-operator` ؛ `crawler` . Producer Idempotent (Python)، Sarama با SCRAM (Go).
چه کار دیگری INFERENCE	**Replay برای مدل جدید:** وقتی ASR یا Classifier بهتر شد، کل تاریخچه را دوباره پردازش کنیم — یک ارزش مستقیم محصولی؛ **Schema Registry** (Avro/Protobuf) تا تغییر `SessionSummarySchema` Consumerها را نشکند؛ **Kafka Connect** (Source به Sink، ClickHouse/S3 از Postgres با Debezium) به‌جای Daemonهای Sync سفارشی؛ **Stream processing** (Flink/ksqlDB) برای تشخیص Trend و هشدار در لحظه؛ Compacted topic برای آخرین وضعیت هر تماس؛ Audit trail کامل (Event sourcing).
کی نه / جایگزین	Request/Response همزمان (gRPC/HTTP)؛ صف Task ساده با Retry (Celery کافی است)؛ حجم خیلی کم (RabbitMQ یا NATS ساده‌تر). Redpanda جایگزین سازگار با Kafka و بدون JVM است.

CODE python/chatbot-iva-agent/src/utils/kafka.py:21 · python/chatbot-iva-agent/src/config.py:315 · golang/grogu/internal/config/config.go:51-58 · python/aiva/aiva/memory/management/commands/aiva_consume_kafka.py · golang/neurobox-gateway/internal/analytics/

Celery — صف Task پس‌زمینه

برای کار طولانی که نباید Request کاربر را معطل کند (Import/Export Knowledge Base، کارهای `coa-api`). در `coa-api` Broker، Postgres و نتیجه در `django-db` است؛ در پروژه‌های Redis، KB Broker است. **قابلیت استفاده‌نشده:** `celery beat` برای زمان‌بندی، Chord/Group برای کار موازی. **کی نه:** Workflow چندروزه با Retry و Human-in-the-loop — آنجا Temporal مناسب‌تر است INFERENCE .

CODE python/coa-api/coa/settings.py:236-245 · python/coa-api/coa/celery.py

صوت و تماس Real-time

LiveKit — Room، Agent، SIP و Egress

چیست	یک SFU (Selective Forwarding Unit) برای WebRTC به‌همراه چهار جزء: **Room** (هر تماس یک Room با Participant و Track)، **Agents** (Worker پایتونی که مثل یک Participant وارد Room می‌شود و VAD/STT/LLM/TTS را اجرا می‌کند)، **SIP** (پل بین شبکه تلفن و Room) و **Egress/Ingress** (ضبط و ورود رسانه).
فلسفه انتخاب	Open-source و Self-hosted؛ کار سخت Real-time (Jitter buffer، حذف اکو، Reconnect، Codec) را حل کرده و ما فقط منطق Agent را می‌نویسیم. یک Agent واحد هم از تلفن (SIP) و هم از وب/موبایل (WebRTC) قابل دسترسی است.
در Phoenix — Room و Agent	`chatbot-iva-agent` با LiveKit Agents، Silero VAD، Turn detector و Avatar ویدئویی Hedra.
در Phoenix — SIP	**Inbound:** تماس از SBC/Kamailio به LiveKit SIP و یک Room می‌رسد؛ Identity تماس از SIP header. **Outbound:** ابزار `call_reminder` در `dobby` با `CreateSIPParticipant` به کاربر **زنگ می‌زند. انتقال به اپراتور:** نه با امکان Transfer خود LiveKit، بلکه با ارسال SIP MESSAGE از سرویس جدای `sip-messager`.
در Phoenix — Egress	ضبط Track جداگانه Caller و Agent به‌صورت OGG (و MP4 برای Avatar) در MinIO.
چه کار دیگری `INFERENCE`	**Outbound campaign** — نظرسنجی صوتی از کارکنان یا پیگیری Ticket (زیرساختش در `dobby` هست)؛ **Warm/Cold transfer بومی** (SIP REFER با `TransferSIPParticipant`) به‌جای سرویس جدا؛ **DTMF** برای منوی پشتیبان، کد تأیید یا رضایت صریح؛ **Supervisor** — اپراتور انسانی بی‌صدا وارد Room شود، گوش دهد یا دخالت کند (QA زنده)؛ **Handoff بین چند Agent** (Agent عمومی → Agent تخصصی HR)؛ **Data channel/RPC** برای نمایش Transcript زنده در Dashboard؛ **Room composite egress** و Stream زنده؛ **Ingress** (RTMP/WHIP) برای ورود صوت از منابع دیگر؛ **Webhook** رویدادهای Room برای Monitoring؛ پلاگین **Noise cancellation** (فعلاً فقط نصب شده). Pluginهای Deepgram و Cartesia هم در Dependencyها هستند ولی در Code صدا زده نمی‌شوند.
کی نه / ریسک	تست ساده یا Load test وب که WebRTC لازم ندارد: `websocall` (Go + WebSocket + STT/LLM/TTS سازگار با OpenAI) همین را بدون LiveKit انجام می‌دهد — ساده‌تر ولی بدون حذف اکو و Jitter buffer. LiveKit Server و SIP Service خود در این Repository نیستند (فقط Image `livekit-cli`).

`CODE` `python/chatbot-iva-agent/pyproject.toml (livekit-agents, plugins silero/turn-detector/hedra/noise-cancellation/deepgram/cartesia)` · `python/chatbot-iva-agent/src/entrypoint.py:15,102 (hedra)` · `golang/dobby/internal/tools/registry.go:90 (NewSIPClient)` · `golang/dobby/internal/tools/call_reminder.go:111 (CreateSIPParticipant)` · `golang/sip-messager/internal/api/api.go:80 (SIP MESSAGE)` · `python/chatbot-iva-agent/src/utils/egress_recorder.py:23-24 (.ogg/.mp4)` · `golang/websocall/internal/aiclient/clients.go:13-15` `INFERENCE` `deepgram/cartesia: grep finds no import in src/`

لبه تلفن: SBC، rtpengine، Kamailio و sipgo

ابزار	نقش	قابلیت استفاده‌نشده `INFERENCE`
Kamailio	SIP proxy/router: فقط Signaling (چه کسی به کجا وصل شود). روی Board آمده، در Repository نیست.	Load balancing بین چند LiveKit SIP (`dispatcher`)، مسیریابی بر اساس ساعت یا شماره، محافظت در برابر Flood (`pike`).

ابزار	نقش	قابلیت استفاده‌نشده INFERENCE
rtpengine	Relay رسانه RTP کنار Kamailio؛ صوت واقعی از آن عبور می‌کند (NAT، رمزنگاری SRTP).	Transcoding بین G.711 و Opus، ضبط در لبه، تشخیص DTMF.
sipgo (`sip-messager`)	Library پروتکل SIP در Go؛ پیام انتقال به اپراتور (Header `X-Destination`، پیش‌فرض `6040`).	یک SIP client کامل برای تست خودکار End-to-end تماس.

BOARD IVA board (Kamailio) CODE cpp/rtpengine · golang/sip-messager/go.mod (emiago/sipgo)
TELECOM roles of SIP proxy / RTP relay

FFmpeg — چاقوی سوئیسی صوت و ویدئو

فلسفه	هر تبدیل Format، Codec، Sample rate یا Channel به یک ابزار استاندارد و آزموده سپرده می‌شود، نه Code سفارشی.
در Phoenix	`grogu` دو فایل جدای Caller و Agent را با فیلترهای `pan` و `join` به یک فایل **Stereo** تبدیل می‌کند (Caller چپ، Agent راست) با خروجی MP3، ۸ کیلوهرتز، ۱۶ kbps؛ `voice-segmenter` صوت Stereo را Decode می‌کند؛ در Image `coa-api` و `ticket-upload` نصب است؛ تبدیل نمونه Benchmark در `rust/vad`.
چه کار دیگری INFERENCE	**Loudness normalization** (`loudnorm`) و **کاهش نویز** (`afftdn`، `arnndn`) قبل از ASR برای بهبود دقت؛ **حذف سکوت** (`silenceremove`) برای کاهش ثانیه‌های ارسالی به ASR = کاهش هزینه؛ **Redaction**: بی‌صدا یا بوق کردن بازه‌ای که شماره کارت یا نام گفته شده (با Timestamp از ASR) — برای حریم خصوصی کارکنان؛ **Opus** برای آرشیو (حجم چند برابر کمتر با کیفیت بهتر)؛ تصویر Waveform (`showwavespic`) برای Dashboard؛ برش (`segment`) برای پردازش موازی.
ریسک	اجرای Process خارجی با فایل موقت روی دیسک (I/O و پاکسازی)، مصرف CPU بالا در حجم زیاد.

CODE golang/grogu/internal/voice/merge.go:195-212 (pan, join, libmp3lame, b:a 16000, ar 8000) · golang/voice-segmenter/internal/api/handler.go:23-24,77 · python/coa-api/Dockerfile · python/ticket-upload/Dockerfile · rust/vad/README.md

فایل صوتی: Multitrack، Stereo، Mono و Codecها — کدام برای چه کاری؟

Channel یعنی یک «جریان» صوت مستقل داخل فایل. **Mono** یک Channel دارد و همه صداها در آن مخلوط‌اند. **Stereo** دو Channel (چپ و راست) دارد. در موسیقی دو Channel برای حس فضاست؛ در مرکز تماس روش رایج **Split-stereo** (یا Dual-channel) است: **هر گوینده روی یک Channel** — Caller چپ، Agent راست. **Multitrack** یعنی هر گوینده در یک فایل جدا ضبط شود؛ Egress در LiveKit همین کار را می‌کند.

نوع	مزیت	عیب	مناسب برای
Mono	کوچک‌ترین حجم؛ ورودی پیش‌فرض بیشتر مدل‌های ASR و VAD	صدای دو نفر مخلوط است؛ تفکیک گوینده (Diarization) با مدل لازم است و خطا دارد؛ صحبت هم‌زمان قابل تفکیک نیست	Stream زنده به ASR/VAD در طول تماس (`rust/vad` فقط ۱۶ کیلوهرتز Mono می‌پذیرد)؛ پیام صوتی تک‌نفره

نوع	مزیت	عیب	مناسب برای
Split-stereo	تفکیک گوینده **رایگان و بی‌خطا**؛ تشخیص هم‌پوشانی و قطع کلام؛ نسبت زمان صحبت؛ سکوت هر طرف؛ یک فایل واحد که پخش و نگهداری‌اش ساده است	دو برابر حجم Mono؛ دو Track باید از نظر زمانی هم‌تراز باشند	**تحلیل تماس پس از پایان** (CCA): Tagهای `overlap` ، رضایت و احوالپرسی دقیقاً به این تفکیک وابسته‌اند؛ ورودی `voice-segmenter`
Multitrack	بالاترین کیفیت و انعطاف؛ هر Track با Codec و Sample rate خودش؛ برای بیش از دو نفر (کنفرانس، Supervisor) هم کار می‌کند	چند فایل؛ Trackها لزوماً هم‌زمان شروع نمی‌شوند و برای هم‌ترازی Timestamp لازم است	ضبط خام اولیه (Egress)؛ تماس چندنفره؛ آموزش مدل

Sample rate (تعداد نمونه در ثانیه) تعیین می‌کند چه فرکانس‌هایی حفظ شوند (حداکثر = نصف Sample rate):

Sample rate	کجا	نکته
۸ کیلوهرتز (Narrowband)	شبکه تلفن سنتی، G.711	فرکانس بالای ۴ کیلوهرتز حذف می‌شود؛ حروف «س، ش، ف» به هم نزدیک می‌شوند و دقت ASR افت می‌کند
۱۶ کیلوهرتز (Wideband)	استاندارد ورودی ASR و VAD، تماس HD (G.722، AMR-WB)	نقطه بهینه برای گفتار
۲۴ تا ۴۸ کیلوهرتز	WebRTC/Opus، خروجی TTS، موسیقی	برای گفتار به ASR سودی ندارد؛ فقط حجم را بالا می‌برد

Codec و Container — حجم تقریبی برای یک ساعت صوت Mono:

Format	حجم/ساعت	ویژگی	مناسب برای
WAV / PCM ۱۶ بیت، ۱۶ کیلوهرتز	~۱۱۵ MB	بدون فشرده‌سازی و بی‌اتلاف؛ پردازش مستقیم بدون Decode	ورودی ASR/VAD، پردازش میانی
FLAC	~۵۰–۷۰ MB	فشرده و **بی‌اتلاف**	آرشیو برای آموزش مدل یا الزام حقوقی
G.711 (μ-law/A-law)	~۲۹ MB	Codec بومی تلفن، ۸ کیلوهرتز، ۶۴ kbps، Latency تقریباً صفر	انتقال روی SIP/RTP
Opus (در OGG/WebM)	~۷–۱۴ MB	بهترین Codec گفتار در Bitrate پایین (۱۶–۳۲ kbps)؛ Codec بومی WebRTC	ضبط Egress (همین OGG)، آرشیو بلندمدت، پخش در Dashboard
MP3	~۷ MB در ۱۶ kbps	پخش در همه جا؛ ولی برای گفتار در Bitrate پایین از Opus بدتر است	اشتراک و پخش عمومی

نکته برای Phoenix: مسیر فعلی این است: Egress دو فایل **OGG/Opus** جدا می‌سازد ← `grogu` آن‌ها را به یک **Stereo MP3، ۸ کیلوهرتز، ۱۶ kbps** (یعنی حدود ۸ kbps برای هر گوینده) تبدیل می‌کند ← ASR. این یک فشرده‌سازی اتلافی دوم روی اولی است، و Channel Agent که در اصل Wideband است به ۸ کیلوهرتز کاهش می‌یابد. اگر دقت ASR در تحلیل تماس مسئله است، اولین آزمایش ارزان این است که خروجی Merge به **PCM یا FLAC با ۱۶ کیلوهرتز** (برای ASR) یا **Opus** (برای آرشیو) تغییر کند و WER قبل و بعد مقایسه شود. Channel تماس‌گیرنده در هر حال از شبکه تلفن ۸ کیلوهرتز می‌آید و Upsample کیفیت آن را بالا نمی‌برد.

`CODE` golang/grogu/internal/voice/merge.go:195-212 · python/chatbot-iva-agent/src/utils/egress_recorder.py:23 `INFERENCE` ASR impact must be measured (WER) `TELECOM` G.711 = 8 kHz

هوش مصنوعی: Serving، Tracing و Tools

ابزار	فلسفه و نقش در Phoenix	قابلیت استفاده‌نشده `INFERENCE`
Gateway سازگار با OpenAI	همه LLMها (`gemma-4-31B-it`، `gpt-oss-120b`) و Embedding از یک پروتکل استاندارد صدا زده می‌شوند؛ تعویض مدل یعنی تغییر Config، نه Code. هفت نقش LLM در `aiva` مستقل قابل تنظیم‌اند.	A/B مدل‌ها روی ترافیک واقعی؛ Routing بر اساس هزینه (مدل کوچک برای Classification، بزرگ برای پاسخ نهایی).
NVIDIA Triton	Server استاندارد اجرای مدل روی GPU (ASR، TTS، Emotion، Gender) با gRPC. `neurobox-gateway` در Go جلوی آن است: Round-robin، Cache برای TTS، ارسال Analytics به Kafka.	**Dynamic batching** (Throughput بیشتر از همان GPU)؛ **Ensemble** (VAD ← ASR ← Emotion در یک فراخوانی)؛ چند نسخه هم‌زمان از یک مدل برای Canary؛ Model Analyzer برای تعیین تعداد GPU لازم؛ TensorRT.
Silero VAD	مدل کوچک ONNX برای تشخیص گفتار؛ در Agent پایتونی و در `rust/vad` (از طریق `wavekat-vad`).	یک سرویس VAD مشترک برای همه مسیرها به‌جای چند نسخه.
Arize Phoenix + OpenTelemetry	Tracing هر فراخوانی LLM (Prompt، Token، Latency) با OpenInference در `agentlib`، `aiva` و `aqa`.	**Dataset و Experiment و Eval** روی همین Traceها — جواب مستقیم به کمبود Eval (تنها Endpoint ارزیابی در `aqa` است)؛ Prompt playground؛ LLM-as-judge روی تماس‌های واقعی.
MLflow	Tracking آزمایش و Model registry؛ فقط یک Dockerfile از ۲۰۲۵. **Legacy/Idle**.	ثبت نسخه مدل‌های ASR/Classifier و نتایج WER به‌صورت قابل ردیابی.
MCP	پروتکل استاندارد ابزار برای LLM؛ `dobby` ابزارهای Phoenix را به هر MCP Client می‌دهد (Header اجباری `X-Phone-Number`).	ارائه ابزارها به تیم‌ها و محصولات دیگر (یک Platform داخلی)؛ مصرف MCP Serverهای سازمانی (CRM، HR) در `aiva`.
Chainlit، Plotly	UI گفتگو و نمودار برای `aqa` (تحلیل با زبان طبیعی).	—
Hedra	Avatar ویدئویی برای Agent.	کانال وب/کیوسک برای کارکنان.

`CODE` python/aiva/aiva/chat/config.py:32,37 · golang/neurobox-gateway/internal/triton/ · dockerfiles/tritonserver-torch/Dockerfile (tritonserver:25.08) · python/agentlib/pyproject.toml

(arize-phoenix-otel, openinference) · dockerfiles/mlflow/Dockerfile · golang/dobby/README.md · python/aqa/pyproject.toml (chainlit, plotly)

Platform، شبکه و Observability

ابزار	فلسفه و نقش	کی نه / نکته
Kubernetes (k3s) + Helm	k3s نسخه سبک Kubernetes برای On-premise؛ Chartهای مشترک `helm/cloud` و `helm/zarebin` (HPA، Probe، ServiceMonitor، Alert) تا هر سرویس فقط `values.yaml` بنویسد.	HPA نسخه ۲ آماده است؛ Autoscale بر اساس طول صف Kafka (KEDA) هنوز نیست **INFERENCE** .
Ansible، Packer، vSphere	ساخت تکرارپذیر VM و Cluster از صفر (هر Role نسخه دارد).	—
Cilium، rawfile-localpv، MetalLB، keepalived	CNI مبتنی بر eBPF؛ Storage محلی روی دیسک Node؛ IP سرویس روی Bare-metal؛ IP مجازی با Failover.	Storage محلی یعنی داده به Node گره خورده — Backup و Replication (Stolon، etcd_backup) حیاتی است.
Envoy	Proxy لبه و Relay (از جمله پشت CGNAT)، Load balancing برای gRPC (مثلاً `neurobox-vad-grpc-proxy`)، و Extension سفارشی.	Envoy ممکن است Sessionهای gRPC بلند را نامتوازن پخش کند (در Benchmark VAD دیده شده).
Xray	Tunnel/Proxy برای اتصال خروجی سرویس‌ها به سرویس‌های خارجی.	وابستگی عملیاتی و حقوقی — باید Owner مشخص داشته باشد.
cert-manager، certbot، Ingress	صدور و تمدید خودکار TLS و ورودی HTTP.	—
Prometheus client + VictoriaMetrics + Grafana	Metric فنی و کسب‌وکار (مثلاً `tool_call_*`، Redirect به اپراتور).	**سه سیستم Tracing همزمان** (Elastic APM، OpenTelemetry/Phoenix، Sentry) — یکپارچه‌سازی روی OpenTelemetry هزینه نگهداری را کم می‌کند **INFERENCE** .
Elastic APM، Sentry	Trace درخواست‌ها و ثبت خطا.	
log-operator	Operator سفارشی با CRD به نام `Harvest` برای جمع‌آوری Log از stdout و فایل Pod به Kafka/Elastic.	
gRPC / Protobuf	قرارداد تایپ‌شده و Streaming دوطرفه؛ برای VAD، Triton و Qdrant — هر جا Stream صوت یا Latency پایین مهم است.	برای API عمومی و Dashboard، REST/JSON ساده‌تر است.

CODE ansible-roles/ (k3s, cni, csi, metallb, keepalived, envoy_edge_cgnat, envoy_relay, xray_edge, cert_manager, certbot, monitoring_stack, packer, vsphere_vm) · helm/zarebin/CHANGELOG · golang/log-operator/README.md · rust/vad/README.md (Envoy uneven distribution) · python/fastlib/README.md:76 (Sentry)

هم‌پوشانی‌ها و فرصت‌های ساده‌سازی

این‌ها Bug نیستند، ولی هر کدام هزینه نگهداری دائمی دارند و برای Roadmap فنی ارزش بحث دارند **INFERENCE** :

هم‌پوشانی	پیشنهاد
ابزارها در `aiva.tools` (Python) و `dobby` (Go)	`aiva` ابزارها را از `dobby` با MCP مصرف کند؛ یک Codebase.
کپی Postgres ← ClickHouse با چند Daemon	Dashboard مستقیم از ClickHouse بخواند، یا Kafka Connect جایگزین شود.
انتقال به اپراتور با SIP MESSAGE از سرویس جدا	ارزیابی Transfer بومی LiveKit (SIP REFER) — یک سرویس کمتر.
Broker سلری: Postgres در یک پروژه، Redis در دیگری	یک استاندارد واحد (Redis).
Backup سفارشی Qdrant	مقایسه با Snapshot بومی Qdrant به MinIO.
سه سیستم Tracing	OpenTelemetry به‌عنوان استاندارد، با Exporter به Backendهای موجود.
Retention دستی (`archive_old_calls`)	ترکیب Partition در Postgres، TTL در ClickHouse و Lifecycle در MinIO.
`websocall` و LiveKit برای وب	مشخص شود `websocall` ابزار تست است یا کانال محصول.

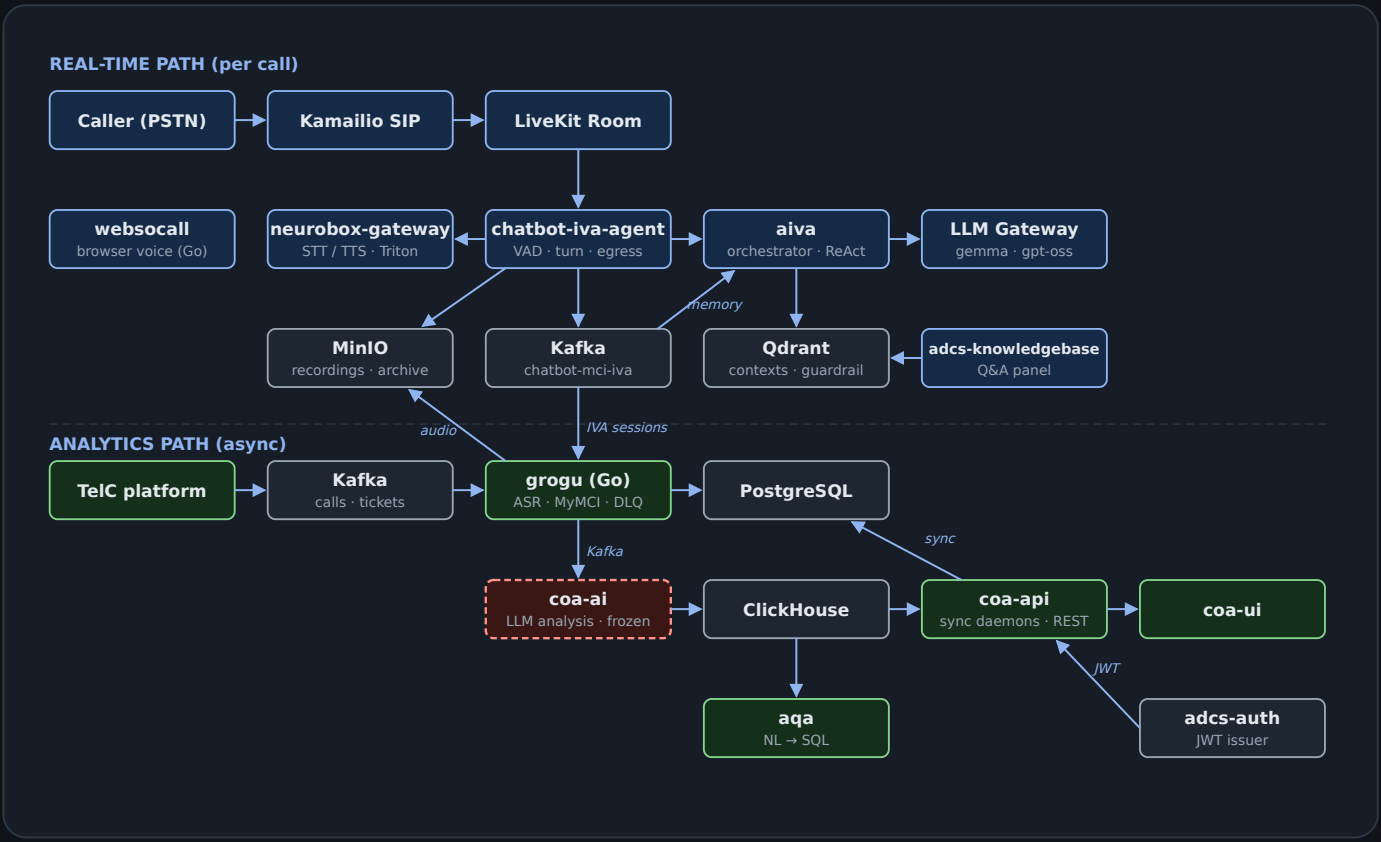
برگه تصمیم CTO: کار جدید، ابزار پیش‌فرض

اگر کار جدید...	پیش‌فرض
منطق Agent، Prompt، RAG یا مدل است	Python (روی `aiva` / `agentlib`)
Gateway، Consumer یا سرویس پرترافیک است	Go
محاسبه روی هر فریم صوت با Latency ثابت است	Rust — فقط پس از Benchmark
داده تراکنشی و رابطه‌ای است	PostgreSQL
گزارش و Aggregation روی حجم بالاست	ClickHouse
جستجوی معنایی یا Classification با مثال است	Qdrant
Cache، Rate limit یا داده موقت است	Redis
فایل صوت، ویدئو یا Backup است	MinIO
رویداد بین سرویس‌ها با نیاز به Replay است	Kafka
Task پس‌زمینه کوتاه است	Celery
تماس صوتی/تصویری Real-time است	LiveKit (Room + Agent + SIP)
تبدیل صوت یا ویدئو است	FFmpeg
اجرای مدل روی GPU است	Triton پشت `neurobox-gateway`
ابزاری است که LLM صدا می‌زند	MCP در `dobby`

جمله آماده برای جلسه: «هر زبان در Phoenix یک کار مشخص دارد: Python برای مغز، Go برای لوله‌کشی، Rust برای مسیر داغ صوت. هر داده هم یک خانه دارد: Postgres برای حقیقت، ClickHouse برای تحلیل، Qdrant برای معنا، Redis برای سرعت،

MinIO برای فایل و Kafka برای رویداد.»

۱۲. Architecture از ابتدا تا انتها



شکل ۱ — دو مسیر اصلی سیستم: Real-time (بالا) و Analytics (پایین). کادر قرمز خط‌چین = `coa-ai` فریزشده.

BOARD IVA + COA boards CODE python/chatbot-iva-agent/src/config.py:315 · python/aiva/aiva/memory/config.py:19 · golang/grogu/env.exmaple:11,14 · golang/grogu/internal/grogu/worker.go:68 · python/coa-api/coa/apps/main/services/sync.py:402

INFERENCE coa-ai -> ClickHouse edge; coa-ai was not read

BLAME config.py:315 → y.fallah · 2026-06-21 · 6d8bb517d feat(chatbot-iva-agent): add whitelist trash code (v0.25.0)

BLAME config.py:19 → Hamed Mahmoudkhani · 2026-06-02 · dfa251cd8 feat(cb-iva-demo): added debounced summary generation using kafka m...

BLAME env.exmaple:11 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME env.exmaple:14 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME worker.go:68 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME sync.py:402 → a.rahimi · 2026-05-23 · 02e7bd3e0 feat(coa-api): change clickhouse columns in syncer v4.29.0

مسیر Real-time — قدم‌به‌قدم

۱. **Kamailio** تماس SIP (Session Initiation Protocol) را می‌پذیرد و به **LiveKit** می‌دهد؛ LiveKit برای تماس یک Room می‌سازد و یک Job به Worker پایتون Dispatch می‌کند.

BOARD IVA board CODE python/chatbot-iva-agent/src/main.py:86,349 (job dispatch)

BLAME main.py:86 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

BLAME main.py:349 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

۲. `chatbot-iva-agent` وارد Room می‌شود: VAD (Voice Activity Detection) و Turn detection گفتار را تکه می‌کنند؛ صدا به **STT (Speech-to-Text)** (از `neurobox-gateway`) می‌رود.

CODE python/chatbot-iva-agent/src/main.py:157 · python/chatbot-iva-agent/src/config.py:364

INFERENCE STT served by neurobox-gateway (python/adcs/README.md 'Neurobox audio')

BLAME main.py:157 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

BLAME config.py:364 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

۳. متن به `aiva` روی `/chat/completions` می‌رود؛ AIVA (Agentic Intelligent Virtual Assistant) با Branchهای موازی و Qdrant پاسخ را Stream می‌کند (بخش ۱۴).

CODE python/aiva/aiva/chat/views/completion.py:35

BLAME completion.py:35 → Hamed Mahmoudkhani · 2026-06-02 · 334867943 feat(cb-iva-demo): added stateless OpenAI compatible APIs

۴. پاسخ جمله‌به‌جمله به **TTS (Text-to-Speech)** و سپس به Room برمی‌گردد.

CODE python/chatbot-iva-agent/src/config.py:164

BLAME config.py:164 → y.fallah · 2026-08-03 · f36e6cc2f feat(chatbot-iva-agent): add openai turn detector (v0.30.0)

۵. هم‌زمان **Egress** صدای Participant و Agent را در **MinIO** ضبط می‌کند.

CODE python/chatbot-iva-agent/src/utils/egress_recorder.py:47 · python/chatbot-iva-agent/src/config.py:158

BLAME egress_recorder.py:47 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

BLAME config.py:158 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

۶. در Shutdown، **Session Summary** روی **Kafka** منتشر می‌شود. دو Consumer دارد: Memory در AIVA (Agentic Intelligent Virtual Assistant) و `grogu` برای Analytics.

CODE python/chatbot-iva-agent/src/entrypoint.py:93 · python/chatbot-iva-agent/src/schema.py:42 · python/aiva/aiva/memory/config.py:19 · golang/grogu/env.exmaple:14 INFERENCE both consume the same stream; topic names are env-configured

BLAME entrypoint.py:93 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

BLAME schema.py:42 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

BLAME config.py:19 → Hamed Mahmoudkhani · 2026-06-02 · dfa251cd8 feat(cb-iva-demo): added debounced summary generation using kafka m...

BLAME env.exmaple:14 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

مسیر Analytics — قدم‌به‌قدم

۱. تماس‌های Contact Center (پلتفرم **TelC**) و Sessionهای IVA (Intelligent Virtual Assistant) از Kafka وارد `grogu` می‌شوند.

CODE golang/grogu/internal/grogu/worker.go:22,72,152

BLAME worker.go:22 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME worker.go:72 → a.rahimi · 2026-08-26 · d60b12d23 fix(grogu): Fix IVA Kafka checkpoint persistence and offset commits...

BLAME worker.go:152 → a.salimi · 2026-07-01 · 91160bf27 feat(grogu): adding ticket pipline v0.22.0

۲. `grogu` صوت را از MinIO می‌گیرد، کانال‌های استریو را Merge و Duration را محاسبه می‌کند، ASR (Automatic Speech Recognition) را با Circuit Breaker صدا می‌زند، اطلاعات حساب را از **MyMCI** می‌گیرد و `calls` ، `call_records` ، `call_ai_tags` را در PostgreSQL Upsert می‌کند. شکست‌ها به Dead-letter می‌روند.

CODE golang/grogu/internal/voice/merge.go · golang/grogu/internal/asr/service.go:58,85 · golang/grogu/internal/mymci/service.go · golang/grogu/internal/call/repository.go:601 · golang/grogu/internal/grogu/worker.go:63

ADDED merge.go → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME service.go:58 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME service.go:85 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

ADDED service.go → a.abolghasemi · 2026-09-23 · 644584f32 feat(grogu): adding mymci account information v0.32.0

BLAME repository.go:601 → a.rahimi · 2026-02-16 · 2e9b40107 feat(grogu):change tables for status changing in db v0.8.0

BLAME worker.go:63 → a.rahimi · 2026-08-25 · f692051db ref(grogu): Fix circuit breaker isolation, dead-letter recovery, an...

۳. نتیجه روی Kafka به `coa-ai` می‌رود که با LLM (Large Language Model) تحلیل (Sentiment، Summary، Category...) می‌کند و در **ClickHouse** می‌نویسد.

CODE golang/grogu/internal/grogu/worker.go:68 (sendResultToKafka) INFERENCE consumer = coa-ai, writes ClickHouse; coa-ai not read (752f89333)

BLAME worker.go:68 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

۴. Daemonهای `coa-api` با Cursor از ClickHouse می‌خوانند و به PostgreSQL همگام می‌کنند؛ REST (Representational State Transfer) API (Application Programming Interface) به `coa-ui` سرویس می‌دهد.

CODE python/coa-api/coa/apps/main/management/commands/clickhouse_sync.py · sync_canonical_mappings.py · python/coa-api/coa/apps/main/services/sync.py:402

ADDED clickhouse_sync.py → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)

ADDED sync_canonical_mappings.py → a.rahimi · 2026-06-09 · 72d37d08f feat(coa-api): Add canonical sync for category and subcategory v4.33.0

BLAME sync.py:402 → a.rahimi · 2026-05-23 · 02e7bd3e0 feat(coa-api): change clickhouse columns in syncer v4.29.0

۵. `aqa` مستقیماً روی ClickHouse Query می‌زند.

CODE python/aqa/aqa/agents/tools/sql_executor.py:16 · python/aqa/aqa/config/database.py:2

BLAME sql_executor.py:16 → p.kazem · 2026-09-15 · 53ba5f20a feat(aqa): add db field to api

BLAME database.py:2 → Hamed Mahmoudkhani · 2026-05-18 · c924b4b6a feat(aqa): base agent

۱۳. agentlib — هسته Framework

`agentlib` (۱۹ هزار خط، ۱۴۰ فایل تست) کتابخانه‌ای است که AIVA (Agentic Intelligent Virtual Assistant)، ADCS (Agentlib Django Customer Support)، AQA (Agentic Query Analyzer) و Voice Agent روی آن ساخته شده‌اند. فهم آن کلید فهم همه سرویس‌های AI (Artificial Intelligence) است.

Event-driven Core

جزء	کار	فایل
`EventBus`	Hub مرکزی Publish/Subscribe. برای هر نوع Event یک `TaskRunner` دارد؛ Listenerها روی Runner ثبت می‌شوند و خطاها روی `err_chan` جمع می‌شوند.	`core/event/bus.py`
`@EventBus.on`	Decorator که نوع Event را از **Type Hint پارامتر** تشخیص می‌دهد. `def handle(self, e: UserInputEvent)` خودکار Handler با Subscribe می‌شود.	`core/event/bus.py`
`Listener`	پوشش یک تابع Handler با Tagها و کانال خطا.	`core/event/listener.py`
`DLQ`	Dead Letter Queue: Event شکست‌خورده تا `max_tries=3` دوباره Publish می‌شود، بعد در صف می‌ماند.	`core/event/dlq.py`
`Chan` ، `TaskSet`	کانال Async و مجموعه Taskها برای Stream و Cancellation.	`utils/aio`
`EventHandlerMixin`	Base برای کلاس‌هایی که Handler دارند؛ `_attach_bus()` همه متدهای `@on` را ثبت می‌کند.	`core/mixin`

CODE python/agentlib/agentlib/core/event/bus.py:25,53,78 · dlq.py:12,18 · listener.py:11 (paths under python/agentlib/agentlib/)

BLAME bus.py:25 → m.ghazanfari · 2026-01-06 · ddd5f4833 feat(agentlib): supporting multiple events

BLAME bus.py:53 → m.ghazanfari · 2026-01-31 · 4954f925d feat(agentlib): tag

BLAME bus.py:78 → m.ghazanfari · 2026-01-31 · 4954f925d feat(agentlib): tag

BLAME dlq.py:12 → h.mahmoudkhani · 2026-02-15 · e4b15a904 refactor(agentlib): event listeners to event handlers

BLAME dlq.py:18 → m.ghazanfari · 2026-01-28 · 97d87ee0c feat(agentlib): DLQ

BLAME listener.py:11 → m.ghazanfari · 2026-01-06 · ddd5f4833 feat(agentlib): supporting multiple events

```
class MyHandler(EventHandlerMixin):
    def __init__(self, bus: EventBus):
        self._attach_bus(bus)          # registers every @EventBus.on method

    @EventBus.on
    async def on_input(self, event: UserInputEvent) -> None:   # type hint =
subscription
        ...
```

INFERENCE illustrative example of the pattern in core/event/bus.py:78 and core/event/dlq.py:27

```
self._bus.publish_nowait(MyResultEvent(...))
```

Agents، LLM (Large Language Model) و Retrieval

جزء	کار
`ReActAgent`	حلقه Reasoning ↔ Tool Execution با Stream داخلی.
`GuardrailAgent`	ارزیابی Policy پیام با LLM (Large Language Model) و Parser خروجی.
`MemoryAgent` ، `ArchivistAgent`	خلاصه‌سازی مکالمه برای Long-term Memory و فشرده‌سازی Context.
`BaseChatOrchestrator` و مشتقات	الگوهای — `WithMemory` ، `WithRetrievedGuardrail` ، `ChatOrchestratorWithGuardrail` آماده Orchestration که AIVA (Agentic Intelligent Virtual Assistant) از آن‌ها ارث می‌برد.
`OpenAILLM` ، `OpenAILLMStream`	کلاینت LLM (Large Language Model) سازگار با OpenAI با Streaming.
`function_tool` ، `ToolContext`	تبدیل تابع Python به Tool قابل فراخوانی LLM (Large Language Model).
`ChatContext`	تاریخچه پیام‌ها (System، User، Assistant، Tool).
Retrievers	`ContextRetriever` ، `InstructionRetriever` ، `ToolRetriever` ، `GuardrailRetriever` ، `QdrantRetriever`.
Monitoring	Logger ساخت‌یافته `L()` ، Trace، Metrics (`EventBusTrace`).

CODE python/agentlib/agentlib/agents/{react,guardrail,memory,archivist}/agent.py · inference/orchestrations/chat/base.py:49, with_guardrail.py:30,122, with_memory.py:23 · llm/__init__.py · retrieve/__init__.py

ADDED agent.py → m.ghazanfari · 2026-01-07 · 0c2938355 feat(agent): ReAct agent

ADDED agent.py → m.ghazanfari · 2026-01-21 · 172d5c84a feat(agentlib): guardrail

ADDED agent.py → m.ghazanfari · 2026-02-07 · f448cbc43 feat(agentlib): memory (v0.20.0)

ADDED agent.py → m.ghazanfari · 2026-07-14 · 8011edb80 feat(agentlib): session memory

Plugins

- `agentlib-plugin-fast` — Middleware یک EventBus برای هر Request در FastAPI.

- `agentlib-plugin-mcp` — `MCPClient`، `MCPPool` و `discover_tools` : Toolهای هر MCP (Model Context Protocol) به‌عنوان Server در agentlib.
- `agentlib-plugin-hakim` — Embedding از مدل‌های داخلی Hakim.

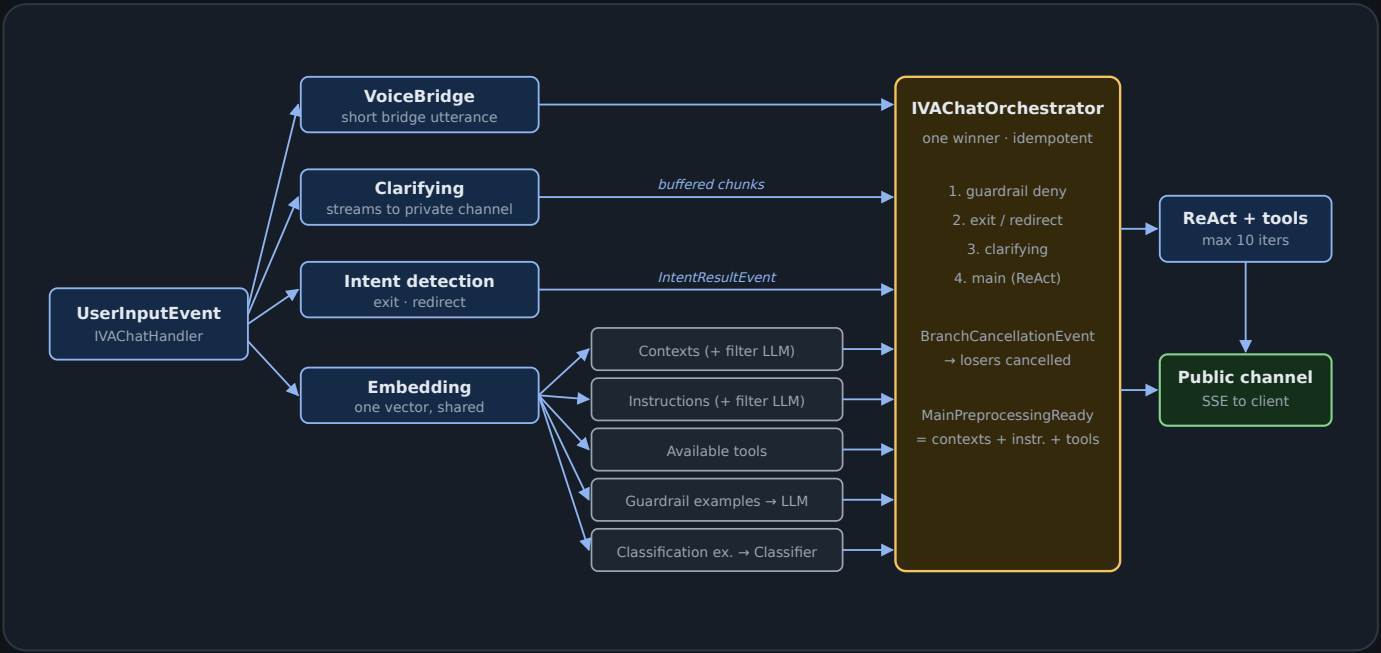
CODE python/agentlib/plugins/agentlib-plugin-{fast,mcp,hakim}/ DOC python/agentlib/plugins/agentlib-plugin-mcp/README.md

LAST python/agentlib/plugins/agentlib-plugin-fast/ → a.salimi · 2026-08-01 · b0f0e9e60 feat(agentlib): shutdown in taskset and eventbus, some log levels, ...

LAST python/agentlib/plugins/agentlib-plugin-mcp/ → m.ghazanfari · 2026-09-20 · 52d9f2fd6 fix(mcp): discovery fixed

LAST python/agentlib/plugins/agentlib-plugin-hakim/ → Hamed Mahmoudkhani · 2026-08-02 · 2211fa708 refactor(agentlib): remove prefixes from mcp tools to avoid llm mis...

۱۴. AIVA (Agentic Intelligent Virtual Assistant) در عمق: Event-driven Orchestration



شکل ۲ — Branchهای موازی AIVA (Agentic Intelligent Virtual Assistant) و داوری `IVAChatOrchestrator`.

DOC `python/aiva/docs/RequestFlow.md:107 (flowchart), :68 (precedence)` CODE `python/aiva/aiva/chat/handlers/orchestrator.py:71`

BLAME `orchestrator.py:71 → a.salimi · 2026-09-05 · 8a06d3d2f feat(aiva): add guadraril retrieval`

نقشه فایل‌ها

فایل	مسئولیت
`aiva/chat/views/session.py` — `SessionView`	ساخت/بازیابی `ChatbotUser` با شماره، ساخت `Session`، برگرداندن Greeting.
`aiva/chat/views/completion.py` — `CompletionsView`	Validate درخواست OpenAI-style (فقط `stream=true`)، ساخت Handler، شروع User Turn یا Stream، Signal Turn کانال Public به SSE (Server-Sent Events).
`aiva/chat/handlers/chat.py` — `IVAChatHandler`	ساخت Agentها، Retrieverها، `ChatContext` مشترک، اتصال به Bus و Orchestrator؛ Truncate تاریخچه به `CHAT_HISTORY_LIMIT`.
`aiva/chat/handlers/orchestrator.py` — `IVAChatOrchestrator`	Readiness Join، انتخاب Winner، ترتیب خروجی، لغو بازنده‌ها، `SyncChatContextEvent`.
`aiva/chat/handlers/branch.py`	`AnswerRouteSelectedEvent`، `ResponseBranch`، `AnswerRoute`، `BranchCancellationEvent`، `CancellableBranchHandler`.
`clarifying.py`، `classification.py`، `intent_detection/`، `voice_bridge.py`، `retrieval.py`	Handler هر Branch.

فایل	مسئولیت
`aiva/agents/`	`AIVAReActAgent`، `IntentDetectionAgent`، `GreetingAgent`.
`aiva/prompts/`	Prompt هر نقش: `react`، `guardrail`، `intent_detection`، `classification`، `clarifying_chitchat`، `voice_bridge`، `greeting`، `retrieval`، `memory`.
`aiva/retrieve/qdrant.py`	`LLMFilteredAdcsQdrantRetriever` — Retrieval + فیلتر LLM (Large Language Model) اختیاری.
`aiva/tools/registry.py`	`TOOL_REGISTRY`؛ فعال‌سازی با `AGENT_ENABLED_TOOLS`.
`aiva/memory/management/commands/aiva_consume_kafka.py`	`ConsumerWorker`: مصرف Session تمام‌شده، Whitelist، `MemoryAgent.summarize`، `save_user_memory`؛ حداکثر ۱۰ Session هم‌زمان؛ Metrics روی پورت ۸۰۰۱.

CODE paths relative to python/aiva/ · orchestrator.py:65,71 · branch.py:13-35 · aiva_consume_kafka.py:52 · memory/config.py:23,25

Eventها به ترتیب زمانی

```
UserInputEvent
 ├─ VoiceBridge ──────────────────────► VoiceBridgeResultEvent
 ├─ Clarifying (private stream) ──────► ClarifyingStreamFinishedEvent
 ├─ Intent ───────────────────────────► IntentResultEvent
 └─ Embedding ─► EmbeddingResultEvent ─► AIVARetrieveEvent
                  ├─ contexts / instructions / tools ─► MainPreprocessingReadyEvent
                  ├─ guardrail examples ─► guardrail allow|deny
                  └─ ClassificationExamplesRetrievedEvent ─► main|clarifying
Orchestrator ─► AnswerRouteSelectedEvent ─► BranchCancellationEvent (losers)
```

DOC python/aiva/docs/RequestFlow.md:16-180 CODE python/aiva/aiva/chat/handlers/*.py (event classes)

```
Winner ─► SyncChatContextEvent (persist) ─► LastChunkEvent
```

قواعد Failure

- Classifier خطا یا JSON (JavaScript Object Notation) نامعتبر → `main`.
- Clarifying Stream شکست → Route به `main` با Preprocessing آماده.
- VoiceBridge شکست → بدون Bridge؛ هیچ Branch منتظر نمی‌ماند.
- Guardrail Deny هرگز با Branchهای Speculative لغو یا معطل نمی‌شود.
- Disconnect Client → `IVAChatHandler.aclose()` همه Taskهای Branch را Cancel و Await می‌کند.
- فقط Winner در DB (Database) ذخیره می‌شود؛ Chunkهای بازنده هرگز به Client نمی‌رسند.

DOC python/aiva/docs/RequestFlow.md:160-178

چرا این طراحی خوب است: Latency برابر کندترین Branch *لازم* است نه مجموع همه؛ Guardrail هیچ‌وقت منتظر نمی‌ماند؛ و Persistence فقط یک پاسخ منسجم دارد. هزینه‌اش: ۵ تا ۷ LLM (Large Language Model) Call در هر Turn.

`INFERENCE` engineering assessment

۱۵. IVA (Intelligent Virtual Assistant) Voice Agent در عمق

Lifecycle یک تماس در Code

۱. `prewarm(proc)` — در شروع هر Worker Process: بارگذاری مدل Silero VAD (Voice Activity Detection) و Avatar Image (برای کاهش Latency اولین تماس).

CODE python/chatbot-iva-agent/src/main.py:345,89
BLAME main.py:345 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...
BLAME main.py:89 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

۲. `JobRequestHandler` — پذیرش Job از LiveKit.

CODE python/chatbot-iva-agent/src/main.py:86
BLAME main.py:86 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

۳. `agent_session_factory` — ساخت `IVAAgentSession` با VAD (Voice Activity Detection)، STT (Speech-to-Text)، LLM (Large Language Model)، TTS (Text-to-Speech) و ثبت Event Handlerها: `user_state_changed` (Away/Speaking)، `agent_state_changed` (Thinking Sound)، `metrics_collected` (Usage)، `conversation_item_added` (تاریخچه)، `function_tools_executed` (Exit/Redirect)، `error`.

CODE python/chatbot-iva-agent/src/main.py:157,197,223,238,252,273,277
BLAME main.py:157 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)
BLAME main.py:197 → y.fallah · 2026-06-03 · 1fb743b0b feat(chatbot-iva-agent): add support for phone number feed in creat...
BLAME main.py:223 → y.fallah · 2026-06-03 · 1fb743b0b feat(chatbot-iva-agent): add support for phone number feed in creat...

۴. `dispatch(ctx)` — انتخاب Agent بر اساس `AGENT_LLM_STRUCTURE`: `ChatbotlibAgent` یا `AgentlibAgent`.

CODE python/chatbot-iva-agent/src/main.py:349 · python/chatbot-iva-agent/src/config.py:283
BLAME main.py:349 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...
BLAME config.py:283 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0)

۵. `Entrypoint.run()` به ترتیب: Avatar (اگر فعال) → `session.start(room)` → مکث RTP (Real-time Transport Protocol) → `session.say(greeting)` → Background Player → ثبت `_log_summary` برای Egress → Shutdown.

CODE python/chatbot-iva-agent/src/entrypoint.py:66-94
BLAME entrypoint.py:66 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

۶. در Shutdown: `_log_summary` یک `SessionSummarySchema` کامل روی Kafka منتشر می‌کند (`KafkaProducer` Idempotent، Singleton، `max_in_flight=1` برای حفظ ترتیب).

CODE python/chatbot-iva-agent/src/entrypoint.py:93 · python/chatbot-iva-agent/src/utils/kafka.py:21 · python/chatbot-iva-agent/src/config.py:305

`BLAME` entrypoint.py:93 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

`BLAME` kafka.py:21 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...

`BLAME` config.py:305 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

ماژول‌ها

فایل	مسئولیت
`src/main.py`	Worker، Session Factory، Event Handlerها
`src/entrypoint.py`	Lifecycle تماس (کلاس `Entrypoint`)
`src/agents/chatbotlib.py`، `agentlib.py`	دو Agent؛ `AgentlibAgent` وضعیت Tool Call را پیگیری می‌کند.
`src/utils/stt.py`، `tts.py`	کلاینت‌های Retryدار
`src/utils/mci_turn_detector.py`، `openai_turn_detector.py`	دو Turn Detector
`src/utils/egress_recorder.py`	شروع/ردیابی ضبط Room، Participant، Agent
`src/utils/kafka.py`	Producer
`src/utils/sms.py`، `avatar.py`، `player.py`، `sentence_tokenizer.py`	SMS (Short Message Service)، Avatar، صداهای پس‌زمینه، تقسیم جمله
`src/utils/apm.py`، `prometheus_metrics.py`	Observability
`src/config.py`	تمام پارامترها (Dynaconf، بدون Prefix)

`CODE` paths relative to python/chatbot-iva-agent/

این سرویس **صفر فایل تست** دارد و Session Summary حاوی شماره تلفن و متن کامل است. هر تغییر در `schema.py` Contract با `grogu` و Memory Consumer را می‌شکند.

`CODE` find python/chatbot-iva-agent -name 'test_*.py' -> 0 · golang/grogu/internal/grogu/worker.go:72 · python/aiva/aiva/memory/management/commands/aiva_consume_kafka.py:300-

`BLAME` worker.go:72 → a.rahimi · 2026-08-26 · d60b12d23 fix(grogu): Fix IVA Kafka checkpoint persistence and offset commits...

`BLAME` aiva_consume_kafka.py:300 → a.salimi · 2026-07-22 · efa2f99ac feat(aiva): Update memory to use aggregated messages

۱۶. Pipeline CCA (Call Center Analyze) در عمق

grogu (Go) — Ingestion

Handler	کار
`MessageHandler` (TelC)	Parse پیام، ثبت Latency کافکا، دانلود صوت از MinIO، Merge کانال‌ها (`voice/merge.go`)، Duration، ASR (Automatic Speech Recognition) (`asr.GetVoiceTranscriptions`) با Circuit Breaker، Upsert `call_records` و `call_ai_tags`، ارسال نتیجه به Kafka؛ خطا → `markCallDeadLetter`.
`IvaMessageHandler`	Upsert `call_ai_tags` از Payload IVA (Intelligent Virtual Assistant)؛ Dead-letter با Call ID (Identifier) همان Payload.
`TicketMessageHandler`	ثبت Ticket و ربط آن به Call.
Enrichment	`mymci` — اطلاعات حساب مشترک (با Flag قابل خاموش‌کردن «Account Overview»).
Resilience	Circuit Breaker جدا برای Kafka، DB (Database) و ASR (Automatic Speech Recognition)؛ لغو از سمت Caller شمرده نمی‌شود؛ Prometheus.

CODE golang/grogu/internal/grogu/worker.go:22,63,68,72,144,149,152 · internal/asr/service.go:58,85 · internal/voice/merge.go · internal/mymci/service.go · internal/config/config.go GIT 'feat(grogu): adding flag for enable and disable account overview v0.33.0'

BLAME worker.go:22 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME worker.go:63 → a.rahimi · 2026-08-25 · f692051db ref(grogu): Fix circuit breaker isolation, dead-letter recovery, an...

BLAME worker.go:68 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

نکته معماری: `grogu` و `coa-api` **روی همان جداول می‌نویسند** (`calls`، `call_records`، `call_ai_tags`). Migration این جداول در `coa-api` است؛ Migrationهای `grogu` فقط جدول Dead-letter خودش (`call_asr_dl`) را می‌سازند. هر تغییر Schema در Django باید در Structهای Go هم اعمال شود.

CODE golang/grogu/internal/call/model.go:24-25,40-41,123-124 · python/coa-api/coa/apps/main/models.py:160,196,422 (db_table) · golang/grogu/migrations/ (call_asr_dl only)
INFERENCE same database instance — inferred from identical table names

BLAME model.go:24 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME model.go:40 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

BLAME model.go:123 → a.rahimi · 2026-02-22 · 3079540e5 feat(grogu):change tables for status changing in db v0.9.0

BLAME models.py:160 → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)

BLAME models.py:196 → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)

BLAME models.py:422 → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)

LAST golang/grogu/migrations/ → a.rahimi · 2026-08-25 · f692051db ref(grogu): Fix circuit breaker isolation, dead-letter recovery, an...

coa-ai — تحلیل (فریز)

مصرف نتیجه ASR (Automatic Speech Recognition)، اجرای Promptهای تحلیل (Sentiment، Summary، Category/Subcategory، Bad Words، Greeting...) و نوشتن در ClickHouse. آخرین Schema Commit: 2025-07-13. تحلیلی که coa-api انتظار دارد در coa/config.py است (sentiment_analysis، summary، bad_words، start_greeting، end_greeting).

GIT 752f89333 2025-07-13 CODE python/coa-api/coa/config.py:456-488 INFERENCE coa-ai's behaviour — the repo was not read

BLAME config.py:456 → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)

coa-api — Django Apps

App	مسئولیت
main	Voiceها، Call، Tag (AI (Artificial Intelligence) و Human)، Thematic Analysis، Dashboard، همه Daemonهای Sync و Archive
trends	Subject، Trend، TrendAnnotation، trend_sync، Webhook
iva	IVA (Intelligent Virtual Assistant) Dashboard و Announcement Voice
tickets	Ticket و آمار
experts	کارشناس، Comment، Export
files	Stream صوت
management، users	Facade کاربر روی adcs-auth؛ Permissionها؛ Viewهای Login قدیمی (Dead Code)

CODE python/coa-api/coa/api.py · python/coa-api/coa/apps/*/ · python/coa-api/coa/apps/users/urls.py:1-8 (dead code note)

ADDED api.py → a.behnamnia · 2024-11-11 · 1d7998164 COA v0.2 AI module (v0.2.2)

BLAME urls.py:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0

Sync Daemons — الگوی مشترک

```
while True:
    result = service.sync()          # read ClickHouse rows after cursor
    #   -> upsert into Postgres in batches
    #   -> persist cursor (CanonicalSyncLog / FailedCallsSyncState / ch_created_at)
    sleep(service.get_wait_seconds(result))
```

INFERENCE pseudocode of python/coa-api/coa/apps/main/management/commands/sync_canonical_mappings.py daemon loop CODE python/coa-api/coa/apps/main/services/canonical_sync.py:70,112 · python/coa-api/coa/apps/main/models.py:430,457

BLAME canonical_sync.py:70 → a.rahimi · 2026-06-09 · 72d37d08f feat(coa-api): Add canonical sync for category and subcategory v4.33.0

BLAME canonical_sync.py:112 → a.rahimi · 2026-06-09 · 72d37d08f feat(coa-api): Add canonical sync for category and subcategory v4.33.0

BLAME models.py:430 → a.rahimi · 2026-06-09 · 72d37d08f feat(coa-api): Add canonical sync for category and subcategory v4.33.0

BLAME models.py:457 → a.abolghasemi · 2026-07-29 · 76e54adb1 feat(coa-api): sync failed calls from cca and can expert view comme...

```
# short if has_more_data, else long
```

Cursorها ماندگارند، پس Restart از همان نقطه ادامه می‌دهد. `sync_canonical_mappings` فقط در پنجره ساعتی تهران اجرا می‌شود و `cursor_offset` ردیف‌های هم‌زمان را مدیریت می‌کند.

۱۷. AQA (Agentic Query Analyzer) در عمق

```
QueryReceived → classifier → IntentClassified
   ├─ chat intent → chat agent → answer
   └─ data intent → planner → PlanningDone (subtasks)
         for each SubTask:
            sql_generator → sql_executor(ClickHouse|Postgres, row cap)
                          → data_reducer → sql_result_analyzer (+ chart data)
         → ResultAchieved → TurnCompleted (recorded)
   any stage: ask_user → pause() → resume on next message
```

CODE python/aqa/aqa/orchestrator/orchestrator.py:70,211,296,304,321,345 · python/aqa/aqa/agents/{classifier,planner,sql_generator,data_reducer,sql_result_analyzer,error_reporter,chat}/ · python/aqa/aqa/agents/tools/clarify.py:7
BLAME orchestrator.py:70 → h.mahmoudkhani · 2026-06-14 · e93340095 feat(AQA): multiple agents, agents orchestrator, refactor: project ...
BLAME orchestrator.py:211 → Hamed Mahmoudkhani · 2026-09-13 · 03c104999 feat(aqa): human clarification in rest API
BLAME orchestrator.py:296 → h.mahmoudkhani · 2026-06-14 · e93340095 feat(AQA): multiple agents, agents orchestrator, refactor: project ...
LAST python/aqa/aqa/agents/classifier/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
LAST python/aqa/aqa/agents/planner/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
LAST python/aqa/aqa/agents/sql_generator/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
LAST python/aqa/aqa/agents/data_reducer/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
LAST python/aqa/aqa/agents/sql_result_analyzer/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
LAST python/aqa/aqa/agents/error_reporter/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
LAST python/aqa/aqa/agents/chat/ → m.ghazanfari · 2026-09-26 · 855d1cfe1 feat(aqa): sync to async
BLAME clarify.py:7 → Hamed Mahmoudkhani · 2026-09-13 · d69677472 refactor(aqa): decouple pause/resume from views into converter/service

errors: error_reporter explains in user language

- `Orchestrator` (`aqa/orchestrator/orchestrator.py`) روی همان EventBus agentlib است و مراحل را با `_emit_stage` به Stream UI (User Interface) می‌کند.

CODE python/aqa/aqa/orchestrator/orchestrator.py:70,501
BLAME orchestrator.py:70 → h.mahmoudkhani · 2026-06-14 · e93340095 feat(AQA): multiple agents, agents orchestrator, refactor: project ...
BLAME orchestrator.py:501 → h.mahmoudkhani · 2026-08-23 · ca7355f46 feat(AQA): chat APIs

- امنیت SQL (Structured Query Language): هیچ Validation سمت Client نیست؛ **کاربر باید Read-only DB (Database) باشد**. سقف ردیف با `MAX_ROWS_THRESHOLD` .

CODE python/aqa/aqa/agents/tools/sql_executor.py:27,34

BLAME sql_executor.py:27 → h.mahmoudkhani · 2026-09-09 · badce7736 fix(aqa): remove sql safety check

BLAME sql_executor.py:34 → Hamed Mahmoudkhani · 2026-05-18 · c924b4b6a feat(aqa): base agent

- State در PostgreSQL خود AQA (Agentic Query Analyzer) با (SQLAlchemy + Alembic)؛ Session، Step، Feedback، User.

CODE python/aqa/aqa/config/postgres.py:4 · python/aqa/aqa/api/models/ · python/aqa/alembic.ini

BLAME postgres.py:4 → m.ghazanfari · 2026-08-02 · 43e5179bc feat(aqa): data layer

LAST python/aqa/aqa/api/models/ → p.kazem · 2026-08-25 · 5402a93f2 feat(aqa): feedback

ADDED alembic.ini → m.ghazanfari · 2026-02-08 · 40a668745 feat(nav): init (v0.1.0)

- دو UI (User Interface): FastAPI + SSE (Server-Sent Events) و Chainlit با Guest Auth) `CHAINLIT_AUTH_SECRET`).

DOC python/aqa/README.md

۱۸. Authorization و Authentication

Token Flow

```
client ──POST /api/v1/token/login──▶ adcs-auth ──▶ access (15 min) + refresh (7 days)
client ──Bearer access──▶ coa-api / adcs dashboard / kb / aqa
         each service verifies the HS256 signature locally with the shared TOKEN_SIGNING_KEY
         (no network call to adcs-auth) and reads permissions from the token claims
client ──POST /api/v1/token/refresh──▶ new pair; old refresh blacklisted (ROTATE + BLACKLIST)
```

CODE python/adcs-auth/adcs_auth/settings.py:155-161 · python/adcs-auth/adcs_auth/config.py:15-19 · python/adcs/adcs/auth/utils/blacklist.py · python/coa-api/coa/apps/users/urls.py:1-4 ('verified statelessly here via adcs.auth')

BLAME settings.py:155 → p.kazem · 2026-05-23 · 8ba5e8f05 feat(adcs-auth): create project structure

BLAME config.py:15 → Hamed Mahmoudkhani · 2026-06-01 · 869e4a731 fix(adcs-auth): missing migrations, common settings, installed apps

ADDED blacklist.py → pooria · 2026-07-12 · 91556e3f2 feat(adcs): check cache for access token blacklisted

BLAME urls.py:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0

logout ──▶ token blacklisted in Redis (shared blacklist checked by services)

مفهوم	پیاده‌سازی
JWT (JSON Web Token)	SimpleJWT، الگوریتم `HS256`، کلید مشترک `TOKEN_SIGNING_KEY`، Access ۱۵ دقیقه، Refresh ۷ روز با Rotation و Blacklist. CODE python/adcs-auth/adcs_auth/settings.py:155-161 · config.py:15-19 BLAME settings.py:155 → p.kazem · 2026-05-23 · 8ba5e8f05 feat(adcs-auth): create project structure BLAME config.py:15 → Hamed Mahmoudkhani · 2026-06-01 · 869e4a731 fix(adcs-auth): missing migrations, common settings, installed apps
Roles	جدول `user_roles` با `assigned_by` و `expires_at` — نقش موقت (مثلاً دسترسی ۲۴ ساعته به هویت) از قبل پشتیبانی می‌شود. CODE python/adcs-auth/adcs_auth/role/models.py:51 BLAME models.py:51 → p.kazem · 2026-05-23 · 8ba5e8f05 feat(adcs-auth): create project structure

مفهوم	پیاده‌سازی
Permission Check	`AuthViewMixin` + Decorator `@with_permissions({...})` ؛ کلاس‌های `HasPermission`، `AnyOfPermission`، `PermissionGroup`، `CheckPasswordExpired`. **CODE** python/coa-api/coa/apps/files/views.py:1-2 · python/coa-api/coa/apps/users/permissions.py:81,143,164 **BLAME** views.py:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0 **BLAME** permissions.py:81 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0 **BLAME** permissions.py:143 → h.samsami · 2024-12-29 · c69547074 feat(rbac): add user management APIs and RBAC (v1.2.0) **BLAME** permissions.py:164 → h.samsami · 2025-05-10 · 91bf74765 add expire password functionality for users (v1.14.0)
Row-level Scope	بدون `view_all_calls`، جست‌وجوی تماس به `agent_id = user_code` محدود می‌شود (`services/call.py:849`). **CODE** python/coa-api/coa/apps/main/services/call.py:849-856 **BLAME** call.py:849 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0
Machine Auth	`APIKeyAuthentication` با Header `x-api-key` برای Ingest. **CODE** python/coa-api/coa/apps/main/views/call_recordings.py:19 **BLAME** call_recordings.py:19 → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)
Chat APIs (Application Programming Interfaces)	`permission_classes = ()` در همه Viewهای چت `aiva` و `adcs` ؛ طبق README، امنیت به Gateway سپرده شده. **CODE** python/aiva/aiva/chat/views/session.py:30 **DOC** python/adcs/README.md:27 **BLAME** session.py:30 → Hamed Mahmoudkhani · 2026-06-02 · 334867943 feat(cb-iva-demo): added stateless OpenAI compatible APIs

نکته برای PRV-01/02: چون Role دارای `expires_at` است، «Resolve هویت فقط توسط مدیر سامانه و فقط موقت» با یک Permission جدید (مثلاً `view_identity`) + Masking در Serializer + Audit Log قابل پیاده‌سازی است؛ زیرساخت نقش از قبل آماده است.

CODE python/adcs-auth/adcs_auth/role/models.py:51 **INFERENCE** design proposal
BLAME models.py:51 → p.kazem · 2026-05-23 · 8ba5e8f05 feat(adcs-auth): create project structure

۱۹. Data Model و Messaging

PostgreSQL (coa-api / grogu)

گروه	جداول
Call Core	`calls` (status: queued، in_progress، succeed، failed، invalid)، `call_records` (TelC)، `call_ai_tags` (1:1 با call)
Human Review	`call_human_tags`، `call_human_tag_descriptions`، `call_human_log`، `call_expert_comments`
Voice Upload	`voices`، `voice_ai_tags`، `voice_human_tags`، `voice_human_tag_descriptions`
Sync State	`canonical_sync_logs`، `failed_calls_sync_state`، ستون‌های `ch_created_at` / `ch_updated_at`
Trends	`subjects`، `trends`، `trend_annotations`
Other	`experts`، `tickets`، `announcement_voices`، Users/Roles

CODE python/coa-api/coa/apps/*/models.py (db_table) · python/coa-api/coa/apps/main/models.py:166-172 (Call.Status) · golang/grogu/internal/call/model.go

BLAME models.py:166 → a.abolghasemi · 2026-01-06 · b08274d3e feat(coa-api): rewrite coa-api for using grogu as worker(v2.0.0)

ADDED model.go → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

سایر Storeها

Store	محتوا
ClickHouse	نتایج تحلیل `coa-ai`؛ منبع Sync و منبع Query در AQA (Agentic Query Analyzer) **CODE** python/coa-api/coa/apps/main/services/sync.py:402 · python/aqa/aqa/config/database.py:2 **INFERENCE** written by coa-ai **BLAME** sync.py:402 → a.rahimi · 2026-05-23 · 02e7bd3e0 feat(coa-api): change clickhouse columns in syncer v4.29.0 **BLAME** database.py:2 → Hamed Mahmoudkhani · 2026-05-18 · c924b4b6a feat(aqa): base agent
Qdrant	`aiva-contexts`، `aiva-instructions`، `guardrail`، `aiva-classification`، `aiva-jokes` + Tools؛ Dimension از Embedding تعیین می‌شود **CODE** python/aiva/aiva/chat/config.py:70-79,178 **BLAME** config.py:70 → a.salimi · 2026-09-05 · 8a06d3d2f feat(aiva): add guadraril retrieval **BLAME** config.py:178 → a.salimi · 2026-09-28 · 638179e33 feat(aiva): add voice bridge, classification and clarification

Store	محتوا
MinIO	`chatbot-mci-iva` (ضبط IVA (Intelligent Virtual Assistant))، صوت تماس‌های Dump، TelCهای Archive، صدای Announcement **CODE** python/chatbot-iva-agent/src/config.py:158 · python/coa-api/coa/apps/main/management/commands/archive_old_calls.py:396 · python/coa-api/coa/apps/iva/services.py:285 · golang/grogu/internal/minio/ **BLAME** config.py:158 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0) **BLAME** archive_old_calls.py:396 → a.abolghasemi · 2026-04-28 · cd37cda37 feat(coa-api): adding command to archive old calls v4.11.0 **BLAME** services.py:285 → a.rahimi · 2026-09-26 · 0a54d0d24 feat(coa-api): add voice pages v5.14.0 **LAST** golang/grogu/internal/minio/ → a.rahimi · 2026-08-25 · f692051db ref(grogu): Fix circuit breaker isolation, dead-letter recovery, an...
Redis	Token Blacklist مشترک و Locale Catalog **CODE** python/adcs/adcs/auth/utils/blacklist.py · python/adcs/adcs/locale/utils/catalog.py **ADDED** blacklist.py → pooria · 2026-07-12 · 91556e3f2 feat(adcs): check cache for access token blacklisted **ADDED** catalog.py → m.ghazanfari · 2026-07-22 · ea0f9e978 feat(adcs): locale module
AIVA (Agentic Intelligent DB Virtual Assistant) (Database)	`ChatbotUser` (phone، **memory**)، `Session`، پیام‌ها **CODE** python/adcs/adcs/chat/models/user.py:25 · python/adcs/adcs/chat/models/session.py:9 **BLAME** user.py:25 → Hamed Mahmoudkhani · 2026-07-05 · 1efc94366 fix: issues with memory **BLAME** session.py:9 → h.mahmoudkhani · 2026-04-07 · f8efe8df1 feat(chat): chat APIs
AQA (Agentic Query DB (Database) Analyzer)	Session، Step، Feedback، User (Alembic) **CODE** python/aqa/aqa/api/models/ · python/aqa/aqa/alembic/ **LAST** python/aqa/aqa/api/models/ → p.kazem · 2026-08-25 · 5402a93f2 feat(aqa): feedback **LAST** python/aqa/aqa/alembic/ → p.kazem · 2026-08-25 · 5402a93f2 feat(aqa): feedback

Kafka Topics

Topic	Producer	Consumer
`chatbot-mci-iva`	`chatbot-iva-agent`	`grogu` (`KAFKA_IVA_TOPIC`)
`chatbot-mci-iva-intent` (پیش‌فرض)	Aggregator (در Code فعال یافت نشد)	AIVA (Agentic Intelligent Virtual Assistant) Memory (Group `chatbot-mci-iva-consumer-demo`)
`KAFKA_ASR_CALL_TOPIC`، `KAFKA_CALL_TOPIC`	پلتفرم TelC / grogu	`coa-ai` / `grogu`
Ticket Topic	سامانه Ticket	`grogu`

CODE python/chatbot-iva-agent/src/config.py:315 · python/aiva/aiva/memory/config.py:18-19 · golang/grogu/env.exmaple:11-14

BLAME config.py:315 → y.fallah · 2026-06-21 · 6d8bb517d feat(chatbot-iva-agent): add whitelist trash code (v0.25.0)

BLAME config.py:18 → Hamed Mahmoudkhani · 2026-06-02 · dfa251cd8 feat(cb-iva-demo): added debounced summary generation using kafka m...

BLAME env.exmaple:11 → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

نام دقیق Topicهای TelC و Ticket از Env در Deployment تعیین می‌شود و در Code پیش‌فرض ندارد. نام Consumer Group حافظه هنوز پسوند `demo` دارد.

۲۰. Configuration Reference

همه سرویس‌ها Config را از Env می‌خوانند (Pydantic Settings یا Dynaconf) — تقریباً هر رفتار بدون Deploy Code قابل تغییر است.

سرویس	Env کلیدی	اثر
chatbot-iva-agent	`AGENT_GREETING_MESSAGE` **CODE** python/chatbot-iva-agent/src/config.py:391 **BLAME** config.py:391 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)	متن معرفی و Privacy Notice (CON-01/02)
	`AGENT_VAD_*` ، `AGENT_*_ENDPOINTING_*` **CODE** python/chatbot-iva-agent/src/config.py:60-78 **BLAME** config.py:60 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)	حساسیت شنیدن و پایان نوبت
	`AGENT_TURN_MODEL` **CODE** python/chatbot-iva-agent/src/config.py:82 **BLAME** config.py:82 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...	`openai` / `mci` / `deactive`
	`AGENT_USER_AWAY_MODE` ، `AGENT_NON_SPEECH_MODE` **CODE** python/chatbot-iva-agent/src/config.py:401,430 **BLAME** config.py:401 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0) **BLAME** config.py:430 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0)	مدیریت سکوت و صدای نامفهوم
	`AGENT_SIP_REDIRECT_*` **CODE** python/chatbot-iva-agent/src/config.py:240-253 **BLAME** config.py:240 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...	مقصد انتقال به کارشناس

سرویس	Env کلیدی	اثر
	`AGENT_DISABLE_EGRESS_*` CODE python/chatbot-iva-agent/src/config.py:124-138 BLAME config.py:124 → y.fallah · 2025-11-18 · 27e79ea7e feat(chatbot-iva): add avatar integration, also sending session dat...	ضبط Room/Participant/Agent
	`AGENT_LLM_STRUCTURE`، `AGENT_STT_*`، `AGENT_TTS_*` CODE python/chatbot-iva-agent/src/config.py:283,364-386 BLAME config.py:283 → y.fallah · 2026-08-24 · 09b74a446 refactor(chatbot-iva-agent): remove whitelist agent variants (v0.32.0) BLAME config.py:364 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)	انتخاب Agent و مدل‌ها
aiva	`AGENT_ENABLED_TOOLS` CODE python/aiva/aiva/chat/config.py:58 BLAME config.py:58 → Hamed Mahmoudkhani · 2026-06-21 · 5b31a30fd refactor(AIVA): changed tools structure, feat: tools toggle env	فهرست Toolهای فعال
	`*_LLM_MODEL`، `*_LLM_BASE_URL` CODE python/aiva/aiva/chat/config.py:37,111,119,160-165,174,184 BLAME config.py:37 → Hamed Mahmoudkhani · 2026-07-25 · 7cf4b2f4a refactor(aiva): use adcs retriever BLAME config.py:111 → Hamed Mahmoudkhani · 2026-07-25 · 7cf4b2f4a refactor(aiva): use adcs retriever BLAME config.py:119 → Hamed Mahmoudkhani · 2026-06-04 · bdad67a6e feat(cb-iva-demo): intent detection agent	مدل هر نقش (۷ نقش)

سرویس	Env کلیدی	اثر
	`VOICE_BRIDGE_ENABLED`، `*_FILTER_LLM_ENABLED`، `WAIT_FUNCTION_CALL_ENABLED` **CODE** python/aiva/aiva/chat/config.py:82,91,156,160 **BLAME** config.py:82 → a.salimi · 2026-08-16 · a45b4756d feat(aiva): add llm tracing **BLAME** config.py:91 → a.salimi · 2026-08-16 · a45b4756d feat(aiva): add llm tracing **BLAME** config.py:156 → a.salimi · 2026-07-20 · c6b80ffa7 feat(aiva): add web search functionality	Featureهای خاموش
	`CONTEXT_RETRIEVE_THRESHOLD`، `*_RETRIEVE_COUNT` **CODE** python/aiva/aiva/chat/config.py:74-79 **BLAME** config.py:74 → Hamed Mahmoudkhani · 2026-07-25 · 7cf4b2f4a refactor(aiva): use adcs retriever	دقت/پوشش RAG (Retrieval-Augmented Generation)
	`CHAT_HISTORY_LIMIT`، `REACT_MAX_ITERS` **CODE** python/aiva/aiva/chat/config.py:43,54 **BLAME** config.py:43 → mr.ahmadi · 2026-09-13 · b8704dd54 feat(aiva): add tools and update prompt **BLAME** config.py:54 → Mohammad Ghazanfari · 2026-06-01 · 665aa2291 feat: structure	طول Context و حلقه Tool
	`PHONE_WHITELIST`، `BYPASS_WHITELIST` **CODE** python/aiva/aiva/memory/config.py:20-21 **BLAME** config.py:20 → Hamed Mahmoudkhani · 2026-06-02 · dfa251cd8 feat(cb-iva-demo): added debounced summary generation using kafka m...	ساخت Memory
coa-api	`ARCHIVE_CALLS_MONTHS` (پیش‌فرض ۴) **CODE** python/coa-api/coa/config.py:909 **BLAME** config.py:909 → a.abolghasemi · 2026-04-28 · cd37cda37 feat(coa-api): adding command to archive old calls v4.11.0	Retention

سرویس	Env کلیدی	اثر
	بازه ساعتی Canonical Sync، `TRENDS_*` `CODE` python/coa-api/coa/apps/main/services/canonical_sync.py:70 `BLAME` canonical_sync.py:70 → a.rahimi · 2026-06-09 · 72d37d08f feat(coa-api): Add canonical sync for category and subcategory v4.33.0	زمان‌بندی Sync
adcs-auth	`ACCESS_TOKEN_MINUTES_LIFETIME` (۱۵)، `REFRESH_TOKEN_DAYS_LIFETIME` (۷) `CODE` python/adcs-auth/adcs_auth/config.py:15-19 `BLAME` config.py:15 → Hamed Mahmoudkhani · 2026-06-01 · 869e4a731 fix(adcs-auth): missing migrations, common settings, installed apps	عمر Token
aqa	`CLICKHOUSE_URL`، `MAX_ROWS_THRESHOLD`، `PHOENIX_*` `CODE` python/aqa/aqa/config/database.py:2 · python/aqa/aqa/agents/tools/sql_executor.py:34 `BLAME` database.py:2 → Hamed Mahmoudkhani · 2026-05-18 · c924b4b6a feat(aqa): base agent `BLAME` sql_executor.py:34 → Hamed Mahmoudkhani · 2026-05-18 · c924b4b6a feat(aqa): base agent	منبع داده، سقف ردیف، Tracing

۲۱. CI (Continuous Integration)/CD، Build و Deployment

- **Image:** هر سرویس Dockerfile دارد با Base `astral-sh/uv:python3.13-bookworm-slim` از Registry داخلی `docker.mci.dev`.

`CODE` python/coa-api/Dockerfile:1 · python/chatbot-iva-agent/Dockerfile:1
`BLAME` Dockerfile:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0
`BLAME` Dockerfile:1 → y.fallah · 2025-06-02 · da3ce5abe feat(chatbot-iva): add chatbot-iva

- **Process-per-container:** `coa-api` یک Image و چند Entrypoint دارد: `entrypoint.sh` (Migrate، Collectstatic، ساخت Superuser، Gunicorn)، `entrypoint_celery.sh`، `entrypoint_sync.sh`، `entrypoint_clickhouse_update_sync.sh`، `entrypoint_failed_calls_sync.sh`، `entrypoint_canonical_sync.sh`، `entrypoint_trends_sync.sh`، `entrypoint_archive.sh`.

`CODE` python/coa-api/entrypoint*.sh

- **CI (Continuous Integration):** Stageها `lint → build-lib → build → deploy → test`؛ کتابخانه‌ها (`agentlib`، `fastlib`) در Artifactory داخلی منتشر می‌شوند؛ Commit حاوی `/rc` نسخه Release Candidate می‌سازد و Merge به `main` نسخه نهایی.

CODE .gitlab-ci.yml:11-16 DOC python/fastlib/README.md:89 (/rc)
BLAME .gitlab-ci.yml:11 → a.salimi · 2023-04-25 · 44f6885a4 Page single browser

- **Versioning:** SemVer (Semantic Versioning) دستی در `pyproject.toml` و CHANGELOG هر پروژه؛ پیام Commit با الگوی `feat(scope): … (vX.Y.Z)`.

CODE python/*/CHANGELOG.md · python/*/pyproject.toml GIT commit subjects e.g. 'feat(websocall): lock on writes (v0.2.0)'

- **Runtime:** Helm Chart مشترک `helm/cloud` روی k3s/Kubernetes؛ Ansible برای Node، Envoy Edge/Relay، Monitoring Stack)، Keepalived/MetalLB (Virtual IP (Internet Protocol)).

CODE helm/cloud/templates/ · ansible-roles/{k3s,envoy_edge,envoy_relay,keepalived,metallb,monitoring_stack} INFERENCE keepalived/metallb provide the Virtual IP on the IVA board
LAST helm/cloud/templates/ → h.mahmoudkhani · 2026-08-16 · 7db6eb9ee fix(helm): empty checksums for deployments without configmap
LAST ansible-roles/k3s/ → Hamed Mahmoudkhani · 2024-06-10 · 4cba5173d Fixed: gitlab-ci
LAST ansible-roles/envoy_edge/ → mo.rouzban · 2025-04-15 · 31f76e30f update docker compose (envoy)
LAST ansible-roles/envoy_relay/ → mo.rouzban · 2025-04-15 · 31f76e30f update docker compose (envoy)
LAST ansible-roles/keepalived/ → a.salimi · 2023-11-25 · 43cb6ea89 k8s roles refactor
LAST ansible-roles/metallb/ → j.jalili · 2023-12-19 · 56ba69e7e metallb
LAST ansible-roles/monitoring_stack/ → a.ghasemi · 2023-12-23 · 96b6c46ab Update 2 files

- **Graceful Deploy:** Voice Agent با `AGENT_DRAIN_TIMEOUT=1800` تماس‌های فعال را قبل از خاموشی تمام می‌کند.

CODE python/chatbot-iva-agent/src/config.py:51
BLAME config.py:51 → y.fallah · 2026-08-02 · e8db10180 feat(chatbot-iva-agent): add signaling (v0.29.0)

۲۲. Developer Cookbook: کجا را عوض کنم؟

می‌خواهم...	کجا
متن Greeting / Privacy Notice را عوض کنم	Env `AGENT_GREETING_MESSAGE` (بدون Code)
یک Tool جدید اضافه کنم	فایل جدید در `aiva/tools/` با `@function_tool` و `@track_tool_call` → ثبت در `registry.py` → نام در `AGENT_ENABLED_TOOLS` ؛ برای MCP (Model Context Protocol) نسخه Go در `dobby`
Toolها را برای Pilot محدود کنم	Env `AGENT_ENABLED_TOOLS=["send_sms"]`
مدل یک نقش را عوض کنم	Env `<ROLE>_LLM_MODEL` و `<ROLE>_LLM_BASE_URL`
Prompt یک نقش را عوض کنم	`aiva/prompts/<role>.py` یا Env `<ROLE>_LLM_TEMPLATE` ؛ در AQA (Agentic Query Analyzer) از `/api/v1/configs/prompts`
مثال Guardrail/Classification اضافه کنم	Panel `/kb/guardrail` یا ابزار `scripts.classification_store`
سقف Follow-up (CON-06) بسازم	شمارنده در `ChatContext` + شرط در `IVAChatOrchestrator` پیش از انتخاب `clarifying`
Guard تماس کوتاه (VIA-07)	در `grogu` یا ورودی `coa-ai` : اگر `session_duration` یا تعداد پیام کاربر زیر آستانه بود، Summary نساز و Status را `incomplete` بگذار
IDOR را ببندم (PRV-03)	`coa/apps/files/views.py` : Permission مخصوص Admin + تطبیق `name` با Callهای مجاز کاربر (همان منطق `view_all_calls`)
Audit Log بسازم (PRV-06)	مدل `AccessAudit(user, action, call_id, at)` در `coa-api` + نوشتن در Viewهای Files و Call Detail
Log هدرها را امن کنم	`coa/middlewares.py` : حذف `authorization` ، `cookie` ، `x-api-key` پیش از Log
Memory را برای EX (Employee Experience) خاموش کنم	Consumer Memory را Deploy نکنید، یا `BYPASS_WHITELIST=false` با `PHONE_WHITELIST=[]`
Retention را عوض کنم	Env `ARCHIVE_CALLS_MONTHS` ؛ تست با `--dry-run`
بعد تحلیلی جدید اضافه کنم	Prompt در `coa-ai` → ستون ClickHouse → فیلد `CallAITag` + Migration → Mapping در `services/sync.py` → Serializer/View
Endpoint Dashboard جدید	View در App مربوط + `@with_permissions` + ثبت در `urls.py` + `@extend_schema` برای Swagger
یک Branch جدید در AIVA (Agentic Intelligent Virtual Assistant)	Handler با `CancellableBranchHandler` + Event + نتیجه + قانون Precedence در `IVAChatOrchestrator`

INFERENCE recommended change locations; each file named in the row exists at the given path

اجرای محلی

```
# Django services (coa-api, aiva, adcs, kb, auth)
uv sync && cp .env.example .env
python manage.py migrate && python manage.py runserver      # Swagger at /docs

# aqa
uv sync && alembic upgrade head && python -m aqa.main

# voice agent (needs LiveKit server)
pip install -r requirements.txt && python3 -m src.main console

# tests
DOC python/chatbot-iva-agent/README.md (console mode)  CODE test counts: find python/<p> -name 'test_*.py' -o -name '*_test.py' | wc -l  INFERENCE other commands are standard uv/Django/Alembic usage — not taken from project docs

pytest            # agentlib 140, coa-api 45, adcs 39, aiva 23 test files
```

۲۳. کیفیت Code و Technical Debt

مورد	توضیح
قوت مستندسازی معماری	`aiva/docs/RequestFlow.md`، Docstringهای دقیق (مثل `TrendAnnotation`)
قوت Resilience	Cursor، DLQ (Dead Letter Queue)، Circuit Breaker ماندگار، Abort-on-failure در Archive
قوت Tooling	`uv`، `ruff`، `pyright`، CHANGELOG، SemVer (Semantic Versioning)
ریسک تست	صفر تست: `adcs-auth`، `chatbot-iva-agent`؛ AQA (Agentic Query Analyzer) فقط ۴ فایل
ریسک Eval	هیچ سنجش کیفیت AI (Artificial Intelligence) برای AIVA (Agentic Intelligent Virtual Assistant) و CCA (Call Center Analyze)
بدهی دوگانگی	`adcs` و `aiva` هر دو «Agentlib Django Customer Support» هستند و Code مشابه دارند
بدهی Dead Code	Viewهای Login قدیمی در `coa/apps/users`
بدهی Feature Flag	TODO صریح برای Flag روی Classification و Clarifying
بدهی ظرفیت	Celery با `concurrency=1`
بدهی نام‌گذاری	Consumer Group با پسوند `demo` در مسیر Production
بدهی Schema مشترک	`grogu` (Go) و `coa-api` (Django) روی یک Schema می‌نویسند؛ Migration فقط در یکی
ریسک فریز	`coa-ai` از 13-07-2025

DOC python/aiva/docs/RequestFlow.md · python/{adcs,aiva}/README.md:1 (same title) CODE python/coa-api/coa/apps/trends/models.py:122 · python/aiva/aiva/chat/config.py:170 (TODO) · python/coa-api/coa/apps/users/urls.py:1-8 · python/coa-api/entrypoint_celery.sh:2 · python/aiva/aiva/memory/config.py:18 · golang/grogu/internal/call/model.go GIT 752f89333

BLAME models.py:122 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0

BLAME config.py:170 → a.salimi · 2026-09-28 · 638179e33 feat(aiva): add voice bridge, classification and clarification

BLAME urls.py:1 → a.rahimi · 2026-07-29 · 595027efc feat(coa-api): using adcs as authentication of coa-api v5.0.0

BLAME entrypoint_celery.sh:2 → a.behnamnia · 2024-11-11 · 1d7998164 COA v0.2 AI module (v0.2.2)

BLAME config.py:18 → Hamed Mahmoudkhani · 2026-06-02 · dfa251cd8 feat(cb-iva-demo): added debounced summary generation using kafka m...

ADDED model.go → a.salimi · 2026-01-06 · 1af1177c2 feat(grogu): rewrite coa worker in golang (v0.1.0)

۲۴. پیوست — Endpoint Catalog

coa-api (`/api/v1`)

Path	کاربرد
`/analyse/voice/{submit,status,report,tag,enums}`	Voice تحلیل و Human Tag
`/analyse/voice/call/<call_id>`	Call Detail
`/analyse/calls/recordings` (x-api-key)	Ingest Call Record
`/analyse/calls/recordings/<id>/reasons`، `/calls/tasks/status`	Reasons، وضعیت Task
`/analyse/calls/Thematic-analysis/{category,subcategory}[/…/timeseries]`	Theme و Trend
`/analyse/calls/special-analysis/timeseries[/report]`	Special Analysis
`/analyse/calls/{category,subcategory,topics}/enums`	Taxonomy
`/analyse/dashboard/{calls/report,expert}`	Dashboard
`/iva/dashboard/{metrics,satisfaction-timeseries,time-buckets}`	Dashboard IVA (Intelligent Virtual Assistant)
`/iva/announcement/{generate-voice,ai-reformatting,voices,voices/usernames,voices/<uuid>[/audio]}`	Announcement
`/trends/`، `/categories/`، `/report/`، `/subjects/<id>/[annotations/]`	Trends
`/tickets`، `/<uuid>`، `/overview`، `/report`، `/trends`، `/hour-distribution`، `/counts/{provinces,categories,subcategories,tags}`، `/problem-category-distribution`، `/search-fields`	Tickets
`/management/experts/`، `/<id>/comments`	Experts
`/coa/users[/…]`، `/management/users[/…]`، `/coa/users/change_password`	User Facade
`/files?name=`	Audio Stream (IDOR)
`/healthz`، `/docs`، `/api/schema`، `/metrics`	Ops

CODE python/coa-api/coa/urls.py · python/coa-api/coa/api.py · python/coa-api/coa/apps/{main,iva,trends,tickets,experts,management,users,files}/urls.py

ADDED urls.py → p.safaei · 2024-06-08 · 8b78eefd3 [chatbot-django/Library] Chatbot_lib django to main

ADDED api.py → a.behnamnia · 2024-11-11 · 1d7998164 COA v0.2 AI module (v0.2.2)

ADDED urls.py → h.mahmoudkhani · 2024-10-06 · 6b76c24f0 Sentiment Analysis: v1 (v0.1.0)
ADDED urls.py → a.abolghasemi · 2026-06-02 · 0c4554459 feat(coa-api): adding iva dashboars apis v4.32.0
ADDED urls.py → a.abolghasemi · 2026-04-20 · 271d368c9 feat(coa-api): adding trends api and syncer v4.5.0
ADDED urls.py → a.abolghasemi · 2026-07-01 · 07129f936 feat(coa-api) adding ticket upload file api v4.40.0
ADDED urls.py → h.samsami · 2025-01-19 · 8c931ff2f add Expert APIs (v1.6.0)
ADDED urls.py → h.mahmoudkhani · 2024-10-26 · 92e8c8243 Sentiment Analysis (v0.2.0)
ADDED urls.py → h.mahmoudkhani · 2024-10-26 · 92e8c8243 Sentiment Analysis (v0.2.0)
ADDED urls.py → h.mahmoudkhani · 2024-11-13 · 0dddb8e5e feat: file stream (v0.3.0)

سایر سرویس‌ها

Path	کاربرد
`aiva: /api/v1/chat/{session,completions}`	Chat (SSE (Server-Sent Events))
`adcs: /api/v1/chat/{asr,tts,init,session,session/feedback,message,message/feedback,signal,completions,history}`	Chat Platform
`adcs: /api/v1/dashboard/chat/{sessions,stats,messages/distribution,user-performance,chatbot-performance,sessions/source/distribution,sessions/source/stats,reports/<source>,prompts/rate,messages/<id>/feedback-admin,messages/feedback-admin/tags}`	Chat Dashboard
`kb: /api/v1/kb/{qa,list_qa,question,answer,guardrail,tag,summarize,import,export}`	Knowledge Base
`auth: /api/v1/token/{login,logout,refresh}` ، `/permissions` ، `/roles` ، `/users[/me]`	Auth
`aqa: /api/v1/chat/{session,sessions,sessions/detail,message}`	Chat AQA (Agentic Query Analyzer)
`aqa: /api/v1/configs/{prompts,sql-examples,schema,data-dictionary}`	Runtime Config (GET/PUT)
`aqa: /api/v1/{eval/run,feedback,database/schema,database/data-dict,dashboard/chat/sessions}`	Eval، Feedback، Schema

CODE python/aiva/aiva/chat/api.py · python/adcs/adcs/chat/api.py · python/adcs/adcs/dashboard/api.py · python/adcs-knowledgebase/adcs_kb/knowledgebase/api.py · python/adcs-auth/adcs_auth/{api.py,jwtoken/api.py} · python/aqa/aqa/api/routers/
ADDED api.py → Hamed Mahmoudkhani · 2026-06-02 · 334867943 feat(cb-iva-demo): added stateless OpenAI compatible APIs
ADDED api.py → h.mahmoudkhani · 2026-03-30 · f871b5fe4 feat(adcs): project structure

ADDED api.py → a.abolghasemi · 2026-06-02 · d42299747 feat(adcs): adding chat dashboard API

ADDED api.py → m.ghazanfari · 2026-04-06 · a7f6ad951 feat(adcs): knowledge base init

ADDED api.py → p.kazem · 2026-05-23 · 8ba5e8f05 feat(adcs-auth): create project structure

ADDED api.py → p.kazem · 2026-05-23 · 8ba5e8f05 feat(adcs-auth): create project structure

LAST python/aqa/aqa/api/routers/ → Mohammad Ghazanfari · 2026-09-27 · 9bb51aede feat(aqa): new routes added to session

۲۶. به‌روزرسانی کد: تغییرات ۲۸ شهریور تا ۱۴ مهر ۱۴۰۵ (۲۰۲۶-۰۹-۲۸ تا ۲۰۲۶-۱۰-۰۶)

این فصل ۱۴ Commit جدید `main` را بعد از تاریخ بازبینی اصلی سند ثبت می‌کند. منبع هر ردیف GIT (Hash کوتاه) و CODE (مسیر نسبت به ریشه Repo) است. مطالب فصل‌های قبلی (مثلاً §14 و §15) تا بازنویسی کامل معتبرند، به‌جز مواردی که اینجا صریحاً تغییر کرده‌اند.

سرویس	تغییر	منبع
aiva-realtime جدید	سرویس FastAPI جدید با WebSocket روی `/realtime` و پروتکل «Realtime» سازگار با OpenAI. دو Session دارد: `RealtimeEchoSession` و `RealtimeVoiceSession` (با `model=aiva-realtime-voice`). Voice Session یک **Cascaded** Pipeline است: ASR → LLM → TTS؛ هر سه مرحله Endpoint مستقل OpenAI-compatible دارند (`REALTIME_ASR/LLM/TTS__BASE_URL، API_KEY، PROXY`) و می‌توانند به vLLM، whisper.cpp یا LiteLLM وصل شوند. TTS با `REALTIME_TTS_CHUNKING_POLICY` (پیش‌فرض `adaptive_fast_first`: ۴ کلمه برای اولین Chunk) برای کاهش time-to-first-audio تکه‌تکه می‌شود. Middleware لاگ، Metrics و Tracing و Health دارد.	9e5f318bd `python/aiva-realtime/aiva_realtime/config/realtime.py`
aiva-realtime — VAD	VAD از یک فایل به بسته `realtime/vad/` تبدیل شد: `energy` (RMS)، `http` (Silero راه‌دور)، `grpc` (Silero راه‌دور، با `rust/vad`) و `windowed`. حالت پیش‌فرض `REALTIME_VAD_MODE=remote-silero-grpc`؛ پنجره ۱۰۰ms (حداقل ۳۲ms برای gRPC)، Timeout ۱s و **Fallback به RMS** (`REALTIME_VAD_FALLBACK_ENABLED=true`). پارامترها: Padding ۲۰۰ms قبل از شروع گفتار، تأخیر تشخیص پایان ۵۰۰ms، حداقل سکوت برای پایان Turn برابر ۵۰۰ms، و Utterance بلند ≥۱۰۰۰ms که ASR را زودتر شروع می‌کند (نتیجه تا پایان Turn نگه داشته می‌شود).	0176b7888 `realtime/vad/grpc.py`
aiva-realtime — Turn Detection	بعد از حداقل سکوت، یک **LLM مستقل بررسی تکمیل جمله** اجرا می‌شود (`REALTIME_TURN_CHECK_*`): Transcript تجمعی را به‌عنوان «محتوا نه دستور» می‌دهد و فقط yes/no می‌گیرد؛ پیش‌فرض‌ها: مدل `gpt-4o-mini`، Timeout ۲s، حداکثر ۶۴ Token، بدون Retry. اگر جمله ناتمام باشد، ۱۰۰۰ms انتظار اضافه (`EXTRA_WAIT_MS`) می‌دهد. این جایگزین Endpointing ثابت در سطح VAD است و مشکل قطع‌شدن میان‌جمله را هدف می‌گیرد. Session حدود ۲۱۷ خط تغییر کرد (Telemetry و State مرتبط).	537415cb8 `realtime/voice/turn.py`
aiva — Voice Bridge	Branch جدید `voice_bridge`: یک جمله کوتاه و متناسب با زمینه، **فوراً قبل از پاسخ اصلی** گفته می‌شود تا سکوت تماس پر شود. با `VOICE_BRIDGE_ENABLED` (پیش‌فرض **false**)، LLM مستقل (`openai/gpt-oss-120b`، Temperature 0.5، حداکثر ۵۰ Token، ۱۰ پیام History) و Prompt قابل‌مدیریت (`aiva_voice_bridge_llm_template`). نتیجه با `VoiceBridgeResultEvent` (فیلد `failed`) منتشر می‌شود.	638179e33 `aiva/chat/handlers/voice_bridge.py`

سرویس	تغییر	منبع
aiva — Classification و Clarifying	Orchestrator (≈۳۰۰ خط تغییر) حالا مسیر پاسخ را **Speculative** انتخاب می‌کند: `ClassificationRetrieverEventHandler` مثال‌های برچسب‌خورده را از `aiva-classification` (Collection Qdrant) می‌آورد و Top-10، Classifier یک `AnswerRoute` برمی‌گرداند؛ Branch `clarifying` (Clarifying و Chit-chat) به‌صورت Speculative موازی اجرا می‌شود و با `CancellableBranchHandler` قابل لغو است. `idle_message.py` و `prompts/idle.py` حذف شدند. در Config یک `TODO` برای Feature Flag جداگانه Clarifying/Classifying باقی است (فعلاً Flag ندارند).	638179e33 `handlers/classification.py`، `clarifying.py`، `branch.py`
aiva — Tool-call Retention و Phoenix Prompts	`TOOL_CALL_RESULT_REPLAY_ENABLED=true` : نتیجه Toolها در Turnهای بعدی دوباره در Context قرار می‌گیرد؛ `LATEST_CONTEXTS_PERSISTENCE_ENABLED=true` : آخرین Contextها از طریق `session_params_persistence.py` ذخیره می‌شود. Prompt Loader جدید (`prompts/loader`) Promptها را با نام و Tag (`production`) از Endpoint Phoenix می‌خواند (`USE_PHOENIX_PROMPTS=false`، `FALL_BACK_ON_PHOENIX_PROMPT_FAILURE=false`، `PHOENIX_PROMPTS_ENDPOINT`) برای ۱۱ Prompt مثل `aiva_react_agent_llm_template` و `aiva_guardrail_llm_template`. Retrieval با Qdrant بازنویسی جزئی شد.	d05a47457 `aiva/chat/config.py`
adcs — مدیریت Greeting	مدل‌های `Greeting` (یک پیام یکتا برای هر `app`، جدول `greetings`) و `GreetingLog` (جدول `greeting_logs`: app، action، message، user_id) با Migrationهای زیر `0007_greeting`. Endpoint Chat API: `GET/POST/PATCH /greeting/<app>`، `GET /greeting/<app>/logs`، `POST /greeting/<app>/suggestions` (پیشنهاد پیام با LLM؛ خطای LLM = `502`، تکراری = `409`). همه پشت Permission تازه `manage_greeting`.	b0aaa6ebd `adcs/chat/views/greeting.py`
adcs — Profile Memory	حافظه کاربر ساختاریافته شد: ۱۴ فیلد در `configs/memory_fields.yaml` (`preferred_name`، `age`، `home_location`، `occupation`، `interests`، `dietary_preferences`، `tone_preferences` و ...)، Parser جدید، Command جدید، مهاجرت `migrate_memory` و Migration `0006` روی `ChatbotUser.memory`. **نکته حریم خصوصی:** فیلدهایی مثل سن، محل زندگی، اعضای خانواده و افراد مهم داده شخصی‌اند INFERENCE — قبل از پایلوت EX با Retention و رضایت‌نامه تطبیق داده شود.	bc8f082b6 `adcs/memory/parser.py`
adcs — پشتیبانی Phoenix / Tracing	`adcs/tracing.py` جدید، پیکربندی Phoenix در `common/config.py` و Instrumentation در Viewهای `completion`، `message` و `signal`؛ `dashboard/services/performance.py` هم تغییر کرد.	16b3abe6f

سرویس	تغییر	منبع
chatbot-iva-agent v0.37–0.39	**v0.37.0 Record-only:** با `AGENT_RECORD_ONLY_ENABLED` فقط یک Greeting پخش و صدای Participant از راه Egress ضبط می‌شود؛ STT/LLM/VAD/Turn Detection خاموش‌اند (`entrypoints/record_only.py`)، و در خطای Egress پیام «متاسفانه در ضبط تماس مشکلی پیش آمده است...» پخش می‌شود. `entrypoint.py` به `entrypoints/default.py` منتقل شد. **v0.38.0:** Trace/Metric بومی LiveKit به OTLP (`utils/livekit_telemetry.py`) و بازنویسی گسترده `utils/apm.py`؛ نرخ Sampling پیش‌فرض از ۱.۰ به **۰.۱** کاهش یافت و `AGENT_APM_MIN_TRANSACTION_DURATION_MS` حذف شد. **v0.39.0:** `AGENT_RECORD_LLM_REQUEST_TIMES` (پیش‌فرض false) زمان درخواست‌های LLM را برای ضبط Participant ثبت می‌کند.	1428f59d1، 8c21540d4، 6e0bd2e24
chatbot-mci v4.20.0	پشتیبانی از Metadata نشست «وب‌سایت» (`handlers/metachat.py`) و اصلاح Handlerهای Billing/Packages.	b8fb88f7e
synapse v0.1.0 جدید	اسکلت سرویس Go (+ Gin + Swagger + Prometheus `/metrics` + Config + CI `httpx` + OpenTelemetry + Logger Zap + `/healthz` و Dockerfile). **هنوز Business Logic ندارد**؛ فقط Probeها و `/docs`. هدف محصولی آن در Code مشخص نیست.	85885ffdd `golang/synapse/internal/api/api.go`
رفع باگ‌ها	**coa-api v5.15.1:** نرمال‌سازی `service_request_type`. **grogu v0.33.1:** نرمال‌سازی شماره تلفن در `mymci/service.go`. `websocall` : اسکریپت Load Test (پیش‌تر ثبت‌شده).	b08e2e7af، 66767229b، 377b34e6c

پیامد برای Developer Cookbook (§22) و Debt (§23): (۱) `aiva-realtime` مسیر دوم Voice است که به `chatbot-iva-agent`/LiveKit وابسته نیست — ریسک دو Pipeline موازی صوتی با Turn Detection متفاوت؛ (۲) Voice Bridge، Classifier و Clarifying هر کدام یک LLM جداگانه‌اند → هزینه و Latency هر تماس در §7 باید بازمحاسبه شود؛ (۳) آزمایش: `aiva-realtime` با ~۹۰ فایل تست شروع شد و `aiva` تست‌های Loader، Tracing و Persistence گرفت، اما هنوز Eval کیفیت LLM وجود ندارد.

فهرست اختصارات کل سند

هر اختصار در متن هم در اولین استفاده در هر بخش با فرم کامل آمده است؛ این فهرست برای مراجعه سریع به همه آن‌هاست. ردیف‌هایی که فرم کامل ندارند **نام داخلی** هستند و فرم کامل‌شان در مخزن تعریف نشده است — حدس زده نشد. اختصارهای مخصوص فصل مسیر تماس (مثل PBX، SLA، BGP) در جدول ابتدای همان فصل آمده‌اند.

اختصار	فرم کامل	معنی / نقش
AQA	Agentic Query Analyzer	سرویس NL→SQL برای تحلیل‌گران (نام قدیمی: Spotter) — python/aqa/README.md
AIVA	Agentic Intelligent Virtual Assistant	مغز مکالمه (Django) — python/aiva/pyproject.toml
IVA	Intelligent Virtual Assistant	دستیار صوتی/چت‌بات تماس — INFERENCE از AIVA؛ در مخزن جدا تعریف نشده
ADCS	Agentlib Django Customer Support	پلتفرم Chat/KB — عنوان python/adcs/README.md
CCA	Call Center Analyze	Pipeline تحلیل تماس‌ها — عنوان روی جلد همین سند
COA	— (نام داخلی)	نام پروژه/برد؛ فرم کامل در مخزن تعریف نشده (coa-api، coa-ai)
TelC	— (نام داخلی)	پلتفرم Contact Center منبع تماس‌ها؛ فرم کامل در مخزن نیست
MyMCI	— (نام داخلی)	سرویس اطلاعات حساب مشتری MCI که grogu از آن غنی‌سازی می‌کند
MCINEXT	— (نام شرکت)	شرکت متولی پروژه
MCI	Mobile Telecommunication Company of Iran	همراه اول
EX	Employee Experience	پروژه Pilot «صدای کارمند» بین MCINEXT و بخش EX همراه اول
UAT	User Acceptance Testing	تست پذیرش کاربر
MVP	Minimum Viable Product	حداقل محصول قابل ارائه
SAP	Systems, Applications and Products (ERP)	سامانه منابع سازمانی/کارکنان (SEG-07)
HR	Human Resources	منابع انسانی
LLM	Large Language Model	مدل زبانی بزرگ
RAG	Retrieval-Augmented Generation	بازیابی دانش و افزودن به Prompt
MCP	Model Context Protocol	استاندارد اتصال Tool/داده به Agent
AI	Artificial Intelligence	هوش مصنوعی
NL	Natural Language	زبان طبیعی (NL→SQL)
ASR	Automatic Speech Recognition	گفتار به متن (هم‌معنی STT؛ در فصل تماس: Answer-Seizure Ratio)
STT	Speech-to-Text	گفتار به متن

اختصار	فرم کامل	معنی / نقش
TTS	Text-to-Speech	متن به گفتار
VAD	Voice Activity Detection	تشخیص وجود گفتار
WER	Word Error Rate	نرخ خطای کلمه
WAV	Waveform Audio File Format	فرمت فایل صوتی خام
SIP	Session Initiation Protocol	پروتکل Signaling تماس
RTP	Real-time Transport Protocol	حمل صدا
SDP	Session Description Protocol	مذاکره Media
SBC	Session Border Controller	دروازه SIP
PSTN	Public Switched Telephone Network	شبکه تلفن ثابت/عمومی
WebRTC	Web Real-Time Communication	استاندارد صدا/تصویر Real-time در مرورگر و LiveKit
NAT	Network Address Translation	ترجمه آدرس شبکه
DLQ	Dead Letter Queue	صف پیام‌های شکست‌خورده
SSE	Server-Sent Events	استریم یک‌طرفه سرور→کلاینت
API	Application Programming Interface	واسط برنامه‌نویسی
REST	Representational State Transfer	سبک API روی HTTP
HTTP	Hypertext Transfer Protocol	پروتکل وب
JSON	JavaScript Object Notation	فرمت داده
CSV	Comma-Separated Values	فرمت جدولی متنی
CRUD	Create, Read, Update, Delete	عملیات پایه داده
DRF	Django REST Framework	چارچوب API در coa-api
JWT	JSON Web Token	توکن احراز هویت بدون State
ID	Identifier	شناسه
DB	Database	پایگاه داده
SQL	Structured Query Language	زبان پرس‌وجوی پایگاه داده
S3	Simple Storage Service	API ذخیره‌ساز شیء (MinIO سازگار با آن)
KB	Knowledge Base	پایگاه دانش
FAQ	Frequently Asked Questions	پرسش‌های پرتکرار
SMS	Short Message Service	پیامک
UI	User Interface	رابط کاربری
TUI	Text User Interface	رابط کاربری متنی (ترمینال)

اختصار	فرم کامل	معنی / نقش
CI	Continuous Integration	ساخت/تست خودکار در GitLab
SemVer	Semantic Versioning	نسخه‌گذاری MAJOR.MINOR.PATCH
APM	Application Performance Monitoring	پایش عملکرد برنامه
TPM	Technical Product Manager	مدیر فنی محصول
HA	High Availability	در دسترس بودن بالا
IP	Internet Protocol	آدرس‌دهی شبکه
UDP	User Datagram Protocol	انتقال بدون اتصال
VLAN	Virtual Local Area Network	شبکه محلی مجازی
ACL	Access Control List	فهرست مجاز/ممنوع
QoS	Quality of Service	اولویت ترافیک
DSCP	Differentiated Services Code Point	برچسب اولویت IP
ACK	Acknowledgement	تأیید دریافت در SIP
RFC	Request for Comments	سند استاندارد اینترنت
PCMA	Pulse Code Modulation A-law	Codec G.711 A-law

واژه‌نامه

اصطلاح	معنا
IDOR — Insecure Direct Object Reference	Insecure Direct Object Reference — دسته‌ای از آسیب‌پذیری Access Control؛ سرور یک شناسه (مثل `call_id` یا نام فایل) را از کاربر می‌گیرد و بدون بررسی اینکه آن رکورد واقعاً متعلق به همان کاربر/دسترسی اوست، مستقیماً آن را واکشی می‌کند. نمونه در این سند: `GET /api/v1/files?name=<call-id>.wav` با تغییر `call-id` ، فایل صوتی تماس دیگران قابل دانلود است.
VAD — Voice Activity Detection	Voice Activity Detection — الگوریتمی (معمولاً یک مدل سبک) که به‌صورت لحظه‌ای روی جریان صوتی اجرا می‌شود و فریم‌های حاوی گفتار را از سکوت/نویز پس‌زمینه جدا می‌کند. خروجی VAD ورودی مرحله‌ی Turn Detection است و در معماری Agent صوتی (LiveKit) پیش از ASR اجرا می‌شود.
Turn Detection / Endpointing	منطقی که روی خروجی VAD (و گاه سیگنال‌های زبانی) تصمیم می‌گیرد کاربر نوبت صحبت خود را تمام کرده و اکنون نوبت پاسخ Agent است. تشخیص زودهنگام باعث قطع‌شدن وسط جمله‌ی کاربر می‌شود و تشخیص دیرهنگام باعث تأخیر محسوس در پاسخ Agent؛ تنظیم آن یک Trade-off مستقیم روی تجربه‌ی مکالمه است.
Barge-in	قابلیتی که به کاربر اجازه می‌دهد وسط صحبت Agent (TTS) حرف بزند و Agent را متوقف کند — رفتار طبیعی مکالمه‌ی انسانی. در کد این پروژه با فلگ‌هایی مثل `allow_interruptions` در `chatbot-iva-agent/src/agents/agentlib.py` و `entrypoint.py` کنترل می‌شود (مثلاً در حالت‌های خاص مثل پیام خوش‌آمدگویی یا سکوت کاربر، Interruption غیرفعال می‌شود).

اصطلاح	معنا
Egress	در LiveKit، فرایند خروج/ضبط محتوای یک Room (صدا/تصویر Real-time) به یک فایل یا Stream خارجی (مثل ذخیره در Object Storage). در این پروژه برای ضبط ترک صدای کاربر و Agent (`participant_track_path` , `agent_track_path`) استفاده می‌شود تا بعداً برای ASR آفلاین یا Audit در دسترس باشد.
RAG — Retrieval-Augmented Generation	Retrieval-Augmented Generation — الگویی که قبل از فراخوانی LLM، اسناد مرتبط از یک پایگاه دانش (Vector DB / Knowledge Base) بازیابی و به‌عنوان Context به Prompt اضافه می‌شوند، تا پاسخ مبتنی بر داده‌ی واقعی سازمان باشد نه فقط دانش ازپیش‌آموخته‌ی مدل. کاهش Hallucination و امکان به‌روزرسانی دانش بدون Fine-tune از مزایای اصلی آن است.
ReAct	الگوی طراحی Agent (Reason + Act) که در آن مدل به‌صورت تکرارشونده بین «استدلال درباره‌ی قدم بعدی» (Reason)، «اجرای یک Tool/Action» (Act) و «مشاهده‌ی نتیجه» (Observe) حرکت می‌کند تا به پاسخ نهایی برسد؛ پایه‌ی بسیاری از فریم‌ورک‌های Agentic از جمله ساختار Tool-calling در این پروژه.
SSE — Server-Sent Events	Server-Sent Events — یک پروتکل HTTP تک‌جهته (سرور به کلاینت) که روی یک Connection باز، رویدادهای متنی را پشت‌سرهم Push می‌کند؛ در این پروژه برای Stream کردن تدریجی پاسخ LLM (Token-by-token) به کلاینت Chat استفاده می‌شود، بدون نیاز به WebSocket کامل.
DLQ — Dead Letter Queue	Dead Letter Queue — یک صف جداگانه (روی Kafka/RabbitMQ) که پیام‌هایی را نگه می‌دارد که پردازش‌شان بارها Fail شده و Consumer اصلی نتوانسته آن‌ها را کامل کند؛ به‌جای گم‌شدن یا Retry بی‌نهایت پیام، امکان بررسی دستی/Replay بعدی را فراهم می‌کند.
Circuit Breaker	الگوی مقاوم‌سازی سرویس: وقتی فراخوانی یک وابستگی بیرونی (مثل سرویس ASR) پی‌درپی خطا می‌دهد، Circuit Breaker به‌طور موقت مسیر فراخوانی را «باز» (Open) می‌کند تا درخواست‌های جدید بلافاصله Fail شوند و به سرویس خراب فشار نیاورند؛ بعد از یک بازه‌ی زمانی دوباره تست می‌کند (Half-Open) و در صورت موفقیت مسیر را می‌بندد. پیاده‌سازی آن در `golang/grogu/internal/circuitbreak` و استفاده‌اش در `golang/grogu/internal/asr/service.go` است.
MCP — Model Context Protocol	Model Context Protocol — یک استاندارد باز برای اتصال مدل‌های زبانی به Tool ها/منابع داده‌ی بیرونی به‌شکل یکنواخت (سرور MCP قابلیت‌ها را اعلام می‌کند، مدل/Agent آن‌ها را فراخوانی می‌کند)، به‌جای اینکه هر Integration به‌صورت Ad-hoc و اختصاصی نوشته شود. در این پروژه `agentlib-plugin-mcp` این نقش را ایفا می‌کند.
WER — Word Error Rate	Word Error Rate — معیار استاندارد دقت ASR: نسبت مجموع کلمات جایگزین‌شده (Substitution)، حذف‌شده (Deletion) و اضافه‌شده (Insertion) در متن پیاده‌شده به تعداد کل کلمات متن مرجع. WER پایین‌تر یعنی Transcript دقیق‌تر؛ مکمل معیار `transcript_confidence` که در سطح هر Utterance در Schema این پروژه ذخیره می‌شود.
سرشماره (Service Number)	شماره کوتاه/خاصی که مشتری برای ورود به یک سرویس می‌گیرد. تخصیص آن فقط یک شناسه ورود است؛ تا Routing، اتصال، SBC و SIP Trunk آماده نشوند سرویس قابل استفاده نیست.
Zone / Core Network (شبکه مرکزی اپراتور)	بخش مرکزی شبکه اپراتور که تماس را کنترل و Route می‌کند؛ تصمیم می‌گیرد تماس به یک سرشماره به کدام Trunk/مقصد برود.
Signaling	پیام‌های کنترلی برقراری، تغییر و پایان تماس (چه کسی، به کجا، آزاد است یا نه، برقرار شد یا قطع). جدا از خود صدا (Media).
Backbone	شبکه اصلی و پرظرفیت انتقال اپراتور (فیبر + تجهیزات Core) که Zoneها و دیتاسنترها را به هم وصل می‌کند. «وصل شدن به Backbone» یعنی Connectivity + Routing + IP + Security، نه فقط کابل.

اصطلاح	معنا
VLAN / ACL / QoS — Virtual Local Area Network / Access Control List / Quality of Service	VLAN: جداسازی منطقی شبکه روی یک زیرساخت فیزیکی. ACL: فهرست مجاز/ممنوع ترافیک بر اساس IP/پورت. QoS: اولویت‌دادن به ترافیک صدا (مثلاً DSCP EF) تا Jitter و Loss کم شود.
SBC — Session Border Controller	Session Border Controller — دروازه امنیتی و کنترلی تماس‌های SIP در مرز شبکه: کنترل Session، امنیت، Topology Hiding، NAT Traversal، محدودسازی ظرفیت و Normalization پیام‌ها. معمولاً SIP Trunk اپراتور روی آن خاتمه می‌یابد.
SIP — Session Initiation Protocol	Session Initiation Protocol (RFC 3261) — پروتکل Signaling تماس صوتی روی IP با پیام‌هایی مثل OPTIONS، BYE، ACK، 200 OK، 180 Ringing، INVITE و MESSAGE. خودش صدا را حمل نمی‌کند.
SIP Trunk (Session Initiation Protocol Trunk)	ارتباط منطقی SIP بین دو سیستم (مثلاً اپراتور و SBC) با IPهای Transport، Peer، سقف کانال هم‌زمان و فرمت شماره توافق‌شده.
RTP — Real-time Transport Protocol	Real-time Transport Protocol — حمل خود صدا (Media) روی UDP، معمولاً در بسته‌های ۲۰ms. مسیر آن ممکن است با مسیر SIP متفاوت باشد.
SDP — Session Description Protocol	Session Description Protocol — بدنه پیام‌های SIP که IP، پورت و Codecهای Media را مذاکره می‌کند (Offer در INVITE، Answer در 200 OK). IP اشتباه در SDP علت رایج «صدا نمی‌آید» است.
Codec (G.711 — ITU-T standard; PCMA/PCMU)	الگوریتم کدگذاری صدا. G.711 (PCMA/PCMU) استاندارد رایج تلفن: ۸ kbps ،۶۴ kHz خالص (~۸۷ kbps با Overhead). STT در IVA با همین صدای Narrowband کار می‌کند.
Jitter / Packet Loss (نوسان تأخیر / گم‌شدن بسته)	Jitter: نوسان زمان رسیدن بسته‌های RTP؛ Packet Loss: بسته‌های گم‌شده. هر دو صدای تکه‌تکه/رباتیک و افت دقت STT ایجاد می‌کنند.
NAT Traversal — Network Address Translation Traversal	مسئله عبور SIP/RTP از NAT: آدرس داخلی که در SDP/Via نوشته شده از بیرون قابل دسترسی نیست؛ نتیجه‌اش صدای یک‌طرفه یا قطع تماس بعد از ~۳۲ ثانیه است. SBC معمولاً آن را حل می‌کند.
LiveKit SIP / Dispatch	پل LiveKit که تماس SIP را به یک Room تبدیل و Caller را با Participant `sip_+98…` Identity می‌کند؛ Dispatch Rule تعیین می‌کند کدام Agent Worker تماس را بگیرد. Headerهای SIP به‌شکل `sip.h.*` در Attributeهای Participant می‌آیند.
SIP MESSAGE (Redirect) — Session Initiation Protocol MESSAGE	روش انتقال به کارشناس در Phoenix: به‌جای REFER استاندارد، `sip-messager` یک پیام MESSAGE با `X-Resolution` و `X-Destination` (پیش‌فرض `6040`) به Hop استخراج‌شده از Via می‌فرستد و انتظار `202` دارد.
HA / Failover — High Availability	High Availability: چند Node/دیتاسنتر تا خرابی یکی سرویس را متوقف نکند. Failover: انتقال خودکار ترافیک به Node سالم.
Concurrent Calls (تماس‌های هم‌زمان)	تعداد تماس‌های هم‌زمان — واحد اصلی ظرفیت: هر تماس هم‌زمان یک SIP Session، دو جریان RTP، یک Agent Job و یک Session هم‌زمان STT/LLM/TTS مصرف می‌کند.
Fine-tuning	آموزش مجدد پارامترهای یک مدل ازپیش‌آموزش‌دیده روی دیتاست مشخص تا رفتار، لحن یا یک Task خاص را بهتر یاد بگیرد. برخلاف RAG برای دانش متغیر (قیمت، رویه) مناسب نیست. RAG و Fine-tuning می‌توانند با هم باشند: RAG دانش را می‌دهد، Fine-tuning رفتار را تنظیم می‌کند. در Phoenix کد Fine-tuning وجود ندارد.

اصطلاح	معنا
k-anonymity	یک مدل حریم‌خصوصی آماری: داده یا آمار تجمیعی فقط زمانی نمایش داده می‌شود که حداقل k رکورد/فرد پشت آن گروه باشند، تا نتوان با ترکیب فیلترها یک فرد خاص را از داخل گروه شناسایی کرد (مثلاً جلوگیری از نمایش «Sentiment منفی» برای گروهی با فقط ۲ نفر که عملاً هویت را لو می‌دهد). در این سند به‌عنوان مکانیزم Suppression گروه‌های کوچک (PRV-05) به‌کار رفته.

سند همراه: `product-management/PRODUCT_DOSSIER_FA.pdf` . بازتولید: `product/README.md` .